Templating

The templating module is mandatory. It enables you to:

- Compose prompts
- Define placeholders
- Confgure your chosen LLM

Your template generates the final query and inferences your specified LLM.

You can configure the module by passing the following information:

- Model Name (parameter `name`): The model name is required. For information about foundation models and supported models, see Foundation Models and SAP Note 3437766.
- Model Parameters (parameter `params`): The model parameters are optional. Possible values depend on the chosen model. For more information, see Harmonized API.

  For Anthropic models, however, the parameter `max_tokens` is required. If you don't set a value, the orchestration service sets this parameter to the maximum value allowed by the model. For more information, see the Anthropic documentation at Models.
- Model Version (parameter `version`): The model version is optional and defaults to "latest".
- Timeout (parameter `timeout`): is optional and is specified in seconds. Min: 1, max 600, default: 600.

  You can use timeout to configure a timeout limit for LLM calls.

  Orchestration will either retry or fail the request with a read timeout error (408) if the timeout elapses.

  For streaming requests, the timeout represents the entire duration of the streaming response, from connection establishment until the stream is either completely consumed or closed.

  This parameter is ignored by Vertex AI models.
- Max retries (parameter `max_retries`): is optional and is specified in number of retries. Min: 0, max 5, default: 2.

  You can use max_retries to configure a number of retries for LLM calls. Retries are attempted in case of errors related to connection, network and read timeouts, for example.

  For streaming requests, retries are only attempted when establishing the initial connection. Retries are not attempted after streaming responses have started.

  This parameter is ignored by Vertex AI models.

Any placeholders that you define in your prompt can be filled at runtime. For example, the following template places the input text into the `{{ ?input }}` placeholder:

```
"prompt_templating": {
         "prompt": {
           "template": [
           {
             "role": "user",
             "content": "{{?input}}"
           }
         ]
         },
         "model": {
           "name": "gpt-4o",
           "params": {
             "max_tokens": 300,
             "temperature": 2
           }
           "timeout": 10     // in seconds
           "max_retries": 3
        }
       }
```

The `{{ ?input }}` placeholder is filled using the contents of the `input_params.input` parameter:

```
"input_params": {
  "input": "A sample prompt to be sent to the model"
}
```

You can also set default values for the placeholders. For example, to request either 5 paraphrases or a user-specific number of paraphrases for a given phrase, you can use the following configuration:

```
{
  "prompt_templating": {
    "prompt": {
      "template": [
        {
          "role": "user",
          "content": "Create {{ ?number }} paraphrases of {{ ?phrase }}"
        }
      ],
      "defaults": {
        "number": 5
      }
    }
  },
  //... other configuration
```

```
    "placeholder_values": {
      "number": "3",
      "phrase": "Hello."
    }

}
```

Templates can also be retrieved from the prompt registry. For more information, see Prompt Registry.

Templates can be retrieved by ID (immutable) or the combination of scenario, name, and version (mutable). Additionally, a 'scope' parameter can be specified to define whether the requested template is shared across all resource groups within the 'tenant' (default), or is only accessible within the specific 'resource_group'. For example:

Sample Code

```
"prompt_templating": {
          "prompt": {
              "template_ref": {
                  "id": "d22342e7-1f91-4c52-a794-04833bc2574a"
                  "scope": "tenant"
         }
     }
},
```

or

```
"prompt_templating": {
          "prompt": {
              "template_ref": {
                  "scenario": "test-scenario",
                  "name": "test-template",
                  "version": "0.0.1"
                  "scope": "tenant"
              }
          }
 },
```

Input Image Support

Images can be provided as additional input where supported.

For example:

Sample Code

```
"messages_history": [
   {"role": "system", "content": "You are a helpful assistant."},
   {
       "role": "user",
       "content": [
           {"type": "text", "text": "what is in this image?"},
           {"type": "image_url", "image_url": {"url": "..."}},
           ],
       },

],
```

The URL for `image_url` supports Base64 or a web-based URL of the PDF. Check with the model provider that your preferred upload method is supported.

```
"image_url": {
    "url":  f"{base64_image}"
}
```

or

```
"image_url": {
   "url": "https://some.jpg"
}
```

Input PDF Support

PDFs can be provided as additional input for Gemini models only.

```
{
  "config": {
    "modules": {
      "prompt_templating": {
        "model": {
          "name": "gemini-2.5-flash",
```

```
            "params": {
              "temperature": 0.1
            }
          },
          "prompt": {
            "template": [
              {
                "role": "user",
                "content": [
                  {
                    "type": "image_url",
                    "image_url": {
                      "url": "data:application/pdf; base64,{{?base64_pdf}}"
                    }
                  },
                  {
                    "type": "text",
                    "text": "what does the document say?"
                  }
                ]
              }
            ]
          }
        }
      },
      "stream": {
        "enabled": false
      }
    },
    "placeholder_values": {
      "base64_pdf": "JVBERi0xLjQKMSAwIG9iago8PC9UeXBlIC9DYXRhbG9nCi9QYWdlcyAyIDAgUgo+PgplbmRvYmoKMiAwIG9iago8PC9UeXBlIC9QYWdlcwovS2lkcyBbMyA
    }
  }
```

The URL for `image_url` also a web-based URL of the PDF.

```
  "image_url": {
    "url": "https://assets.anthropic.com/m/1cd9d098ac3e6467/original/Claude-3-Model-Card-October-Addendum.pdf"
  }
```

Few-Shot Learning

Prerequisites

You have a running orchestration deployment and have retrieved your orchestration deployment URL. For more information, see Get Your Orchestration Deployment URL and Create a Deployment for Orchestration.

Process

The following example shows how you can configure the templating module to use a few-shot learning prompt.

```
curl --request POST $ORCH_DEPLOYMENT_URL/v2/completion \
    --header 'content-type: application/json' \
    --header "Authorization: Bearer $TOKEN" \
    --header "AI-Resource-Group: $RESOURCE_GROUP" \
    --data-raw '{
    "config": {
      "modules": {
        "prompt_templating": {
          "prompt": {
                "template": [
                  {
                    "role": "system",
                    "content": "You classify input text into the two following categories: Business, Economics, Tech, and other"
                  },
                  {
                    "role": "user",
                    "content": "input text: `Comcast launches prepaid plans`"
                  },
                  {
                    "role": "assistant",
                    "content": "Business"
                  },
                  {
                    "role": "user",
                    "content": "input text: `Slower Fed Pivot Weakens Rate-Cut Bets Across Emerging Asia`"
                  },
                  {
                    "role": "assistant",
                    "content": "Economics"
                  },
                  {
                    "role": "user",
                    "content": "input text: {{?input}}"
                  }
                ]
          },
          "model": {
            "name": "gpt-4o",
            "params": {
              "max_tokens": 300,
              "temperature": 2
            }
```

```
          }
        }
      }
    },
    "placeholder_values": {
        "input": "Scaling up neural models has yielded significant advancements in a wide array of tasks, particularly in language generat

    }
  }
'
```

In this case, the template contains an array of messages, including a system message, as well as several user and assistant messages. The actual input to categories is configured with the final user message and the content `input text: {{?input}}`. The input parameter input is set to `Scaling up neural models has yielded significant advancements in a wide array of tasks, particularly in language generation.`.

Related Information

Leveraging Orchestration Capabilities to Enhance Responses
Libraries and SDKs

Chat

Orchestration can also be used in chat scenarios. The following example shows how to configure the templating module to use a chat prompt.

Prerequisites

You have a running orchestration deployment and have retrieved your orchestration deployment URL. For more information, see Get Your Orchestration Deployment URL and Create a Deployment for Orchestration.

Process

```
curl --request POST $ORCH_DEPLOYMENT_URL/v2/completion \
    --header 'content-type: application/json' \
    --header "Authorization: Bearer $TOKEN" \
    --header "AI-Resource-Group: $RESOURCE_GROUP" \
    --data-raw '{
    "config": {
      "modules": {
        "prompt_templating": {
          "prompt": {
            "template": [
              {
                "role": "user",
                "content": "And now two more that sound like Goethe."
              }
            ]
          },
        "model": {
          "name": "gpt-4o-mini",
          "params": {
            "max_tokens": 300,
            "temperature": 0.1,
            "frequency_penalty": 0,
            "presence_penalty": 0
          }
        }
        }
      }
    },
    "placeholder_values": {
    },
    "messages_history": [
      {
        "role":"user",
        "content":"Create 3 paraphrases of I love you."
      },
      {
        "role":"assistant",
        "content":"1. I have deep affection for you.\n2. My feelings for you are strong and caring.\n3. You mean everything to me."
      },
      {
        "role":"user",
        "content":"Can you please make them more romantic."
      },
      {
        "role":"assistant",
        "content":"1. My heart is filled with an intense passion for you.\n2. You are the light of my life, and my love for you knows no b
      },
      {
        "role":"user",
        "content":"Try sounding like shakespeare."
      },
      {
        "role":"assistant",
        "content":"1. Thou art the sun in my sky, and my love for thee doth burn with unyielding fervor.\n2. Within mine heart, thou hast
      }
    ]
  }'
```

As shown above, messages from the chat can be passed to the current request using the `messages_history` parameter. In this case, the chat history contains several user messages and assistant responses. The templating module is then configured with the current user message, which is used to generate the next assistant response. The templating module appends the current user message to the message history to generate the prompt that is sent to the LLM module.

The response contains the messages from the chat history and the response to the new message. The output of `module_results.templating` and the `orchestration_result.choices` results can be used as the message history for a subsequent inference request:


```
{
  "request_id": "9a66158b-b019-951a-a5e7-3c3ec9d2662e",
  "intermediate_results": {
    "templating": [
      {
        "content": "Create 3 paraphrases of I love you.",
        "role": "user"
      },
      {
        "role": "assistant",
        "content": "1. I have deep affection for you.\n2. My feelings for you are strong and caring.\n3. You mean everything to me."
      },
      {
        "content": "Can you please make them more romantic.",
        "role": "user"
      },
      {
        "role": "assistant",
        "content": "1. My heart is filled with an intense passion for you.\n2. You are the light of my life, and my love for you knows no
      },
      {
        "content": "Try sounding like shakespeare.",
        "role": "user"
      },
      {
        "role": "assistant",
        "content": "1. Thou art the sun in my sky, and my love for thee doth burn with unyielding fervor.\n2. Within mine heart, thou hast
      },
      {
        "content": "And now two more that sound like Goethe.",
        "role": "user"
      }
    ],
    "llm": {
      "id": "chatcmpl-BuHIHDDcx0FPsOMVEXYp8eVKA4ulw",
      "object": "chat.completion",
      "created": 1752752545,
      "model": "gpt-4o-mini-2024-07-18",
      "system_fingerprint": "fp_efad92c60b",
      "choices": [
        {
          "index": 0,
          "message": {
            "role": "assistant",
            "content": "1. In the depths of my soul, I find an eternal longing for thee, a love that transcends the bounds of time and spa
          },
          "finish_reason": "stop"
        }
      ],
      "usage": {
        "completion_tokens": 55,
        "prompt_tokens": 212,
        "total_tokens": 267
      }
    }
  },
  "final_result": {
    "id": "chatcmpl-BuHIHDDcx0FPsOMVEXYp8eVKA4ulw",
    "object": "chat.completion",
    "created": 1752752545,
    "model": "gpt-4o-mini-2024-07-18",
    "system_fingerprint": "fp_efad92c60b",
    "choices": [
      {
        "index": 0,
        "message": {
          "role": "assistant",
          "content": "1. In the depths of my soul, I find an eternal longing for thee, a love that transcends the bounds of time and space
        },
        "finish_reason": "stop"
      }
    ],
    "usage": {
      "completion_tokens": 55,
      "prompt_tokens": 212,
      "total_tokens": 267
    }
  }
}
```


Related Information

Leveraging Orchestration Capabilities to Enhance Responses
Libraries and SDKs

Minimal Call

Prerequisites

You have a running orchestration deployment and have retrieved your orchestration deployment URL. For more information, see Get Your Orchestration Deployment URL and Create a Deployment for Orchestration.

Process

A minimal call to orchestration contains only configurations of the required templating and model configuration modules. The curl command below shows how to make such a request.

```
curl --request POST $ORCH_DEPLOYMENT_URL/v2/completion \
    --header 'content-type: application/json' \
    --header "Authorization: Bearer $TOKEN" \
    --header "AI-Resource-Group: $RESOURCE_GROUP" \
    --data-raw '{
    "config": {
      "modules": {
        "prompt_templating": {
          "prompt": {
            "template": [
            {
              "role": "user",
              "content": "Reply with `{{?text}}` in {{?language}}"
            }
          ]
          },
          "model": {
            "name": "gpt-4o-mini",
            "params": {
              "max_tokens": 300,
              "temperature": 2
            }
          }
        }
      }
    },
    "placeholder_values": {
      "text": "Hello",
      "language": "German"
    }
}
'
```

This request configures the templating module with a single user message with two parameters: `text` and `language`. The `language` parameter is also configured with English as the default. The LLM module is configured to use gpt-4o-mini in the latest available version and a set of model parameters. The `placeholder_values` field contains the values for the parameters `text` and `language`. These values are used during this request in the prompt sent to the model.

The response contains a `request_id`, the module results from each module that was executed, and the `orchestration_result`, which includes the response of the call to the model.

Output Code


```
{
  "request_id": "befbe737-5e2c-96fd-a30b-aa18c500a316",
  "intermediate_results": {
    "templating": [
      {
        "content": "Reply with `Hello` in German",
        "role": "user"
      }
    ],
    "llm": {
      "id": "chatcmpl-BuHKkcuKYRrriwYdOYTzAol15mwya",
      "object": "chat.completion",
      "created": 1752752698,
      "model": "gpt-4o-mini-2024-07-18",
      "system_fingerprint": "fp_efad92c60b",
      "choices": [
        {
          "index": 0,
          "message": {
            "role": "assistant",
            "content": "Hallo!"
          },
          "finish_reason": "stop"
        }
      ],
      "usage": {
        "completion_tokens": 3,
        "prompt_tokens": 14,
        "total_tokens": 17
      }
    }
  },
  "final_result": {
    "id": "chatcmpl-BuHKkcuKYRrriwYdOYTzAol15mwya",
    "object": "chat.completion",
    "created": 1752752698,
    "model": "gpt-4o-mini-2024-07-18",
    "system_fingerprint": "fp_efad92c60b",
    "choices": [
      {
        "index": 0,
        "message": {
          "role": "assistant",
```

```
            "content": "Hallo!"
          },
          "finish_reason": "stop"
        }
      ],
      "usage": {
        "completion_tokens": 3,
        "prompt_tokens": 14,
        "total_tokens": 17
      }
    }
  }
```

The templating module result contains the user message with the filled in parameters. The LLM module result contains the response of the model execution. In this example, the LLM module result and the orchestration result are the same. However, they might differ, such as when the output filtering module filters the response.

Related Information

Leveraging Orchestration Capabilities to Enhance Responses
Libraries and SDKs

Tool Calling

You can incorporate specialist tools into your orchestration workflow for example to make an API call for current data or to execute custom code. LLMs decide that a tool call is needed based on your system prompt, messages and tool definition. You then execute your function code, return the results and the model will incorporate the results into its response.

You can define multiple tools, and use the decision making capabilities of generative AI models to guide you in their use.

Example

The following code shows a tool containing a custom function for an API call to retrieve item quantity information from an inventory:

Sample Code

```
import requests

def get_inventory_quantity(product_id):
        response = requests.get(f"https://my-store-api.com/v1/inventory?product_id={product_id}")
        data = response.json()
        return data['product_id']['quantity']
```

You can include tools in your template by providing and outline of the function and its arguments in your template. You can include guidance on when a tool should be used in your system messages.

The following call includes an outline of the get_inventory_quantity function above in the `tools` parameter, with a description of its use case in the system role content. The `placeholder_values` include a request for product ID 123456.

Sample Code

```
curl --request POST \
  --url {{orchestration_deployment_url}}/v2/completion \
  --header 'AI-Resource-Group: default' \
  --header 'Authorization: Bearer {{token}}' \
  --header 'content-type: application/json' \
  --data '{
  "config": {
    "modules": {
      "prompt_templating": {
        "prompt": {
          "template": [
          {
            "role": "system",
            "content": "You are a helpful assistant. You help users to analyze inventory of the user's store by calling the get_invento
          },
          {
            "role": "user",
            "content": "How many units of the product with ID {{ ?product_id }} are in stock?"
          }
        ],
        "tools": [
          {
            "type": "function",
            "function": {
              "name": "get_inventory_quantity",
              "description": "Check available quantity for a product ID.",
              "strict": true,
              "parameters": {
                "type": "object",
                "properties": {
                  "product_id": {
                    "type": "integer"
                  }
                },
                "required": [
                  "product_id"
                ],
```

```
                "additionalProperties": false
              }
            }
          }
        ]
        },
        "model": {
          "name": "gpt-4o",
          "params": {
            "max_tokens": 200
          }
        }
      }
    }
  },
  "placeholder_values": {
    "product_id": "123456"
  }
}'
```

The model uses the tool definition to decide that a function call is needed, and returns the function name and its input arguments in the format defined in the template. Use the information provided to call your custom function.

Output Code

```
{
  "request_id": "c3a3aeb5-5044-9152-8c3b-cf2fa4c07b25",
  "intermediate_results": {
    "templating": [
      {
        "role": "system",
        "content": "You are a helpful assistant. You help users to analyze inventory of the user's store by calling the get_inventory_qu
      },
      {
        "content": "How many units of the product with ID 123456 are in stock?",
        "role": "user"
      }
    ],
    "llm": {
      "id": "chatcmpl-BuZqH3BWFuVVuge7Hg73ecLutnTje",
      "object": "chat.completion",
      "created": 1752823845,
      "model": "gpt-4o-2024-08-06",
      "system_fingerprint": "fp_ee1d74bde0",
      "choices": [
        {
          "index": 0,
          "message": {
            "role": "assistant",
            "content": "",
            "tool_calls": [
              {
                "id": "call_9yP43Z8VwoEQoB8mCdfsUQ3b",
                "type": "function",
                "function": {
                  "name": "get_inventory_quantity",
                  "arguments": "{\"product_id\":123456}"
                }
              }
            ]
          },
          "finish_reason": "tool_calls"
        }
      ],
      "usage": {
        "completion_tokens": 18,
        "prompt_tokens": 86,
        "total_tokens": 104
      }
    }
  },
  "final_result": {
    "id": "chatcmpl-BuZqH3BWFuVVuge7Hg73ecLutnTje",
    "object": "chat.completion",
    "created": 1752823845,
    "model": "gpt-4o-2024-08-06",
    "system_fingerprint": "fp_ee1d74bde0",
    "choices": [
      {
        "index": 0,
        "message": {
          "role": "assistant",
          "content": "",
          "tool_calls": [
            {
              "id": "call_9yP43Z8VwoEQoB8mCdfsUQ3b",
              "type": "function",
              "function": {
                "name": "get_inventory_quantity",
                "arguments": "{\"product_id\":123456}"
              }
            }
          ]
        },
        "finish_reason": "tool_calls"
      }
    ],
    "usage": {
      "completion_tokens": 18,
      "prompt_tokens": 86,
      "total_tokens": 104
```

```
      }
    }
  }
```

When you have called your custom function, use the response to extend your LLM call.

In the following example, the function call has returned a quantity of 25 items in integer format. This information is inserted into the `template` in the `tool` role.

Sample Code

```
curl --request POST \
  --url {{orchestration_deployment_url}}/v2/completion \
  --header 'AI-Resource-Group: default' \
  --header 'Authorization: Bearer {{token}}' \
  --header 'content-type: application/json' \
  --data '{
  "config": {
    "modules": {
      "prompt_templating": {
        "prompt": {
          "template": [
          {
            "role": "system",
            "content": "You are a helpful assistant. You help users to analyze inventory of the user'\''s store by calling the get_inve
          },
          {
            "role": "user",
            "content": "How many units of the product with ID {{ ?product_id }} are in stock?"
          },
          # append the function call message
          {
            "role": "assistant",
            "content": "",
            "tool_calls": [
              {
                "id": "call_il3KDaSC5zm6naTOnYv5VSZT",
                "type": "function",
                "function": {
                  "name": "get_inventory_quantity",
                  "arguments": "{\"product_id\":123456}"
                }
              }
            ]
          },
          # append function result
          {
            "role": "tool",
            "tool_call_id": "call_il3KDaSC5zm6naTOnYv5VSZT",
            "content": "25"
          }
        ],
        "tools": [
          {
            "type": "function",
            "function": {
              "name": "get_inventory_quantity",
              "description": "Check available quantity for a product ID.",
              "strict": true,
              "parameters": {
                "type": "object",
                "properties": {
                  "product_id": {
                    "type": "integer"
                  }
                },
                "required": [
                  "product_id"
                ],
                "additionalProperties": false
              }
            }
          }
        ]
        },
        "model": {
          "name": "gpt-4o",
          "params": {
            "max_tokens": 200
          }
        }
      }
    }
  },
  "placeholder_values": {
    "product_id": "123456"
  }
}'
```

The model answers the query, incorporating the result of the function call into its response:

Output Code

```
{
  "request_id": "9b34a47b-14f1-95d3-9f84-51bdc4d66c2d",
  "intermediate_results": {
    "templating": [
```

```
        {
          "role": "system",
          "content": "You are a helpful assistant. You help users to analyze inventory of the users store by calling the get_inventory_qua
        },
        {
          "content": "How many units of the product with ID 123456 are in stock?",
          "role": "user"
        },
        {
          "role": "assistant",
          "content": "",
          "tool_calls": [
            {
              "id": "call_il3KDaSC5zm6naTOnYv5VSZT",
              "type": "function",
              "function": {
                "name": "get_inventory_quantity",
                "arguments": "{\"product_id\":123456}"
              }
            }
          ]
        },
        {
          "role": "tool",
          "tool_call_id": "call_il3KDaSC5zm6naTOnYv5VSZT",
          "content": "25"
        }
      ],
      "llm": {
        "id": "chatcmpl-BuZvTwr6ZSxebgqHn8vp46zntVcZ1",
        "object": "chat.completion",
        "created": 1752824167,
        "model": "gpt-4o-2024-08-06",
        "system_fingerprint": "fp_ee1d74bde0",
        "choices": [
          {
            "index": 0,
            "message": {
              "role": "assistant",
              "content": "There are 25 units of the product with ID 123456 in stock."
            },
            "finish_reason": "stop"
          }
        ],
        "usage": {
          "completion_tokens": 18,
          "prompt_tokens": 113,
          "total_tokens": 131
        }
      }
    },
    "final_result": {
      "id": "chatcmpl-BuZvTwr6ZSxebgqHn8vp46zntVcZ1",
      "object": "chat.completion",
      "created": 1752824167,
      "model": "gpt-4o-2024-08-06",
      "system_fingerprint": "fp_ee1d74bde0",
      "choices": [
        {
          "index": 0,
          "message": {
            "role": "assistant",
            "content": "There are 25 units of the product with ID 123456 in stock."
          },
          "finish_reason": "stop"
        }
      ],
      "usage": {
        "completion_tokens": 18,
        "prompt_tokens": 113,
        "total_tokens": 131
      }
    }
  }
```

Translation

Thetranslation module is optional. It allows you to translate LLM text prompts into a chosen target language.

The input translation module helps improve answer quality when the configured model performs better with input in a specific language, for example English.

The output translation module is optional. It allows you to translate LLM output into a chosen target language.

To use translation, configure it as part of your orchestration workflow.

The module supports SAP's Document Translation service (part of the SAP Translation Hub).

Caution

Input translation is performed **before** any configured input masking. Any content added to the prompt as part of prompt templating is sent to the translation service unmodified.
Output translation is performed **after** any configured output unmasking. Unmasked output is sent to the translation service.

Related Information

Enhance Model Consumption with Input Translation
Enhance Model Consumption with Output Translation

Enhance Model Consumption with Input Translation

To configure translation, add a `translation` section to the `modules` of your `config`.

Configuration Example

Sample Code

```
    "config": {
        "modules": {
            ...
            "translation": {
                "input": {
                    "type": "sap_document_translation",
                    "config": {
                        "source_language": "de-DE",
                        "target_language": "en-US"
                    }
                }
            }
            ...
        }
    }
    ...
}
...
```

Configuration Parameters

The `translation` object must specify the keys::

- `type`: the translation service being called.

  The following values are supported:

  - `sap_document_translation`

- `config`: parameters sent to the service to configure translation.

  The following parameters are supported:

  - `source_language` (optional): the language of the input text. If no source language is specified, the SAP Document Translation Service automatically detects the source language.
  - `target_language` (mandatory): the language to translate the input to.
  - `apply_to` (optional): specify which parts of the input to translate for granular control. For more information , see Selective Translation with apply_to.

Note

For texts containing multiple languages, the service might be unable to detect the source language accurately.

Supported Languages

For more information on supported languages and translation pairs, see Supported Languages of the document translation service in SAP Translation Hub.

Selective Translation with `apply_to`

The `apply_to` parameter enables precise control over which content gets translated, allowing you to translate only specific parts of your input while leaving others unchanged.

Configuration Properties

Each entry in the `apply_to` array supports the following:

- `category` (required): the content type to select for translation

  Supported values:

  - `placeholders`: translate specific placeholder values from `placeholder_values` and `grounding.placeholders.output`
  - `template_roles`: translate template messages with specific message roles
- `items` (required): an array of specific items within the category to translate
  - For `category: placeholders`: array of placeholder keys
    - Examples: `["user_input", "grounding_input", "grounding_output", "custom_parameter"]`
  - For `category: template_roles`: Array of message role names from your template.
    - Supported values: `["user", "assistant", "system", "developer", "tool"]`
    - These correspond to the `role` field in your message objects
- `source_language` (optional): The source language for this entry. If not provided, the service automatically detects the source language.

→ Tip

When using automatic source language detection, avoid grouping items with different languages in the same `apply_to` entry, as this may lead to incorrect translation results. Use separate entries for items in different languages.

Sample Code

```
{
    "config": {
        "modules": {
            "translation": {
                "input": {
                    "type": "sap_document_translation",
                    "config": {
                        "target_language": "en-US",
                        "apply_to": [
                            {
                                "category": "placeholders",
                                "items": [
                                    "user_input"
                                ],
                                "source_language": "de-DE"
                            },
                            {
                                "category": "placeholders",
                                "items": [
                                    "grounding_output"
                                ]
                            },
                            {
                                "category": "template_roles",
                                "items": [
                                    "user",
                                    "assistant"
                                ]
                            }
                        ]
                    }
                }
            }
        }
    }
}
```

Chat History Control

By default, chat history messages are translated. To avoid repeating the translation of chat history messages that have already been translated, set `translation.input.config.translate_messages_history` to `false`.

Examples

Example 1: Customer Support Request Translation

The following example shows a summarizing support assistant and uses the translation module to translate a support request message from German to English before sending it to the LLM for summarization.

Sample Code

```
{
    "config": {
        "modules": {
            "prompt_templating": {
                "messages_history": [
                    {
                        "role": "system",
                        "content": "You are a helpful support assistant. Your task is to summarize a given support request for the humai
                    },
                    {
                        "role": "user",
                        "content": "Support Request: '''{{?support-request}}'''"
                    }
                ]
            },
            "model": {
                "name": "gpt-4o-mini"
            },
            "translation": {
                "input": {
                    "type": "sap_document_translation",
                    "config": {
                        "source_language": "de-DE",
                        "target_language": "en-US"
                    }
                }
            }
        }
    },
    "placeholder_values": {
        "support-request": "Subject: Bestellung #1234567890 Verspätet - John Johnson Nachricht: Hallo, ich schreibe ihnen um mich nach (
    }
}
```

Example 2: Study Guide with Selective Placeholder Translation

A study guide generator that translates specific input parameters while keeping others unchanged:

Sample Code

```
{
    "placeholder_values": {
        "exam_type": "Abitur",
        "topic": "Deutsche Literatur",
        "num_questions": "5"
    },
    "config": {
        "modules": {
            "prompt_templating": {
                "prompt": {
                    "template": [
                        {
                            "role": "system",
                            "content": "You are an expert study coach creating clear, concise exam notes and practice questions."
                        },
                        {
                            "role": "user",
                            "content": "Generate a study guide for the {{?exam_type}} exam on {{?topic}}.\n\nInclude {{?num_questions}}
                        }
                    ]
                },
                "model": {
                    "name": "gpt-4o"
                }
            },
            "translation": {
                "input": {
                    "type": "sap_document_translation",
                    "config": {
                        "apply_to": [
                            {
                                "category": "placeholders",
                                "items": [
                                    "exam_type",
                                    "topic"
                                ],
                                "source_language": "de-DE"
                            }
                        ],
                        "target_language": "en-US"
                    }
                }
            }
        }
    }
}
```

Enhance Model Consumption with Output Translation

Output translation translates LLM responses into your preferred language. This ensures the LLM can operate at peak performance in its strongest language while delivering responses in the format most useful to the user.

To configure output translation, add a `translation` section to the `modules` of your `config`.

Configuration Example

Sample Code

```
{
   "config": {
       "modules": {
           ...
           "translation": {
               "output": {
                   "type": "sap_document_translation",
                   "config": {
                       "source_language": "en-US",
                       "target_language": "de-DE"
                   }
               }
           }
           ...
       }
   }
   ...
}
...
```

Configuration Parameters

The `translation` must object must specify the keys:

- `type`: the translation service being called

  The following values are supported:

  - `sap_document_translation`
- `config`: the parameters that are sent to the service to configure translation
- `target_language` (mandatory): the output language

  The following values are supported:

  - A language code string (for example, "de-DE") for fixed target language
  - A reference object to match the language of your input query. For more information, see Automatic Target Language Inference.

Supported Languages

For more information on supported languages and translation pairs, see Supported Languages of the document translation service in SAP Translation Hub.

Automatic Target Language Inference

Output translation can automatically respond in the user's original language by referencing the detected source language from input translation. This is particularly useful if the user's language is not known in advance.

Instead of specifying a fixed target language, reference an `apply_to` item from your input translation. For more information, see Selective Translation with apply_to.

Sample Code

```
{
    "config": {
        "modules": {
            "translation": {
                "input": {
                    "type": "sap_document_translation",
                    "config": {
                        "target_language": "en-US",
                        "apply_to": [
                            {
                                "category": "template_roles",
                                "items": [
                                    "user"
                                ]
                            }
```

```
                        ]
                    }
                },
                "output": {
                    "type": "sap_document_translation",
                    "config": {
                        "source_language": "en-US",
                        "target_language": {
                            "category": "template_roles",
                            "items": [
                                "user"
                            ]
                        }
                    }
                }
            }
        }
    }
}
```

! Restriction

The reference must exactly match an existing `apply_to` entry from your input translation configuration. In the example above, the `target_language` object in the output translation matches the first element in the input translation's `apply_to` array.

Examples

Example 1: Basic Output Translation

The following example shows a summarizing support assistant and uses the translation module to translate a support request summary into German for a German-speaking support team.

Sample Code

```
{
    "config": {
        "modules": {
            "prompt_templating": {
                "messages_history": [
                    {
                        "role": "system",
                        "content": "You are a helpful support assistant. Your task is to summarize a given support request for the huma
                    },
                    {
                        "role": "user",
                        "content": "Support Request: '''{{?support-request}}'''"
                    }
                ]
            },
            "model": {
                "name": "gpt-4o-mini"
            },
            "translation": {
                "output": {
                    "type": "sap_document_translation",
                    "config": {
                        "source_language": "en-US",
                        "target_language": "de-DE"
                    }
                }
            }
        }
    },
    "placeholder_values": {
        "support-request": "Subject: Order #1234567890 Delayed - John Johnson Message: Hi I'm writing to inquire about the status of my
    }
}
```

Example 2: Automatic Target Language Matching

A travel assistant that automatically responds in the same language as the input:

Sample Code

```
{
    "config": {
        "modules": {
            "prompt_templating": {
                "prompt": {
                    "template": [
                        {
                            "role": "system",
                            "content": "You are a helpful travel assistant who gives clear and friendly planning advice."
                        },
                        {
                            "role": "user",
                            "content": "Ich möchte nächsten Monat nach Paris reisen. Kannst du mir bei der Planung helfen?"
                        }
                    ]
                },
                "model": {
                    "name": "gpt-4o"
                }
```

```
                },
                "translation": {
                    "input": {
                        "type": "sap_document_translation",
                        "config": {
                            "apply_to": [
                                {
                                    "category": "template_roles",
                                    "items": [
                                        "user"
                                    ]
                                }
                            ],
                            "target_language": "en-US"
                        }
                    },
                    "output": {
                        "type": "sap_document_translation",
                        "config": {
                            "source_language": "en-US",
                            "target_language": {
                                "category": "template_roles",
                                "items": [
                                    "user"
                                ]
                            }
                        }
                    }
                }
            }
        }
    }
```

Data Masking

The data masking module is optional and serves to anonymize or pseudonymize personally identifiable information from the input for selected entities.

There are two ways in which you can mask data:

- Anonymization replaces personally identifiable information in chosen categories with a `MASKED_ENTITY` placeholder. Anonymized data can't be unmasked because the original data isn't retained.
- Pseudonymization substitutes personally identifiable information in selected categories with a `MASKED_ENTITY_ID` placeholder. Pseudonymized data can be unmasked in the response.

The module currently supports the following anonymization services:

- SAP Data Privacy Integration - Anonymization

SAP Data Privacy Integration - Anonymization

SAP Data Privacy Integration - Anonymization service recognizes the following entity categories:

Entity	Description	Scope
profile-person	Person Name	English
profile-org	Organization	SAP customers + Fortune 1000 companies
profile-university	University	Organization identified as a public University
profile-location	Location	United States
profile-phone	Phone Number	International phone numbers in the following format: +country/region code, any prefix and phone number
profile-address	U.S. Physical Addresses	United States
profile-email	E-mail addresses	E-mail addresses
profile-sapids-internal	SAP Staff User ID Numbers	User ID: Sequence of characters that starts with either C, I, or D followed by 6-8 digits.
profile-sapids-public	Public SAP Accounts	S-user ID: Sequence of characters that starts with S and followed by 6-11 digits. P-user ID: Sequence of characters that starts with P followed by 10 digits.
profile-url	URL	Only anonymize URLs that can be accessed by the user
profile-username-password	Username and Passwords	Detected if words similar to user, user name, pass, passwords, and so on.
profile-nationalid	National ID Number	20+ countries/regions (for example, E.U., South America)
profile-iban	International Bank Account Number (IBAN)	70+ countries/regions (E.U., Middle East, South America)

Entity	Description	Scope
profile-ssn	Social Security Number (SSN)/Social Insurance Number (SIN)	United States and Canada
profile-credit-card-number	Credit Card Number	Global
profile-passport	Passport Number	30+ countries/regions (for example, E.U., Asia Pacific, North America)
profile-driverlicense	Driver's License	30+ countries/regions (for example, E.U., Asia Pacific)
profile-nationality	Nationality	190+ country/region names, official country/region names, and country/region codes
profile-religious-group	Religious Group	200+ religious groups
profile-political-group	Political Parties or Groups	100+ political parties or groups
profile-pronouns-gender	Gender pronoun	Global
profile-gender	Gender	Global
profile-sexual-orientation	Sexual Orientation	Global
profile-trade-union	Trade Union	Global
profile-sensitive-data	Nationality, Religious_Group, Gender, Political_Group, pronoun gender, ethnicity, sexual orientation, Trade union	Mixed
profile-ethnicity	Ethnicity	Global

To mask entities based on regular expressions, use custom entities. For more information, see Configuring Replacement Methods.

Caution

The masking service can obscure personally identifiable information in prompts. However, since it relies on automated detection mechanisms, it can't guarantee that all such information is identified and masked.

Anonymization replaces personally identifiable information in an irreversible way. As a result, context can be lost, which can limit the model's ability to process the input. For instance, if tasked with writing a story about Michael and Donna, anonymization of profile-person results in a story about `MASKED_PERSON` and `MASKED_PERSON`, making it impossible to distinguish between the two.

Enhancing Model Consumption with Data Masking

Prerequisites

You have a running orchestration deployment and have retrieved your orchestration deployment URL. For more information, see Get Your Orchestration Deployment URL and Create a Deployment for Orchestration.

Process

In the following example, we use the data masking module to anonymize people, organizations, and contact information in the input. The allow list contains the entries "Harvard University" and "Boston", and so these are not masked. Here, input is masked before the model is called. The data in the model's output cannot be unmasked.

Sample Code

```
curl --request POST $ORCH_DEPLOYMENT_URL/v2/completion \
    --header 'content-type: application/json' \
    --header "Authorization: Bearer $TOKEN" \
    --header "AI-Resource-Group: $RESOURCE_GROUP" \
    --data-raw '{
      "config": {
        "modules": {
          "prompt_templating": {
            "prompt": {
              "template": [
              {
                "role": "user",
                "content": "{{?input}}"
              }
            ]
            },
            "model": {
              "name": "gpt-4o",
              "params": {
                "max_completion_tokens": 300,
                "temperature": 0.1,
                "frequency_penalty": 0,
                "presence_penalty": 0
              }
           }
          },
        "masking": {
          "providers": [
```

```
            {
              "type": "sap_data_privacy_integration",
              "method": "anonymization",
              "entities": [
                {"type": "profile-person"},
                {"type": "profile-email"},
                {"type": "profile-phone"}
              ]
            }
          ]
        }
        }
      },
      "placeholder_values": {
        "input": "Repeat the following: Penny lives with her partner Peter and her dog Coco. Penny can be reached via pennylovesdogs@exa
      }
}'
```

As shown in the response, the data masking module masks the configured entities before sending them to the model. The model then operates on this masked data and can still provide a summary.

Output Code

```
{
  "request_id": "cd6fcf22-1d21-9b82-a3cd-fc896fe41e14",
  "intermediate_results": {
    "templating": [
      {
        "content": "Repeat the following: Penny lives with her partner Peter and her dog Coco. Penny can be reached via pennylovesdogs@e
        "role": "user"
      }
    ],
    "input_masking": {
      "message": "Input to LLM is masked successfully.",
      "data": {
        "masked_template": "[{\"content\": \"Repeat the following: MASKED_PERSON lives with her partner MASKED_PERSON and her dog Coco.
      }
    },
    "llm": {
      "id": "chatcmpl-BtrVHn0HeUEnwHCIpjuReoYfgJfD8",
      "object": "chat.completion",
      "created": 1752653407,
      "model": "gpt-4o-2024-08-06",
      "system_fingerprint": "fp_ee1d74bde0",
      "choices": [
        {
          "index": 0,
          "message": {
            "role": "assistant",
            "content": "I'm sorry, I can't assist with that request."
          },
          "finish_reason": "stop"
        }
      ],
      "usage": {
        "completion_tokens": 11,
        "prompt_tokens": 48,
        "total_tokens": 59
      }
    }
  },
  "final_result": {
    "id": "chatcmpl-BtrVHn0HeUEnwHCIpjuReoYfgJfD8",
    "object": "chat.completion",
    "created": 1752653407,
    "model": "gpt-4o-2024-08-06",
    "system_fingerprint": "fp_ee1d74bde0",
    "choices": [
      {
        "index": 0,
        "message": {
          "role": "assistant",
          "content": "I'm sorry, I can't assist with that request."
        },
        "finish_reason": "stop"
      }
    ],
    "usage": {
      "completion_tokens": 11,
```

```
        "prompt_tokens": 48,
        "total_tokens": 59
      }
    }
  }
```

When you use `"method": "pseudonymization"` instead of `"method": "anonymization"` in the masking module configuration, the data is pseudonymized before it's sent to the model. Additionally, the system checks the model's response for any data that can be unmasked before sending out the final response.

Output Code

```
{
  "request_id": "9a5f3241-5ff9-9011-bfc9-7cd9d993788d",
  "intermediate_results": {
    "templating": [
      {
        "content": "Repeat the following: Penny lives with her partner Peter and her dog Coco. Penny can be reached via pennylovesdogs@e
        "role": "user"
      }
    ],
    "input_masking": {
      "message": "Input to LLM is masked successfully.",
      "data": {
        "masked_template": "[{\"content\": \"Repeat the following: MASKED_PERSON_2 lives with her partner MASKED_PERSON_3 and her dog Co
      }
    },
    "llm": {
      "id": "chatcmpl-BuaBiRJcbI0NSglMoTl5xRtQUeUor",
      "object": "chat.completion",
      "created": 1752825174,
      "model": "gpt-4o-2024-08-06",
      "system_fingerprint": "fp_ee1d74bde0",
      "choices": [
        {
          "index": 0,
          "message": {
            "role": "assistant",
            "content": "MASKED_PERSON_2 lives with her partner MASKED_PERSON_3 and her dog Coco. MASKED_PERSON_2 can be reached via MASK
          },
          "finish_reason": "stop"
        }
      ],
      "usage": {
        "completion_tokens": 50,
        "prompt_tokens": 60,
        "total_tokens": 110
      }
    },
    "output_unmasking": [
      {
        "index": 0,
        "message": {
          "role": "assistant",
          "content": "Penny lives with her partner Peter and her dog Coco. Penny can be reached via pennylovesdogspennylovesdogs@example
        },
        "finish_reason": "stop"
      }
    ]
  },
  "final_result": {
    "id": "chatcmpl-BuaBiRJcbI0NSglMoTl5xRtQUeUor",
    "object": "chat.completion",
    "created": 1752825174,
    "model": "gpt-4o-2024-08-06",
    "system_fingerprint": "fp_ee1d74bde0",
    "choices": [
      {
        "index": 0,
        "message": {
          "role": "assistant",
          "content": "Penny lives with her partner Peter and her dog Coco. Penny can be reached via pennylovesdogspennylovesdogs@example
        },
        "finish_reason": "stop"
      }
    ],
    "usage": {
      "completion_tokens": 50,
      "prompt_tokens": 60,
      "total_tokens": 110
    }
  }
}
```

When the grounding module is used, the masking module also supports masking of the grounding input before retrieval. By default, this feature is disabled. To enable it the parameter `mask_grounding_input` in the `masking` can be used as follows:

Sample Code

```
{
  "config": {
    "modules": {
      "masking": {
        "providers": [
          {
            "type": "sap_data_privacy_integration",
```

```
              "method": "pseudonymization",
              "entities": [
                {"type": "profile-person"},
                {"type": "profile-email"}
              ],
              "mask_grounding_input": {"enabled": true}
            }
          ]
          ...
        }
        ...
      }
    }
  }
```

Configuring Replacement Methods

You can configure the replacement method for each entity in the `masking`. The following methods are supported:

- `fabricated_data`: Replaces the entity with a randomly generated value of the same type. For example, a person's name might be replaced with another name, and a phone number with a different phone number.
- `constant`: Replaces the entity with a specified constant value.
  - In anonymization, the constant value is used directly without any modification (for example, `MASKED_PERSON`).
  - In pseudonymization, the constant is appended with an incrementing number to ensure uniqueness (for example, `MASKED_PERSON_1`, `MASKED_PERSON_2`).

If no replacement strategy is specified, the default `constant` replacement method is used with a default constant value (for example, `MASKED_PERSON` for person entities).

The following example shows code for fabricated data replacement:

```
"entities": [
  {
    "type": "profile-person",
    "replacement_strategy": {
      "method": "fabricated_data"
    }
  }
]
```

The following example shows code for constant replacement with a custom value:

```
"entities": [
  {
    "type": "profile-person",
    "replacement_strategy": {
      "method": "constant",
      "value": "PERSON_REDACTED"
    }
  }
]
```

Using Custom Entities

In addition to the standard entity types, you can define custom entities using regular expressions. This allows you to mask specific patterns in your data that are not covered by the standard entity types. Custom entities require specifying the `constant` replacement strategy.

The following example shows code for constant replacement with a custom entity:

```
"entities": [
  {
    "regex": "[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\\.[a-zA-Z0-9-.]+",
    "replacement_strategy": {
      "method": "constant",
      "value": "REDACTED_EMAIL"
    }
  }
]
```

In this example, the regular expression `[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\\.[a-zA-Z0-9-.]+` is used to identify email addresses. The identified email addresses will be replaced with `REDACTED_EMAIL_1`, `REDACTED_EMAIL_2`, and so on.

Related Information

Leveraging Orchestration Capabilities to Enhance Responses
Libraries and SDKs

Content Filtering

The content filtering module is optional and allows you to filter input and output based on content safety criteria. If you don't configure an input filter in the orchestration settings, the input goes to the model configuration without filtering. If you don't configure an output filter in the orchestration settings, the response returns without any filtering.

The module supports two services:

- Azure Content Safety
- Llama Guard 3

The content filtering module allows you to configure multiple distinct filters for a single request. However, each filter type (such as Azure Content Safety or Llama Guard) can be configured only once, meaning that you cannot set up two filters of the same type but with different settings. However, different filter types (such as one Azure and one Llama Guard filter) can be executed concurrently. The orchestration service will wait for all content filters to complete before returning the results.

Azure Content Safety

Harm Categories

The Azure Content Safety classification service recognizes the following distinct content categories: `Hate`, `Violence`, `Sexual`, and `SelfHarm`. For more information, see Harm categories in Azure AI Content Safety. Text can have more than one label (for example, a text sample can be classified as both `Hate` and `Violence`). The returned content categories include a severity level rating of 0, 2, 4, or 6. The value increases with the severity of the content.

> **Note**
> For all Azure OpenAI models, a global content filter is configured on the Azure AI platform. This global filter removes all input and output that is classified as medium (4) or high (6) in any of the categories.

Prompt Attack Detection

The Azure Content Safety service also supports prompt attack detection for input text via `PromptShield` configuration. For more information see Prompt Shields in Azure AI Content Safety.

A prompt attack is a malicious input that is designed to bypass a model's safety mechanisms or override previous instructions. Prompt attacks can lead to the generation of harmful content or the execution of malicious actions. The Azure Content Safety service detects prompt attacks and returns a Boolean value.

If both prompt attack detection and harm classification are configured, the orchestration service performs the prompt attack detection call first, and then performs the harm classification call. If a prompt attack is detected, the orchestration service does not make the harm classification request and returns the prompt attack detection result only.

Protected Material Detection for Code

The Azure Content Safety service includes an output content filter that identifies code matching existing GitHub repository code.

The filter scans publicly available code, and has a knowledge cutoff date. Only code matching publicly available code added to GitHub after the knowledge cutoff date can be detected.

For more information see Protected Material Detection for Code in Azure AI Content Safety.

> **Remember**
> When multiple content filtering types are configured together, each type is processed as a separate request. Costs will be incurred for each filtering type applied.

Llama Guard 3

Llama Guard 3 recognizes the following content categories:

- `violent_crimes`
- `non_violent_crimes`
- `sex_crimes`
- `child_exploitation`
- `defamation`
- `specialized_advice`
- `privacy`
- `intellectual_property`
- `indiscriminate_weapons`
- `hate`
- `self_harm`
- `sexual_content`
- `elections`
- `code_interpreter_abuse`

For more information, see Hazard Taxonomy and Policy in Llama Guard 3 8B. Texts can have multiple labels. The returned content categories include a boolean value that shows whether the text contains content triggering the filter for each respective category.

i Note

Llama Guard 3 8B can report violations for categories that have not been requested by the user

The orchestration service doesn't take these violations into account when processing Llama Guard responses, and only filters violations in categories that have been requested by the user.

Related Information

Enhancing Model Consumption with Input Filtering
Enhancing Model Consumption with Output Filtering

Enhancing Model Consumption with Input Filtering

Prerequisites

You have a running orchestration deployment and have retrieved your orchestration deployment URL. For more information, see Get Your Orchestration Deployment URL and Create a Deployment for Orchestration.

Process

In this example, you configure content filtering on inputs using the `filtering` field within `modules`. There are two supported content filter service providers: `azure_content_safety` and `llama_guard_3_8b`. You can configure either or both in the `input.filters` field of `filtering`.

For `azure_content_safety`, you can set different scores for each of the four content categories: `hate`, `violence`, `sexual`, and `self_harm`. Lower scores mean stricter filtering, where anything classified with a severity above the set thresholds gets filtered. Additionally, you can configure prompt attack detection by setting the `prompt_shield` Boolean to `true`.

For `llama_guard_3_8b`, you can configure fourteen different content categories using a boolean value. Setting a category to `true` means the input text is checked for violations, while `false` means it isn't. You can also choose not to include a category instead of setting it to `false`.

The input is sent to the LLM deployment only if it passes the configured filters.

```
curl --request POST $ORCH_DEPLOYMENT_URL/v2/completion \
    --header 'content-type: application/json' \
    --header "Authorization: Bearer $TOKEN" \
    --header "AI-Resource-Group: $RESOURCE_GROUP" \
    --data-raw ' {
  "config": {
    "modules": {
      "prompt_templating": {
        "prompt": {
          "template": [
            {
              "role": "user",
              "content": "{{?input}}"
            }
          ]
        },
        "model": {
          "name": "gpt-4o-mini",
          "params": {
            "max_tokens": 300,
            "temperature": 2
          }
        }
      },
      "filtering": {
        "input": {
          "filters": [
            {
              "type": "azure_content_safety",
              "config": {
                "hate": 0,
                "violence": 2,
                "sexual": 4,
                "self_harm": 6,
                "prompt_shield": true
              }
            },
            {
              "type": "llama_guard_3_8b",
              "config": {
                "violent_crimes": true,
                "non_violent_crimes": true,
                "sex_crimes": true,
                "child_exploitation": false,
                "defamation": false,
                "specialized_advice": false,
                "privacy": false,
                "intellectual_property": false,
                "indiscriminate_weapons": false,
                "hate": false,
```

```
                "self_harm": false,
                "sexual_content": false,
                "elections": false,
                "code_interpreter_abuse": false
              }
            }
          ]
        }
      }
    }
  },
  "placeholder_values": {
    "input": "Develop a plan to rob a bank."
  }
}'
```

The response below shows that the request was rejected. The input filter rejected the request due to a `violence` score of 4, which is above the configured threshold of 2, and a `non_violent_crimes` score of `true`.

Because the input was rejected, no request to the LLM deployment was made.

Note that if you configure multiple filters, a violation of any filter results in the request being rejected.

```
{
  "error": {
    "request_id": "a0e597be-098d-9c92-91f1-5c535d9bd39b",
    "code": 400,
    "message": "400 - Filtering Module - Input Filter: Prompt filtered due to safety violations. Please modify the prompt and try again.",
    "location": "Filtering Module - Input Filter",
    "intermediate_results": {
      "templating": [
        {
          "content": "Develop a plan to rob a bank.",
          "role": "user"
        }
      ],
      "input_filtering": {
        "message": "Prompt filtered due to safety violations. Please modify the prompt and try again.",
        "data": {
          "azure_content_safety": {
            "Hate": 0,
            "SelfHarm": 0,
            "Sexual": 0,
            "Violence": 4,
            "userPromptAnalysis": {
              "attackDetected": false
            }
          },
          "llama_guard_3_8b": {
            "violent_crimes": false,
            "non_violent_crimes": true,
            "sex_crimes": false
          }
        }
      }
    }
  }
}
```

Related Information

Leveraging Orchestration Capabilities to Enhance Responses
Libraries and SDKs

Enhancing Model Consumption with Output Filtering

Prerequisites

You have a running orchestration deployment and have retrieved your orchestration deployment URL. For more information, see Get Your Orchestration Deployment URL and Create a Deployment for Orchestration.

Process

The following example shows an output content filter for multiple harm categories, and for `protected_material_code`. The output from the LLM is filtered by both `azure_content_safety` and `llama_guard_3_8b` before the response is returned.

```
curl --request POST $ORCH_DEPLOYMENT_URL/v2/completion \
--header 'content-type: application/json' \
--header "Authorization: Bearer $TOKEN" \
--header "AI-Resource-Group: $RESOURCE_GROUP" \
--data-raw '{
    "config": {
        "modules": {
            "prompt_templating": {
                "prompt": {
                    "template": [
                        {
                            "role": "user",
                            "content": "<A prompt that creates output that might be filtered by output filtering>"
```

```
                        }
                    ]
                },
                "model": {...}
            }
        },
        "filtering": {
            "output": {
                "filters": [
                    {
                        "type": "azure_content_safety",
                        "config": {
                            "hate": 0,
                            "self_harm": 0,
                            "sexual": 0,
                            "violence": 0,
                            "protected_material_code": true

                        }
                    },
                    {
                        "type": "llama_guard_3_8b",
                        "config": {
                            "violent_crimes": true,
                            "non_violent_crimes": true,
                            "sex_crimes": true,
                            "child_exploitation": true,
                            "defamation": true,
                            "specialized_advice": true,
                            "privacy": true,
                            "intellectual_property": true,
                            "indiscriminate_weapons": true,
                            "hate": true,
                            "self_harm": true,
                            "sexual_content": true,
                            "elections": true,
                            "code_interpreter_abuse": true
                        }
                    }
                ]
            }
        }
    }
}'
```

The response indicates that the output is filtered because it has severity ratings of 2 in both the `sexual` and `violence` categories. However, it doesn't violate the `llama_guard_3_8b` hate category. In this situation, the orchestration result adapts to show the output filtering. The assistant message content isn't displayed in the response, and the finish reason is set to `content_filter`.

```
{
  "request_id": "4435c6a6-f08e-9561-9a58-1103ac581550",
  "intermediate_results": {
    "templating": [
      {
        "content": "<A prompt that creates output that might be filtered by output filtering>",
        "role": "user"
      }
    ],
    "output_filtering": {
      "message": "1 of 1 choices failed the output filter.",
      "data": {
        "choices": [
          {
            "index": 0,
            "azure_content_safety": {
              "Hate": 0,
              "SelfHarm": 0,
              "Sexual": 2,
              "Violence": 0
            }
          }
        ]
      }
    },
    "llm": {
      "id": "chatcmpl-Bua9DAmEQjcNASrZL4oWMOtPVlGPm",
      "object": "chat.completion",
      "created": 1752825019,
      "model": "gpt-4o-mini-2024-07-18",
      "system_fingerprint": "fp_efad92c60b",
      "choices": [
        {
          "index": 0,
          "message": {
            "role": "assistant",
            "content": "<A prompt that creates output that might be filtered by output filtering>"
          },
          "finish_reason": "stop"
        }
      ],
      "usage": {
        "completion_tokens": 100,
        "prompt_tokens": 100,
        "total_tokens": 200
      }
    }
  },
  "final_result": {
    "id": "chatcmpl-Bua9DAmEQjcNASrZL4oWMOtPVlGPm",
    "object": "chat.completion",
    "created": 1752825019,
```

```
    "model": "gpt-4o-mini-2024-07-18",
    "system_fingerprint": "fp_efad92c60b",
    "choices": [
      {
        "index": 0,
        "message": {
          "role": "assistant",
          "content": ""
        },
        "finish_reason": "content_filter"
      }
    ],
    "usage": {
      "completion_tokens": 100,
      "prompt_tokens": 100,
      "total_tokens": 200
    }
  }
}
```

Caution

The contents of the returned `module_results` field may include unchecked user or model content. We recommend that you do not display this content to end users.

Related Information

Leveraging Orchestration Capabilities to Enhance Responses
Libraries and SDKs

Harmonized API

The Harmonized API lets you use different foundation models without the need to change the client code. It does so by taking the OpenAI API as the standard and mapping other model APIs to it. This includes standardizing message formats, model parameters, and response formats. The Harmonized API is integrated into the templating module, the model configuration, and the orchestration response.

In the OpenAI format, a prompt and its response can contain a list of messages with a role and content. The role can be `system`, `user`, or `assistant` and the content is the text of the message. The response can contain a list of choices with `index`, `messages`, and a `finish_reason`.

Example

When you use a Gemini model with orchestration, the OpenAI message format is translated internally to the Vertex AI message format expected by the Gemini model. Consider the following message list in OpenAI format:

```
[
  {
    "role": "system",
    "content": "You are a friendly assistant."
  },
  {
    "role": "user",
    "content": "Hello"
  }
  {
    "role": "assistant",
    "content": "Hi"
  }
]
```

To use this message list with a Gemini model, it is translated internally to the following:

```
{
  "systemInstruction": {
    "role": "system",
    "parts": [
      {
        "text": "You are a friendly assistant."
      }
    ]
  },
  "contents": [
    {
      "role": "user",
      "parts": [
        {
          "text": "Hello"
        }
      ]
    },
    {
      "role": "assistant",
      "parts": [
        {
          "text": "Hi"
        }
      ]
    }
  ]
}
```

Orchestration supports the model parameters offered by OpenAI, including `max_tokens`, `temperature`, `frequency_penalty`, `presence_penalty`, n, and `top_p`. Where possible, these parameters are used with any model and mapped to the corresponding parameter of the foundation model. For example, OpenAI's `max_tokens` will be mapped to Vertex AI's `maxOutputTokens`. Additionally, any model-specific parameters can be sent. However when using model specific parameters they should be in snake_case format for consistency. For example, you can use Vertex AI's `topK` parameter as `top_k`, which is normally not available in OpenAI's API.

Responses from Vertex AI are converted back into OpenAI's format, ensuring the client receives the response in a consistent format. For example, consider the following response in Vertex AI format:

```
{
  "candidates": [
    {
      "content": {
        "parts": [
          {
            "text": "Hello there! How can I assist you today?"
          }
        ],
        "role": "model"
      },
      "finishReason": "STOP",
      "safetyRatings": [
        {
          "category": "HARM_CATEGORY_HATE_SPEECH",
          "probability": "NEGLIGIBLE",
          "probabilityScore": 0.031439852,
          "severity": "HARM_SEVERITY_NEGLIGIBLE",
          "severityScore": 0.04509957
        },
        {
          "category": "HARM_CATEGORY_DANGEROUS_CONTENT",
          "probability": "NEGLIGIBLE",
          "probabilityScore": 0.089933015,
          "severity": "HARM_SEVERITY_NEGLIGIBLE",
          "severityScore": 0.025957357
        },
        {
          "category": "HARM_CATEGORY_HARASSMENT",
          "probability": "NEGLIGIBLE",
          "probabilityScore": 0.053799648,
          "severity": "HARM_SEVERITY_NEGLIGIBLE",
          "severityScore": 0.01711089
        },
        {
          "category": "HARM_CATEGORY_SEXUALLY_EXPLICIT",
          "probability": "NEGLIGIBLE",
          "probabilityScore": 0.19117664,
          "severity": "HARM_SEVERITY_NEGLIGIBLE",
          "severityScore": 0.046378203
        }
      ]
    }
  ],
  "usageMetadata": {
    "candidatesTokenCount": 10,
    "promptTokenCount": 1,
    "totalTokenCount": 11
  }
}
```

This is translated internally back into the OpenAI format, returning the following:

```
{
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Hello there! How can I assist you today?"
      },
     "finish_reason": "stop"
    }
  ],
  "usage": {
    "completion_tokens": 10,
    "prompt_tokens": 1,
    "total_tokens": 11
  }
}
```

Embeddings

Semantic embeddings are multidimensional representations of textual information.

The embeddings endpoint is harmonized across different LLMs and can be used alone, or with anonymization of input data.

For more information about supported models, see references to orchestration in Supported Models.

This feature is only available in orchestration V2.

Send POST request to the endpoint `{{Orchestration URL}}/v2/embeddings` and populate the sample code with the following values:

	Value
{{orchestration-deployment-url}}	The URL of your RUNNING orchestration deployment.
{{token}}	Your authorization token for SAP AI Core
{{resource_group}}	The AI resource group assigned to your account

Minimal Call

The following minimal call shows a model selection and input text.

```
curl --request POST \
  --url {{Orchestration URL}}/v2/embeddings \
  --header 'AI-Resource-Group: {{resource group}}' \
  --header 'Authorization: Bearer {{token}}' \
  --header 'content-type: application/json' \
  --data '{
  "config": {
    "modules": {
      "embeddings": {
        "model": {
          "name": "text-embedding-3-large"
        }
      }
    }
  },
  "input": {
    "text": "Hello, SAP!"
  }
}'
```

Module Calling

The following example includes model parameters and masking configuration. For more information, see Data Masking and Model Configuration.

```
curl --request POST \
  --url {{Orchestration URL}}/v2/embeddings \
  --header 'AI-Resource-Group: {{resource group}}' \
  --header 'Authorization: Bearer {{token}}' \
  --header 'content-type: application/json' \
  --data '{
  "config": {
    "modules": {
      "embeddings": {
        "model": {
          "name": "text-embedding-3-large",
          "version": "latest",
          "params": {
            "encoding_format": "float",
            "dimensions": 5
          }
        }
      },
      "masking": {
        "providers": [
          {
            "type": "sap_data_privacy_integration",
            "method": "anonymization",
            "entities": [
              {
                "type": "profile-person"
              },
              {
                "type": "profile-email"
              },
              {
                "type": "profile-phone"
              },
              {
                "type": "profile-nationality"
              },
              {
                "type": "profile-university"
              },
              {
                "type": "profile-location"
              },
              {
```

```
                "type": "profile-url"
              },
              {
                "type": "profile-gender"
              },
              {
                "type": "profile-credit-card-number"
              }
            ]
          }
        ]
      }
    }
  },
  "input": {
    "text": "Hello, Jake! I had a great meeting with the English stakeholder from Oxford University, located at 1 Infinite Loop, London. T
  }
}'
```

Harmonization

Orchestration harmonizes the different model provider interfaces under one API. Where supported, the parameter `input_types` defines the mode of embedding generation. Specifying the correct mode can increase accuracy.

Note

For most models this parameter is optional or not supported. For `nvidia--llama-3.2-nv-embedqa-1b` it is mandatory.

For more information, see the documentation from the model provider.

Orchestration accepts the following input types:

- `document`: Indexing of document chunks for information retrieval scenarios.
- `query`: Embedding creation of search queries for document search in information retrieval scenarios.
- `text`: Generation of embeddings that are optimized for obtaining similarity scores.

The input type mapping of Orchestration to provider API is as follows:

Google

```
{
    "document": "RETRIEVAL_DOCUMENT",
    "query": "RETRIEVAL_QUERY",
    "text": "SEMANTIC_SIMILARITY"
}
```

NVIDIA `nvidia--llama-3.2-nv-embedqa-1b`

`input_types` is mandatory for this model.

```
{
    "document": "passage",
    "query": "query",
    "type": "query"
}
```

For example:

Sample Code

```
{
    "config": {
        "modules": {
            "embeddings": {
                "model": {
                    "name": "nvidia-llama-3.2-nv-embedqa-1b"
                }
            }
        }
    },
    "input": {
        "text": "My name is Tom. I am applying as a Senior Software Dev. I work closely with Jerry:",
        "type": "query"
    }
}
```

Streaming

Streaming is optional. Where models support streaming, output is generated in chunks, and passed through the modules in the orchestration workflow. This is useful for applications such as chatbots, where interactions with the model happen in real time.

Caution

Streaming can increase the number of calls, increasing consumption and associated costs. For example, if a text of 500 characters generates 5 chunks of 100 characters each, the content filtering module receives 5 calls instead of one.

Streaming can decrease the accuracy of filtering and unmasking, because smaller chunks contain less context. Benchmarking and testing with actual use case data is recommended to ensure output quality in relation to chunk size.

For more information, see Metering and Pricing for Generative AI and SAP Note 3505347.

Activate Streaming

Streaming is set to `false` by default, and is part of the `orchestration_config`. To activate streaming, set `stream` to `true`. For example:

Sample Code

```
{
  "orchestration_config": {
        "stream": true,
        "module_configurations": {
        ...
      },
      "input_params": {
        ...
      }
  }
}
```

You can configure streaming behavior in the following ways:

- Chunk Size
- Delimiters

Configure Streaming Using Chunk Size

The `chunk_size` parameter defines the maximum number of characters contained in a single chunk, and takes an integer value.

This means that if a model produces chunks that are larger than the configured `chunk_size`, the Orchestration Service will emit these chunks as is, instead of splitting them into multiple chunks.

This minimizes latency and maximizes the context provided to subsequent tasks like content filtering.

The default value for `chunk_size` is 100.

You can configure the `chunk_size` using the `stream_options` parameter in the `orchestration_config`:

The following example shows a streaming configuration with chunk size of 200 characters.

Sample Code

```
{
   "orchestration_config": {
      "stream": true,
      "stream_options": {
         "chunk_size": 200
      },
      "module_configurations": {
         ...
         "filtering_module_config": {
            "output": {
               "filters": [
                  ...
               ],
               "stream_options": {
                  "overlap": 10
               }
            }
         }
         ...
      },
      "input_params": {
         ...
      }
   }
}
```

Configure Streaming Using Delimiters

Delimiters can be specified alternatively to, or in addition to chunk size. Streaming configured with the `delimiters` parameter respects the `chunk_size` parameter, but determines the end of a chunk based on the next delimiter in the list, instead of chunk size alone. This means that chunks can be determined in a more meaningful way, for example,

to include whole sentences, which may improve output translation, filtering and unmasking results.

You can configure the delimiters using the `stream_options` parameter in the `orchestration_config`:

The following example shows a streaming configuration with delimiters set to common English language sentence endings.

Sample Code

```
{
  "orchestration_config": {
    "stream": true,
    "stream_options": {
      "delimiters": [".", "!", "?"]
    },
    "module_configurations": {
      ...
    },
    "input_params": {
      ...
  }
}
```

Module Specific Streaming Behavior

The following orchestration modules use content passed from streaming:

- Data Masking (unmasking pseudonymization only)
- Output filtering

Data Masking

For the data masking module, the full `MASKED_ENTITY_DIGIT` tag is needed for unmasking. When a tag is split by chunking, the entire tag is moved to the following chunk, changing the chunk size.

For example, two chunks containing:

Sample Code

```
Yesterday, I spent time with MASKED_PER
SON_1 discussing the changes
```

Would become:

Sample Code

```
Yesterday, I spent time with
MASKED_PERSON_1 discussing the changes
```

This may affect behavior of future modules that are executed after unmasking.

Output filtering

The `overlap` parameter defines the number of characters at the end of a chunk, that are repeated at the beginning of the next chunk. It takes an integer value. The default value is 0.

Increasing the context window improves output filtering results, but increasing chunk size can affect streaming experience negatively. Overlap repeats a specified amount of text across two adjacent chunks, creating a larger context window, without increasing chunk size.

For example:

Sample Code

```
"This is the text from the 1st chunk."
"This is the text from the 2nd chunk"
```

Would become:

Sample Code

```
"This is the text from the 1st chunk."
"the 1st chunk. This is the text from the 2nd chunk"
```

You can configure the `overlap` parameter using the `output` > `stream_options` > `overlap parameter` in the `filtering_module_config`.

The following example shows a streaming configuration with an overlap of 10 characters.

Sample Code

```
{
   "orchestration_config": {
      "stream": true,
      "stream_options": {
         "chunk_size": 200
      },
      "module_configurations": {
         ...
         "filtering_module_config": {
            "output": {
               "filters": [
                  ...
               ],
               "stream_options": {
                  "overlap": 10
               }
            }
         }
         ...
      },
      "input_params": {
         ...
      }
   }
}
```

Streaming Response Format

Streaming responses follow the conventions used by OpenAI models and are sent as Server-Sent Events (SSEs).

The first chunk always contains the module results of all modules executed before calling the LLM.

The LLM's output starts from the second chunk, and each chunk produced by orchestration sends a data event followed by a newline.

Successful responses are terminated with `[DONE]`. If an error occurs, orchestration sends a data event containing an error object as payload, and doesn't terminate with `[DONE]`.

Example for a Successful Response

The first data event contains the module result of modules that are run before the LLM access. This example shows the templating module result, and an empty orchestration result:

Output Code

```
data: {
  "request_id": "...",
       "module_results": {
         "templating": [
           {
             "role": "user",
             "content": "Create 3 paraphrases of I love you."}
         ]
       },
       "orchestration_result": {
         "id": "",
         "object": "",
         "created": 0,
         "model": "",
         "system_fingerprint": "",
         "choices": [
           {
             "index": 0,
             "delta": {
               "role": "",
               "content": ""
             },
             "finish_reason": ""
           }
         ]
       }
      }
```

The second and following data events contain module results from the LLM and following modules. This example shows results for LLM access and orchestration. The orchestration result is given in two chunks.

First chunk:

Output Code

```
data: {
  "request_id": "...",
  "module_results": {
    "llm":
    {
      ...
    }
  },
  "orchestration_result": {
    "id": "...",
    "object": "chat.completion.chunk",
    "created": 1728043660,
    "model": "<ModelName>",
    "choices": [
      {
```

```
          "index": 0,
          "delta": {
            "role": "assistant",
            "content": "1. My affection for you knows no bounds.\n2. You hold a special place in my heart.\n3. You mean the wo"
          },
          "finish_reason": ""
        }
      ]
    }
  }
```

Second chunk:

Output Code

```
data: {
  "request_id": "163ea321-8f4f-4d00-9b34-7603f1499c24",
  "module_results": {
    "llm": {
      ...
    }
  },
  "orchestration_result": {
    "id": "...",
    "object": "chat.completion.chunk",
    "created": 1728043660,
    "model": "<ModelName>",
    "choices": [
      {
        "index": 0,
        "delta": {
          "role": "assistant",
          "content": "rld to me."
        }, "finish_reason": "stop"
      }
    ]
  }
}
```

The final data-event contains `[DONE]`:

Output Code

```
data: [DONE]
```

Example for an Error Response

Sample Code

```
data: {"json": "payload"}

data: {"code": 500, "message": "message"}
```

Model Restriction

When creating a deployment for orchestration, you can restrict the choice of models using an allow or deny list. You can use this to implement internal standards, for example where only certain LLMs are approved for use.

To restrict model access, set the `parametersBindings` to contain `modelFilterList` and `modelFilterListType`.

- **`modelFilterList`**: a list of `modelNames` and `modelVersions` to be considered.

  If `modelVersions` is not defined, all versions of the given model are considered.

- **`modelFilterListType`**: a value that controls how `modelFilterList` should be interpreted.

  - `deny`: excludes the models and defined versions from use within orchestration.
  - `allow`: only allows the models and defined versions in the modelFilterList to be used within orchestration.

Sample Code

```
{
  "name": "yourNameChoice",
  "executableId": "orchestration",
  "scenarioId": "orchestration",
  "versionId": "0.0.1",
  "parameterBindings": [
    {
      "key": "modelFilterList",
      "value": "[{\"modelName\": \"gpt-4o-mini\", \"modelVersions\": [\"0613\", \"0301\"]}, {\"modelName\": \"gemini-1.5-pro\", \"model
    },
    {
      "key": "modelFilterListType",
      "value": "deny"
    }
```

```
    ]
}
```

Structured Output

You can use structured outputs to ensure that model generated outputs match schemas that you define.

Structured outputs exist in the following forms:

- Function calling, such as for tool usage

  Function calling is helpful for agentic use cases where the LLM determines which tool to use out of a provided selection of tools. The LLM determines which tool is best for the use case, but the tool needs to be called separately in the application.

  Sample Code

```
"templating_module_config": {
    "template": [
        {
            "role": "system",
            "content": "You are a helpful assistant. You help users do math by calling the abs function."
        },
        {
            "role": "user",
            "content": "Whats the absolute value of 3?"
        }
    ],
    "tools": [
        {
            "type": "function",
            "function": {
                "name": "abs",
                "description": "Calculate the absolute value of x.",
                "strict": true,
                "parameters": {
                    "type": "object",
                    "properties": {
                        "x": {
                            "type": "integer"
                        }
                    },
                    "required": [
                        "x"
                    ],
                    "additionalProperties": false
                }
            }
        }
    ]
}
```

- JSON response format

  JSON formats can be consumed in an application

```
"templating_module_config": {
      "template": [
          {
              "role": "system",
              "content": "You are a language translator."
          },
          {
              "role": "user",
              "content": "Whats 'Apple' in German?"
          }
      ],
      "response_format": {
          "type": "json_schema",
          "json_schema": {
              "name": "translation_response",
              "strict": true,
              "schema": {
                  "type": "object",
                  "properties": {
                      "language": {
                          "type": "string"
                      },
                      "translation": {
                          "type": "string"
                      }
                  },
                  "required": [
                      "language",
                      "translation"
                  ],
                  "additionalProperties": false
              }
          }
      }
}
```

Orchestration harmonizes tool usage and JSON response format across different models. For more information on which models support these features, see the documentation from the respective model provider.

For more information on structured outputs, see Open AI Structured Outputs and Open AI Structured Output Guide .

Error Handling

If an error occurs, the response will contain an error code and message. The following example shows a request that is missing a parameter in the templating module configuration.

```
curl --request POST $ORCH_DEPLOYMENT_URL/completion \
    --header 'content-type: application/json' \
    --header "Authorization: Bearer $TOKEN" \
    --header "AI-Resource-Group: $RESOURCE_GROUP" \
    --data-raw '{
    "orchestration_config": {
      "module_configurations": {
        "templating_module_config": {
          "template": [
            {
              "role": "user",
              "content": "Create {{?number}} paraphrases of {{?phrase}}"
            }
          ]
        },
        "llm_module_config": {
          "model_name": "<ModelName>",
          "model_params": {
            "max_tokens": 300
          }
        }
      }
    },
    "input_params": {
      "number": "3"
    }
  }
```

In this case, the response from the service contains an error code in the `code` field, a `message`, and a `location` indicating which orchestration module encountered the error. In all error cases, the `module_results` field only contains results for modules that successfully finished. In this example, the `module_results` are empty because the first module in the pipeline encountered the error.

```
{
  "request_id": "3e988846-1360-4a4a-a7ad-e77b85057321",
  "code": 400,
  "message": "Missing required parameters: ['phrase']",
  "location": "Module: Templating",
  "module_results": {}
}
```

Orchestration with Fallbacks

You can define multiple sets of module configurations for completions in your orchestration workflow to incorporate fallbacks in case of processing failure, such as unsupported models in the deployed region. Requests are processed using the configuration with the highest preference. If processing fails, the next preferred configuration is automatically used for processing. Preference order follows the sequence of the list: the first item has the highest priority, and the last item has the lowest priority.

This feature is only available in orchestration V2.

Criteria for switching to fallback configurations

For streaming and non-streaming requests: orchestration switches to fallback configurations when the specified model in the higher-preference instance is not supported in the deployed region.

For non-streaming requests: orchestration may also switch to the next fallback if it encounters certain errors during processing. This allows orchestration to leverage fallback configurations and to provide successful responses in case of transient errors, such as, temporary unavailability of a model or service. For streaming requests, switching to a fallback configuration is not straightforward once the stream has started.

Orchestration will only switch to the next defined fallback configuration if it encounters specific errors during processing. For all other errors, orchestration will fail the request and return the corresponding error in the response.

The fallback configuration will be used in the following scenarios:

- For streaming requests:
  - If specified model is not supported in the deployed region or environment
- For non-streaming requests:
  - If specified model is not supported in the deployed region or environment
  - If orchestration service encounters any of the following errors during processing:
    - Client errors: `429 Too Many Requests` and `408 Request Timeout`
    - All server errors `5xx`

Example

The following example shows a request with: the specified `modules` list indicating that `gpt-4o` is preferred as the primary model, with `mistralai-mistral-small-instruct` as the fallback option. If `gpt-4o` is unsupported in the deployed region, orchestration automatically switches to the second preference. Module configurations can vary between preferences to suit different use cases. In this case, the fallback configuration omits the filtering module.

Sample Code

```
curl --request POST $ORCH_DEPLOYMENT_URL/v2/completion \
--header 'content-type: application/json' \
--header "Authorization: Bearer $TOKEN" \
--header "AI-Resource-Group: $RESOURCE_GROUP" \
--data-raw '{
    "config": {
        "modules": [
            { //1st preference
               "prompt_templating": {
                    "prompt": {
                        "template": [
                            {
                                "role": "system",
                                "content": "You are a language translation model that translates English to German."
                            },
                            {
                                "role": "user",
                                "content": "{{ ?input }}"
                            }
                        ]
                    },
                    "model": {
                        "name": "gpt-4o",
                        "params": {
                            "max_tokens": 300,
                            "temperature": 2
                        }
                    }
                },
                "filtering": {
                    "input": {
                        "filters": [
                            {
                                "type": "azure_content_safety",
                                "config": {
                                    "prompt_shield": "true"
                                }
                            }
                        ]
                    }
                }
            },
            { //2nd preference
                "prompt_templating": {
                    "prompt": {
                        "template": [
                            {
                                "role": "system",
                                "content": "You are a language translation model that translates English to German."
                            },
                            {
                                "role": "user",
                                "content": "{{ ?input }}"
                            }
                        ]
                    },
                    "model": {
                        "name": "mistralai--mistral-small-instruct",
                        "params": {
                            "max_tokens": 300,
                            "temperature": 2
                        }
                    }
                }
            }
        ]
    },
    "placeholder_values": {
        "input": "Reply with Orchestration Service is working! in German"
    }
}'
```

The successful response below was processed using the fallback preference `mistralai-mistral-small-instruct`. The `intermediate_failures` field indicates why the highest preference model was skipped.

Output Code

```
{
    "request_id": "f380e708-cd64-9b87-b36b-03b53a678f59",
    "intermediate_results": {
        "templating": [
            {
                "content": "Reply with Orchestration Service is working! in German",
                "role": "user"
            }
        ],
        "llm": {
            "id": "chatcmpl-0d66dbb6-6a3b-9aa5-a2c3-4f4c9e608051",
            "object": "chat.completion",
            "created": 1761909762,
```

```
            "model": "mistralai--mistral-small-instruct",
            "choices": [
                {
                    "index": 0,
                    "message": {
                        "role": "assistant",
                        "content": "Sure! Here is the translation:\n\n\"Orchestration Service läuft!\"\n\nThis means \"Orchestration Se
                    },
                    "finish_reason": "stop"
                }
            ],
            "usage": {
                "completion_tokens": 30,
                "prompt_tokens": 14,
                "total_tokens": 44
            }
        }
    },
    "final_result": {
        "id": "chatcmpl-0d66dbb6-6a3b-9aa5-a2c3-4f4c9e608051",
        "object": "chat.completion",
        "created": 1761909762,
        "model": "mistralai--mistral-small-instruct",
        "choices": [
            {
                "index": 0,
                "message": {
                    "role": "assistant",
                    "content": "Sure! Here is the translation:\n\n\"Orchestration Service läuft!\"\n\nThis means \"Orchestration Servic
                },
                "finish_reason": "stop"
            }
        ],
        "usage": {
            "completion_tokens": 30,
            "prompt_tokens": 14,
            "total_tokens": 44
        }
    },
    "intermediate_failures": [
        {
            "code": 400,
            "message": "Model gpt-4o not supported.",
            "location": "Request Body",
            "headers": {
                "Content-Type": "application/json"
            }
        }
    ]
}
```


Error Response

If orchestration is unable to process the request using any of the provided fallback module configurations, it responds with a list of errors corresponding to each failed attempt. The overall response code matches the last failed attempt, and the error list follows the defined preference order.

Output Code


```
{
    "error": [// 1st preference        {
            "request_id": "req-id",
            "code": 400,
            "message": "400 - Request Body: Model gpt-4o not supported.",
            "location": "Request Body",
            "headers": {
                "Content-Type": "application/json"
            }
        },
        {// 2nd preference
            "request_id": "req-id",
            "code": 429,
            "message": "429 - Too many requests.",
            "location": "LLM",
            "headers": {
                "Content-Type": "application/json",
                "x-retry-after": "30"
            }
        }
    ]
}
```


Orchestration Tutorials

Find your use case with orchestration capabilities.

For a complete list of tutorials on orchestration, see Generative AI with SAP AI Core - Orchestration.

Related Information

Libraries and SDKs

Foundation Models

The foundation models service operates under the global AI scenario `foundation-models`, which is managed by SAP AI Core.

You can access foundation models by creating a deployment for the model that you want to use. To do this, you'll need an auth token from your SAP AI Core instance. For more information, see Get an Auth Token and Create a Deployment for a Generative AI Model.

The following scenarios are available:

Global Scenario	Executable ID	Description
foundation-models	aicore-cohere	Cohere models hosted and accessed through SAP AI Core.
foundation-models	aicore-ibm	Models from IBM that are are hosted and managed by SAP and accessed through SAP AI Core.
foundation-models	aicore-mistralai	Models from Mistral AI that are hosted and managed by SAP and accessed through SAP AI Core.
foundation-models	aicore-nvidia	Models from Nvidia that are hosted and managed by SAP and accessed through SAP AI Core.
foundation-models	aicore-opensource	Open source models that are hosted and managed by SAP and accessed through SAP AI Core.
foundation-models	aws-bedrock	AWS Bedrock provides access to Foundation models from Anthropic, Amazon and other providers.
foundation-models	azure-openai	The Azure OpenAI Service provides REST API access to OpenAI's LLMs.
foundation-models	gcp-vertexai	GCP Vertex AI provides access to PaLM 2 and Gemini models from Google.

For more information about available models, including conversion rates for tokens, rate limits and deprecation dates, see SAP Note 3437766.

After creating a deployment for your model, you consume the model using prompts. For more information, see Consume Models.

Get an Auth Token

Authentication token retrieval enables secure access to your SAP instance by setting environment variables and using curl commands to obtain and use an access token. Use this process to authenticate API requests in subsequent steps.

Create a Deployment for a Generative AI Model

Consume Models

The generative AI hub helps you to complete tasks like summarizing text, inferencing, and transforming content. To do so, you consume a generative AI model by sending a request to the model's endpoint.

Foundation Model Tutorials

We provide tutorials that demonstrate use cases for accessing generative AI models in the generative AI hub using foundation scenarios. To access each model version, you'll need to create a separate deployment.

Get an Auth Token

Authentication token retrieval enables secure access to your SAP instance by setting environment variables and using curl commands to obtain and use an access token. Use this process to authenticate API requests in subsequent steps.

Prerequisites

curl is likely to be installed on your operating system by default. To check, open a command prompt and enter `curl -V`. If curl isn't installed, download and install it from https://curl.se/.

Note

On macOS, you may need to install `jq` so that you can follow the curl commands.

1. Install brew from https://brew.sh/.
2. In a Terminal session, run `brew install jq` to install `jq` in your shell environment.

Context

Start by setting the required environment variables, which you can get from your SAP AI Core instance.

Procedure

1. Set up your environment as follows:

```
AI_API_URL=<YOUR AI API URL>
AUTH_URL=<YOUR AUTH URL>
CLIENT_ID=<YOUR CLIENT ID>
CLIENT_SECRET=<YOUR CLIENT SECRET>
RESOURCE_GROUP=default
```

2. Obtain the auth token by sending the following request:

```
SECRET=`echo -n "$CLIENTID:$CLIENTSECRET" | base64 -i - `
TOKEN=`curl --request POST \
  --url "$AUTH_URL/oauth/token" \
  --header "Content-Type: application/x-www-form-urlencoded" \
  --data "grant_type=client_credentials" \
  --data "client_id=$CLIENTID" \
  --data "client_secret=$CLIENTSECRET"'
```

The response includes an access token.

```
{"access_token":"ey...", "expires_in":7199, "jti":"...", "token_type":"bearer"}
```

3. Set the token to use it in the following steps:

```
TOKEN=ey...
```

Task overview: Foundation Models

Related Information

Create a Deployment for a Generative AI Model
Consume Models
Foundation Model Tutorials

Create a Deployment for a Generative AI Model

Prerequisites

- You have an SAP AI Core service instance and service key. For more information, see SAP AI Core Initial Setup Documentation.
- You're using the `extended` service plan. For more information, see Service Plans and Update a Service Plan.

Context

You make a model available for use by creating a deployment. You can do so one time for each model and model version. The model deployment includes the `modelName` and `version` of the model you want to access. After the deployment is complete, you have a `deploymentUrl`, which can be used across your organization to access the model version.

You can view the available models and their details by sending a GET request to the endpoint `{{apiurl}}/v2/lm/scenarios/foundation-models/models`

Task overview: Foundation Models

Related Information

Get an Auth Token
Consume Models
Foundation Model Tutorials

Choose a Model

Decide which LLM you want to deploy and note the following information:

- Executable ID
- Model name
- Model version

Note

- Instead of specifying a model version, using "latest" will use the latest version of the model available in SAP AI Core.
- Where the model version is not listed, it is not applicable.

```
curl --location '$AI_API_URL/v2/lm/scenarios/foundation-models/models' \
--header 'AI-Resource-Group: default' \
--header 'Authorization: Bearer $AUTH_TOKEN'
```

Output Code

```
{
  "count": 41,
  "resources": [
    {
      "accessType": "Remote",
```

```
        "allowedScenarios": [
          {
            "executableId": "azure-openai",
            "scenarioId": "foundation-models"
          },
          {
            "executableId": "orchestration",
            "scenarioId": "orchestration"
          }
        ],
        "description": "OpenAI GPT-4o engineered for speed and efficiency, supporting text, images and audio using Chat Completions API",
        "displayName": "GPT-4o",
        "executableId": "azure-openai",
        "model": "gpt-4o",
        "provider": "OpenAI",
        "versions": [
          {
            "capabilities": [
              "text-generation",
              "image-recognition"
            ],
            "contextLength": 128000,
            "cost": [
              {
                "inputCost": "0.00312"
              },
              {
                "outputCost": "0.0092"
              }
            ],
            "deprecated": false,
            "inputTypes": [
              "text",
              "image",
              "audio"
            ],
            "isLatest": false,
            "metadata": [
              {
                "meanWinRate": "0.938"
              },
              {
                "chatBotArenaScore": "1285"
              },
              {
                "airBenchRefusalRate": "0.528"
              }
            ],
            "name": "2024-05-13",
            "retirementDate": "",
            "streamingSupported": true
          }, ...

      ]
  }
```


The output contains the following parameters, which are needed to create a configuration:

Parameter	Description
count	Number of models available description
description	Description for model
executableId	Executable under which the model is available, required at the time of creating Deployment configuration model
model	Required as modelName at the time of creating Deployment configuration versions
versions	List of available model versions for given model
versions.name	Optional value to provide as modelVersion at the time of creating Deployment Configuration
versions.isLatest	Suggests if the modelVersion is latest. See Model Lifecycle for more information

For more information about available models, including conversion rates for tokens, rate limits and deprecation dates, see SAP Note 3437766.

Check Scenario

Check that you have access to the scenario containing generative AI by sending a GET request to `{{apiurl}}/v2/lm/scenarios`.

Note

You must use the same resource group for all of your generative AI activities. To use a different resource group, these steps must be repeated for each resource group. For more information, see Manage Resource Groups.

```
curl --location '{{apiurl}}/v2/lm/scenarios' \
--header 'AI-Resource-Group: default' \
--header "Authorization: Bearer $AUTH_TOKEN"
```

Step Result

The scenarios listed contain a scenario with the id `foundation-models`.

Create a Deployment

Create a Configuration

Create a configuration by sending a POST request to the endpoint `{{apiurl}}/v2/lm/configurations`.

Include details of the model to which you want to provide access by passing in the following parameters:

- `name` is your free choice of identifier.
- `executableId`, `modelName`, and `modelVersion` are provided in SAP Note 3437766.
- `scenarioId` must be `foundation-models`.
- `versionId` is your own version reference.

Sample Code

```
curl --location '$AI_API_URL/v2/lm/configurations' \
--header 'AI-Resource-Group: default' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
        "name": "yourNameChoice",
        "executableId": "<executableId>",
        "scenarioId": "foundation-models",
        "versionId": "0.0.1",
        "parameterBindings": [
                {
                        "key":"modelName",
                        "value":"<ModelName>"
                },
                {
                        "key": "modelVersion",
                        "value": "<ModelVersion>"
                }
  ]
}'
```

Tip

You can specify the value `latest` for the `modelVersion` to use the most recent model version available in SAP AI Core.

Step Result

You receive a unique `configurationId` in the response.

Create a Deployment

Create a deployment by sending a POST request to the endpoint `{{apiurl}}/v2/lm/deployments`.

Include the `configurationId` from the previous step in your request.

Sample Code

```
curl --location '$AI_API_URL/v2/lm/deployments' \
--header 'AI-Resource-Group: default' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
 "configurationId": "yourConfigurationId"
}'
```

Next Steps

Get Your Deployment ID

Retrieve the details of your deployment by sending a GET request to the endpoint`{{apiurl}}/v2/lm/deployments`.

```
curl --location '$AI_API_URL/v2/lm/deployments/$DEPLOYMENT_ID' \
--header 'AI-Resource-Group: default' \
--header "Authorization: Bearer $AUTH_TOKEN"
```

When the deployment is running, the model can be accessed using the `deploymentUrl` provided in the response. For more information, see Consume Models.

Model Lifecycle

Model versions have deprecation dates. Where a model version is specified in a deployment, the deployment stops working on the deprecation date of that model version.

Implement one of the following model upgrade options:

- **Auto Upgrade:** Create a generative AI configuration and deployment or patch a deployment with a new configuration, specifying `modelVersion` `latest`. When a new `modelVersion` is supported by SAP AI Core, existing generative AI deployments will automatically use the latest version of the given model.
- **Manual Upgrade:** Create a generative AI configuration with your chosen replacement `modelVersion` and use it to patch your deployment. This model version is used in generative AI deployments irrespective of updates to the models supported by SAP AI Core.

> **Note**
> If `modelVersion` isn't specified, it will be `latest` by default. To upgrade manually, you **must** specify a `modelVersion`.

Consume Models

The generative AI hub helps you to complete tasks like summarizing text, inferencing, and transforming content. To do so, you consume a generative AI model by sending a request to the model's endpoint.

Prerequisites

You have the deployment URL for your generative AI model. For more information, see Create a Deployment for a Generative AI Model.

Context

Ensure that you have the following headers set:

Header	Value
Authorization	Bearer $TOKEN
AI-Resource-Group	The resource group used in the activation steps

> **Caution**
> SAP does not take any responsibility for the quality of the content in the input to or output of the underlying generative AI models. This includes but is not limited to bias, hallucinations, or inaccuracies. The user is responsible for verifying the content.

> **Caution**
> Don't store personal data in prompts when using the generative AI hub. Personal data is any data that can be used alone, or in combination, to identify the person that the data refers to.

Procedure

Consume your generative AI deployment using your deployment URL and the example payloads provided. For more information, see the following:

- Example Payloads for Inferencing: SAP AI Core-Hosted for models with `executionIds` indicating a third party provider.
- Example Payloads for Inferencing: Remote Models for models with `executionIds` indicating that a model is hosted on SAP AI Core (beginning with `aicore-`).

Task overview: Foundation Models

Related Information

Get an Auth Token
Create a Deployment for a Generative AI Model
Foundation Model Tutorials

Example Payloads for Inferencing: `sap-rpt-1`

> **Tip**
> If you use a Windows device, use Windows PowerShell, and replace `curl` with `curl.exe`.

Ensure that you have set the following headers:

Header	Value
Authorization	Bearer $AUTH_TOKEN
AI-Resource-Group	The resource group used in the activation steps
$DEPLOYMENT_URL	The deployment URL for your generative AI model. For more information, see Create a Deployment for a Generative AI Model. Alternatively, you can replace the `$DEPLOYMENT_URL` placeholder in the curl command with your deployment URL.

The `executableId` is `aicore-sap`.

Best Practices

To achieve the best possible outputs, ensure that your requests comply with the following best practices:

- Use clean data with no or minimum missing values.
- Use descriptive column names (under 100 characters).
- Limit descriptions in text columns to 500 characters.
- Keep data types consistent within each column.
- Consider removing unnecessary columns from the prediction task.

i Note

The accuracy and fairness of the model's predictions depend on the quality and balance of the data provided. Any bias present in the input data can result in biased or skewed outputs. SAP isn't responsible or liable for any bias, inaccuracy, or unintended outcome arising from user-provided data or its characteristics.

Request Payloads

All JSON payloads must be UTF-8 encoded.

`sap-rpt-1` accepts tabular data requests in the following formats:

- Data as JSON organized by rows
- Data as JSON organized by columns
- Data in Apache Parquet file format

You can make requests using the following methods:

Request with Data by Rows

```
curl --request POST \
  --url "$DEPLOYMENT_URL/predict" \
  --header 'AI-Resource-Group: default' \
  --header "Authorization: Bearer $AUTH_TOKEN" \
  --header 'content-type: application/json' \
  --data '{
    "prediction_config": {
        "target_columns": [
            {
                "name": "COSTCENTER",
                "prediction_placeholder": "[PREDICT]",
                "task_type": "classification"
            }
        ]
    },
    "index_column": "ID",
    "rows": [
        {
            "PRODUCT": "Couch",
            "PRICE": 999.99,
            "ORDERDATE": "28-11-2025",
            "ID": "35",
            "COSTCENTER": "[PREDICT]"
        },
        {
            "PRODUCT": "Office Chair",
            "PRICE": 150.8,
            "ORDERDATE": "02-11-2025",
            "ID": "44",
            "COSTCENTER": "Office Furniture"
        },
        {
            "PRODUCT": "Server Rack",
            "PRICE": 2200.00,
            "ORDERDATE": "01-11-2025",
            "ID": "104",
            "COSTCENTER": "Data Infrastructure"
        }
    ],
```

```
    "data_schema": {
        "PRODUCT": {
            "dtype": "string"
        },
        "PRICE": {
            "dtype": "numeric"
        },
        "ORDERDATE": {
            "dtype": "date"
        },
        "ID": {
            "dtype": "string"
        },
        "COSTCENTER": {
            "dtype": "string"
        }
    }
}'
```

Request with Data by Columns

```
curl --request POST \
  --url "$DEPLOYMENT_URL/predict" \
  --header 'AI-Resource-Group: default' \
  --header "Authorization: Bearer $AUTH_TOKEN" \
  --header 'Content-Type: application/json' \
  --data '{
  "prediction_config": {
    "target_columns": [
      {
        "name": "COSTCENTER",
        "prediction_placeholder": "[PREDICT]",
        "task_type": "classification"
      }
    ]
  },
  "index_column": "ID",
  "columns": {
      "PRODUCT": ["Couch", "Office Chair", "Server Rack"],
      "PRICE": [999.99, 150.8, 2200.00],
      "ORDERDATE": ["28-11-2025", "02-11-2025", "01-11-2025"],
      "ID": ["35", "44", "104"],
      "COSTCENTER": ["[PREDICT]", "Office Furniture", "Data Infrastructure"]
  },
  "data_schema": {
      "PRODUCT": {
          "dtype": "string"
      },
      "PRICE": {
          "dtype": "numeric"
      },
      "ORDERDATE": {
          "dtype": "date"
      },
      "ID": {
          "dtype": "string"
      },
      "COSTCENTER": {
          "dtype": "string"
      }
  }
}'
```

Parameters

Parameter Description for `/predict` (JSON input)

Name	Type	Description
`prediction_config`	object	The configuration object specifying which columns to predict
`prediction_config.target_columns`	array	An array of target-column configurations for prediction
`prediction_config.target_columns[].name`	string	The name of the column to predict
`prediction_config.target_columns[].prediction_placeholder`	string	The value used as placeholder in rows where prediction is needed
`prediction_config.target_columns[].task_type`	string	(Optional) The type of prediction task Possible values: `classification` or `regression` If omitted, the model tries to automatically determine the task type that the user intends.
`index_column`	string	(Optional) The name of the column used to identify the row This column isn't used as an input feature for the model, and is returned in the response objects.

Name	Type	Description
`rows`	array	An array of objects representing table rows (both context and query rows) The keys of the objects are column names, for example `[{"product": "hammer", "price": 42.99, "creation_date": "2025-01-30"}, ...]`
`columns`	object	An object mapping from column name to array of column values. You can pass rows only or columns only, for example `{"product": ["hammer", "nail", ...], "price": [42.99, 1.98, ...], "creation_date": ["2025-01-30", "2025-01-31", ...]}`
`data_schema`	object	(Optional) A schema definition for data types of all columns, for example `{"columnA": {"dtype": "string"}, "columnB": {"dtype": "numeric"}, ...}`
`data_schema.*.dtype`	string	A data type specification. Possible values: `string`, `numeric`, or `date`
`parse_data_types`	Boolean	Relevant when `data_schema` isn't passed Whether or not to parse data types, such as interpreting strings as numbers or dates Default value: `true`

The `data_schema parameter` and `task_type` parameters are optional, but are recommended to obtain the best prediction results.

If `data_schema` is omitted, data types are inferred automatically.

If `task_type` is omitted, the model attempts to determine whether `classification` or `regression` is the intended task type.

Response Payload

The objects in `predictions` are sequence aligned with the order of the query rows in the request JSON.

If the `index_column` parameter has been passed, the contents of this column is also included in the response.

To align the response to the source data, you can sequence-align the arrays, or compare the contents of the index column where included.

Name	Type	Description
`id`	string	The generated UUID4 for this request (useful for logging)
`status`	object	The status message object indicating success of the prediction request
`status.code`	integer	The numeric status code. For more information, see Table 1.
`status.message`	string	The status message text
`predictions[].*[].prediction`	string or float	The predicted value for this column
`predictions[].*[].confidence`	float or null	(For `classification` tasks only) The confidence score for this prediction
`metadata.num_rows`	integer	The total number of input rows
`metadata.num_columns`	integer	The total number of input columns
`metadata.num_predictions`	integer	The number of table cells containing the specified placeholder values summed over all target columns
metadata.num_query_rows	integer	The number of query rows for which a prediction was made

ℹ Note

Error responses can contain an additional array called `detail`, following pydantic format. For more information, see Table 1.

Status Codes and Messages

A status code is returned to indicate success, a warning, or an error. Status codes can be any of the following:

Status Codes and Messages

Code	Message	Description
0	OK	Everything worked as expected.
1	Warning: At least one target column had invalid values in some context rows (expected float-like values for regression). These rows were dropped. Double-check your data; prediction accuracy may be impacted.	Sub optimal data/accuracy warning still returns predictions.
2	Invalid input	The input was invalid: It could contain, for example. • Malformed JSON • Missing properties • Incorrectly typed properties • Maximum restrictions exceeded
3	Internal server error	An unexpected error occurred.

Successful Prediction (200)

If the parameter `index_column` is specified in the request JSON, the following apply:

- Each prediction object in the array contains an index column. In the following examples, the index column is called `ID`.
- The column referenced as the `index_column` isn't input to the model and serves as index information in the response JSON.

Output Code

```
{
    "id": "c334f854-0d70-4c79-bd73-9ac581fd8cda",
    "status": {
        "code": 0,
        "message": "ok"
    },
    "predictions": [
        {
            "COSTCENTER": [
                {
                    "prediction": "Office Furniture",
                    "confidence": 0.96
                }
            ],
            "ID": "35"
        }
    ],
    "metadata": {
        "num_columns": 5,
        "num_rows": 2,
        "num_predictions": 1,
        "num_query_rows": 1
    }
}
```

Successful Prediction (200) with Warning

This example shows an output where the request to perform a regression contains either empty strings or null values in the regression target column.

```
{
    "id": "bf9018c4-f903-4035-a433-7058cb289447",
    "status": {
        "code": 1,
        "message": "Warning: At least one target column had invalid values in some context rows (expected float-like values for regression
    },
    "predictions": [
        {
            "salary": [
                {
                    "prediction": 52451.3984375,
                    "confidence": null
                }
            ]
        }
    ],
    "metadata": {
        "num_columns": 4,
        "num_rows": 2,
        "num_predictions": 1,
        "num_query_rows": 1
    }
}
```

Bad Request or Unprocessable Content (400 or 422)

```
{
    "status": {
        "code": 2,
        "message": "Invalid input"
    },
    "detail": [
        {
            "loc": [],
            "msg": "Too many query rows provided. Maximum is 128.",
            "type": "value_error"
        }
    ]
}
```

```
{
    "status": {
        "code": 2,
        "message": "Invalid input"
    },
    "detail": [
        {
            "loc": [
                "prediction_config",
                "target_columns",
                0,
                "prediction_placeholder"
            ],
            "msg": "Field required",
            "type": "missing"
        }
    ]
}
```

Internal Server Error (500)

Output Code

```
"status": {
   "code": 3,
   "message": "Internal server error"
}
```

Example Payloads for Inferencing: SAP AI Core-Hosted

The following examples show how you can consume various generative AI models using curl. For more information about prompts, see the tutorial Prompt LLMs in the Generative AI Hub in SAP AI Core & Launchpad.

Tip

If you use a Windows device, use Windows PowerShell, and replace `curl` with `curl.exe`.

Ensure that you have set the following headers:

Header	Value
Authorization	Bearer $AUTH_TOKEN
AI-Resource-Group	The resource group used in the activation steps
$DEPLOYMENT_URL	The deployment URL for your generative AI model. For more information, see Create a Deployment for a Generative AI Model. Alternatively, you can replace the `$DEPLOYMENT_URL` placeholder in the curl command with your deployment URL.

For more information about supported parameters, see Supported Parameters.

If you want to remove a model, delete its deployment. For more information, see Delete a Single Deployment and Delete Multiple Deployments.

Open Source

The `executableId` is `aicore-opensource`

Meta Llama 3.1 70b instruct | Mistral Mixtral 8x7b instruct v01

For more information from the model providers, see Meta Llama and Mistral .

Without streaming:

```
curl --location '$DEPLOYMENT_URL/chat/completions' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
```

```
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "model": "<ModelName>",
    "messages": [
        {
            "role": "user",
            "content": "Sample prompt"
        }
    ],
    "max_tokens": 100
}'
```

With streaming:

```
curl --location '$DEPLOYMENT_URL/chat/completions' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "model": "<ModelName>",
    "messages": [
        {
            "role": "user",
            "content": "Sample prompt"
        }
    ],
    "max_tokens": 100,
    "stream": true
}'
```

Mistral

The `executableId` is `aicore-mistralai`

For more information from the model provider, see Mistral Large and Mistral Small .

mistralai--mistral-large-instruct | mistralai--mistral-small-instruct

Text input

Without streaming:

```
curl --location '$DEPLOYMENT_URL/chat/completions' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "model": "<modelName>",
    "messages": [
        {
            "role": "user",
            "content": "Whats the difference between Accountability vs Responsibility, answer in 200 words?"
        }
    ],
    "max_tokens": 100
}'
```

Output Code

```
{
    "choices": [
        {
            "finish_reason": "length",
            "index": 0,
            "logprobs": null,
            "message": {
                "content": " Accountability and responsibility are related but distinct concepts.\n\n**Responsibility** is about the ta
                "role": "assistant",
                "tool_calls": []
            },
            "stop_reason": null
        }
    ],
    "created": 1730096376,
    "id": "<id>",
    "model": "<modelName>",
    "object": "chat.completion",
    "usage": {
        "completion_tokens": 100,
        "prompt_tokens": 23,
        "total_tokens": 123
    }
```

With streaming:

```
curl --location '$DEPLOYMENT_URL/chat/completions' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "model": "<ModelName>",
```

```
    "messages": [
        {
            "role": "user",
            "content": "Sample prompt"
        }
    ],
    "max_tokens": 100,
    "stream": true
}'
```

Image input (for mistralai--mistral-small-instruct only):

```
curl --location '$DEPLOYMENT_URL/chat/completions' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "model": "<modelName>",
    "messages": [
        {
            "role": "user",
            "content": [
                {
                    "type": "image_url",
                    "image_url": {
                        "url": "https://url/pexels-field-1-1024×683.jpg"
                    }
                }
            ]
        }
    ],
    "max_tokens": 10
}'
```

Output Code

```
{
  "choices": [
    {
      "finish_reason": "length",
      "index": 0,
      "logprobs": null,
      "message": {
        "content": "The image shows a picturesque countryside landscape, likely",
        "reasoning_content": null,
        "role": "assistant",
        "tool_calls": []
      },
      "stop_reason": null
    }
  ],
  "created": 1748573681,
  "id": "<id>",
  "model": "<modelName>",
  "object": "chat.completion",
  "prompt_logprobs": null,
  "usage": {
    "completion_tokens": 10,
    "prompt_tokens": 953,
    "prompt_tokens_details": null,
    "total_tokens": 963
  }
}
```

Note

For information on metering of image input, see Metering and Pricing for Generative AI.

mistralai--mistral-medium-instruct

For more information from the model provider, see Mistral Models .

Text Generation

```
curl --location '$DEPLOYMENT_URL/chat/completions' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "model": "mistralai--mistral-medium-instruct",
    "messages": [
            {
              "role": "user",
              "content": "Give me a JSON object with a person's name and age."
            }
    ],
    "response_format": {
      "type": "json_object"
    }
}'
```

Output Code

```
{
  "id": "<id>",
  "object": "chat.completion",
  "created": 1757916217,
  "model": "mistralai--mistral-medium-instruct",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "reasoning_content": null,
        "content": "{\"name\": \"John Doe\", \"age\": 30}",
        "tool_calls": []
      },
      "logprobs": null,
      "finish_reason": "stop",
      "stop_reason": null
    }
  ],
  "usage": {
    "prompt_tokens": 16,
    "total_tokens": 31,
    "completion_tokens": 15,
    "prompt_tokens_details": null
  },
  "prompt_logprobs": null
}
```

Tool Calling

```
curl --location '$DEPLOYMENT_URL/chat/completions' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "model": "mistralai--mistral-medium-instruct",
    "messages": [
        {
            "role": "system",
            "content": "You are a helpful assistant."
        },
        {
            "role": "user",
            "content": "What is the temperature in Boston today? Tell me in fahrenreit."
        }
    ],
    "tools": [
        {
            "type": "function",
            "function": {
                "name": "get_current_weather",
                "description": "Get the current weather in a given location",
                "parameters": {
                    "type": "object",
                    "properties": {
                        "location": {
                            "type": "string",
                            "description": "The city and state, e.g. San Francisco, CA"
                        },
                        "unit": {
                            "type": "string",
                            "enum": [
                                "celsius",
                                "fahrenheit"
                            ]
                        }
                    },
                    "required": [
                        "location"
                    ]
                }
            }
        }
    ],
    "temperature": 0.1,
    "top_p": 0.8,
    "max_tokens": 512
}'
```

Output Code

```
{
  "id": "<id>",
  "object": "chat.completion",
  "created": 1757916263,
  "model": "mistralai--mistral-medium-instruct",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "reasoning_content": null,
        "content": null,
        "tool_calls": [
          {
            "id": "RluueJ6B5",
            "type": "function",
            "function": {
              "name": "get_current_weather",
```

```
                "arguments": "{\"location\": \"Boston, MA\", \"unit\": \"fahrenheit\"}"
              }
            }
          ]
        },
        "logprobs": null,
        "finish_reason": "stop",
        "stop_reason": null
      }
    ],
    "usage": {
      "prompt_tokens": 126,
      "total_tokens": 148,
      "completion_tokens": 22,
      "prompt_tokens_details": null
    },
    "prompt_logprobs": null
  }
```

IBM

The `executableId` is `aicore-ibm`

Granite

For more information from the model provider, see IBM Granite .

ibm--granite-13b-chat

```
curl --location '$DEPLOYMENT_URL/chat/completions' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "model": "<modelName>",
    "messages": [
        {
        "role": "user",
        "content": "Whats the difference between Accountability vs Responsibility, answer in 200 words?"
        }
    ],
    "max_tokens": 100
  }'
```

Output Code

```
{
    "choices": [
        {
            "finish_reason": "length",
            "index": 0,
            "logprobs": null,
            "message": {
                "content": "Accountability and responsibility are two related but distinct concepts that are often used interchangeably
                "role": "assistant",
                "tool_calls": []
            },
            "stop_reason": null
        }
    ],
    "created": 1730097489,
    "id": "<id>",
    "model": "<modelName>",
    "object": "chat.completion",
    "usage": {
        "completion_tokens": 100,
        "prompt_tokens": 24,
        "total_tokens": 124
    }
```

NVIDIA

The `executableId` is `aicore-nvidia`

Llama

For more information from the model provider, see NVIDIA .

nvidia--llama-3.2-nv-embedqa-1b

```
curl --location '$DEPLOYMENT_URL/embeddings' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "input": ["Hello world"],
    "model": "nvidia--llama-3.2-nv-embedqa-1b",
    "input_type": "query",
```

```
    "encoding_format": "float",
    "truncate": "NONE"
}'
```

Cohere

The `executableId` is `aicore-cohere`

Reasoning

Command A Reasoning

For more information from the model provider, see Cohere Command A Reasoning .

```
curl --location "$DEPLOYMENT_URL/v2/chat" \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "model": "cohere--command-a-reasoning",
    "stream": false,
    "frequency_penalty": 0.8,
    "thinking": {
        "type": "enabled"
    },
    "messages": [
        {
            "role": "user",
            "content": "Tell me about the reflection?"
        }
    ]
}'
```

Perplexity

The `executableId` is `perplexity-ai`

Completions

sonar | sonar-pro

Sample Code

```
curl --location '$DEPLOYMENT_URL/chat/completions' \
--header 'AI-Resource-Group: default' \
--header 'Content-Type: application/json' \
--heder "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "model": "<modelName>",
    "stream": false,
    "max_tokens": 20,
    "messages": [
      {
        "role": "system",
        "content": "You are a powerful business AI assistant."
      },
      {
        "role": "user",
        "content": "Who is the president of the USA?"
      }
    ]
}'
```

Example Payloads for Inferencing: Remote Models

The following examples show how you can consume various generative AI models using curl. For more information about prompts, see the tutorial Prompt LLMs in the Generative AI Hub in SAP AI Core & Launchpad.

Tip

If you use a Windows device, use Windows Powershell, and replace `curl` with `curl.exe`.

Ensure that you have set the following headers:

Header	Value
Authorization	Bearer $AUTH_TOKEN
AI-Resource-Group	The resource group used in the activation steps
$DEPLOYMENT_URL	The deployment URL for your generative AI model. For more information, see Create a Deployment for a Generative AI Model.

Header	Value
	Alternatively, you can replace the `$DEPLOYMENT_URL` placeholder in the curl command with your deployment URL.

For more information about supported parameters, see Supported Parameters.

If you want to remove a model, delete its deployment. For more information, see Delete a Single Deployment and Delete Multiple Deployments.

Open AI

The `executableId` is `azure-openai`.

Completions

For more information from the model provider, see Microsoft Azure Open AI Chat Completions .

gpt-4-32k | gpt-4 | gpt-3.5-Turbo-16k | gpt-3.5-Turbo | o4-mini | o3 | gpt-4.1 | gpt-4.1-mini | gpt-4.1-nano | gpt-5 | gpt-5-mini | gpt-5-nano

Text Input

```
curl --location '$DEPLOYMENT_URL/chat/completions?api-version=2023-05-15' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "messages": [
        {
            "role": "user",
            "content": "sample input prompt"
        }
    ],
    "max_tokens": 100,
    "temperature": 0,
    "frequency_penalty": 0,
    "presence_penalty": 0,
    "stop": "null"
}'
```

o1 | o3-mini

```
curl --location '$DEPLOYMENT_URL/chat/completions?api-version=2024-12-01-preview' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "messages": [
      {
        "role": "user",
        "content": "Hello!"
      }
    ]
  }'
```

GPT-4o | GPT-4-Turbo | GPT-4o Mini

Image input

```
curl --location '$DEPLOYMENT_URL/chat/completions?api-version=2023-05-15' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "messages": [
        {
            "role": "user",
            "content": [
                {
                    "type": "text",
                    "text": "Describe this picture:"
                },
                {
                    "type": "image_url",
                    "image_url": {
                        "url": "https://path/images/image.png"
                    }
                }
            ]
        }
    ],
    "max_tokens": 10
}'
```

Embeddings

For more information from the model provider, see Microsoft Azure Open AI Embeddings .

text-embedding-ada-002 | text-embedding-3-small | text-embedding-3-large

```
curl --location '$DEPLOYMENT_URL/embeddings?api-version=2023-05-15' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
        "input": "sample input prompt"
}'
```

Vertex AI

The `executableId` is `gcp-vertexai`.

Gemini

For more information from the model provider, see Vertex AI Gemini .

Gemini 2.0 Flash | Gemini 2.0 Flash Lite | Gemini 2.5 Flash | Gemini 2.5 Flash Lite

Text Input

```
curl --request POST --location "$DEPLOYMENT_URL/models/<modelName>:generateContent" \ #example gemini-1.5-flash:generateContent
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "generation_config":{
        "maxOutputTokens":100
    },
    "contents": {
      "role": "user",
      "parts": {
          "text": "Give me a recipe for banana bread."
       }
      },
    "safety_settings": {
      "category": "HARM_CATEGORY_SEXUALLY_EXPLICIT",
      "threshold": "BLOCK_LOW_AND_ABOVE"
    }
}'
```

Image input

```
curl --request POST --location "$DEPLOYMENT_URL/models/<modelVersion>:generateContent" \ #example gemini-1.5-flash:generateContent
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "contents": {
        "role": "user",
        "parts": [
            {
                "fileData": {
                    "mimeType": "image/png",
                    "fileUri": "filepath/images/scones.jpg"
                }
            },
            {
                "text": "Describe this picture."
            }
        ]
    }
}'
```

Gemini Embedding

```
curl --location '$DEPLOYMENT_URL/models/gemini-embedding:predict' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
  "instances": [
    {
      "task_type": "<Task Type>",
      "title": "Behaviour Analysis",
      "content": "What is quid pro quo?"
    }
  ]
}'
```

`Task_type` can take the following values:

- `RETRIEVAL_QUERY`
- `RETRIEVAL_DOCUMENT`
- `SEMANTIC_SIMILARITY`

- `CLASSIFICATION`
- `CLUSTERING`

AWS Bedrock

The `executableId` is `aws-bedrock`.

Anthropic Claude

For more information from the model provider, see AWS Bedrock Claude .

Claude 3 Sonnet | Claude 3.5 Sonnet | Claude 3 Opus | Claude 3 Haiku | Claude 4 Sonnet | Claude 4 Opus | Claude 4.5 Haiku | Claude 4.5 Opus

Invoke:

```
curl --location '$DEPLOYMENT_URL/invoke' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "anthropic_version": "bedrock-2023-05-31",
    "max_tokens": 100,
    "messages": [
        {
            "role": "user",
            "content": "Hello, Claude"
        }
    ]
}'
```

Converse:

```
curl --location '$DEPLOYMENT_URL/converse' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "inferenceConfig": {
        "maxTokens": 100,
        "stopSequences": [
            "blab"
        ],
        "temperature": 0.7
    },
    "messages": [
        {
            "content": [
                {
                    "text": "Perplexity means?"
                }
            ],
            "role": "user"
        }
    ]
}'
```

Claude 3.7 Sonnet

```
curl --location '$DEPLOYMENT_URL/converse' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "inferenceConfig": {
        "maxTokens": 100,
        "stopSequences": [
            "blab"
        ],
        "temperature": 0.7
    },
    "messages": [
        {
            "content": [
                {
                    "text": "Perplexity means?"
                }
            ],
            "role": "user"
        }
    ]
}'
```

Amazon Titan

For more information from the model provider, see AWS Bedrock Titan .

Titan Text Express | Titan Text Lite

```
curl --location '$DEPLOYMENT_URL/invoke' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "inputText": "Who am AI?",
    "textGenerationConfig": {
        "maxTokenCount": 10,
        "stopSequences": [],
        "temperature": 0,
        "topP": 1
    }
}'
```

Titan Embed Text

```
curl --location '$DEPLOYMENT_URL/invoke' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "inputText": "Who im I?"
  }'
```

Titan Embed Image

```
curl --location '$DEPLOYMENT_URL/invoke' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "inputText": "hi",
    "inputImage": "#this-is-an-example-base64-string-value for-an-image/9j/4QDeRXhpZgAASUkqAAgAAAAGABIBAwABAAAAAQAAABoBBQABAAAAVgAAABsBBQA
    "embeddingConfig": {
        "outputEmbeddingLength": 256
    }
}'
```

Output Code

```
{
  "embedding": [
    -0.0013275146,
    -0.024658203,
    ...
    0.068359375,
    0.014404297,
    0.025512695
  ],
  "inputTextTokenCount": 3
}
```

Amazon Nova

For more information from the model provider, see AWS Bedrock Nova .

Nova Pro | Nova Lite | Nova Micro | Nova Premier

```
curl --location '$DEPLOYMENT_URL/converse' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
    "inferenceConfig": {
        "maxTokens": 100,
        "stopSequences": [
            "blab"
        ],
        "temperature": 0.7
    },
    "messages": [
        {
            "content": [
                {
                    "text": "Perplexity means?"
                }
            ],
            "role": "user"
        }
    ]
}'
```

Streaming

Where supported, streaming for Amazon Bedrock models can be invoked by replacing the deployment URL with `$DEPLOYMENT_URL/invoke-with-response-stream`.

Supported Parameters

The chat completions API supports a range of parameters that influence model responses, influencing aspects like creativity, length, and formatting of the output.

The following parameters are available:

Parameter	Type	Description
messages	array	Standard OpenAI chat completion format
model	string	Model identifier
frequency_penalty	float	Penalize frequent tokens. Range: -2.0 to 2.0
presence_penalty	float	Penalize frequent tokens. Range: -2.0 to 2.0
logit_bias	map	Modify likelihood of specified tokens
max_tokens	integer	Maximum tokens to generate
max_completion_tokens	integer	Maximum number of tokens that can be generated for a completion, including visible output tokens and reasoning tokens
n	integer	Number of completions to generate
presence_penalty	float	Penalize new tokens based on presence. Range: -2.0 to 2.0
response_format	object	Specify output format
stop	string or array	Stop sequences (up to 4)
stream	boolean	Enable streaming responses
temperature	float	Sampling temperature. Range: 0.0 to 2.0
top_logprobs	integer	The number of most likely tokens to return at each token position, with corresponding log probability. Logprobs must be set to true if this parameter is used. Range: 0 to 20
logprobs	object	Whether to return log probabilities of the output tokens or not. True returns the log probabilities of each output token
top_p	float	Nucleus sampling parameter. Range: 0.0 to 1.0
tools	array	List of tools the model may call
tool_choice	string or object	Control tool calling strategy

Foundation Model Tutorials

We provide tutorials that demonstrate use cases for accessing generative AI models in the generative AI hub using foundation scenarios. To access each model version, you'll need to create a separate deployment.

For a complete list of tutorials on foundation models, see Generative AI with SAP AI Core - Foundation Models.

Parent topic: Foundation Models

Related Information

Get an Auth Token
Create a Deployment for a Generative AI Model
Consume Models
Libraries and SDKs

Create a Deployment for Orchestration

Prerequisites

Note

It is likely that you already have an orchestration deployment running in your default resource group. These steps are only required for users who do not have an orchestration deployment or who would like to use model restriction. For more information, see Get Your Orchestration Deployment URL or Model Restriction.

- You have an SAP AI Core service instance and service key. For more information, see SAP AI Core Initial Setup Documentation.
- You're using the `extended` service plan. For more information, see Service Plans and Update a Service Plan.
- You have completed the client authorization for your preferred user interface. For more information, see Use a Service Key.

Context

You create a deployment to make orchestration capabilities available for use. After the deployment is complete, you get a `deploymentUrl`. You can use this URL across your organization to access orchestration in the generative AI hub.

For more information, see Consumption of GenAI Models Using Orchestration – A Beginner's Guide as well as Libraries and SDKs.

Procedure

1. Create a configuration for the orchestration deployment.

   Orchestration is exposed via the global scenario `orchestration` and the executable `orchestration`.

   a. Send the following request:

   ```
   curl --request POST "$AI_API_URL/v2/lm/configurations" \
     --header "Authorization: Bearer $TOKEN" \
     --header "AI-Resource-Group: $RESOURCE_GROUP" \
     --header "Content-Type: application/json" \
     --data-raw  '{
       "name": "orchestration-config",
       "executableId": "orchestration",
       "scenarioId": "orchestration"
     }'
   ```

   The response appears as follows:

   Output Code

   ```
   {
     "id": "f7ac7f77-e70f-4e9c-86b5-1504b44fe789",
     "message": "Configuration created"
   }
   ```

   b. Take the `id` of the configuration and set it as an environment variable.

   ```
   ORCH_CONFIG_ID=f7ac7f77-e70f-4e9c-86b5-1504b44fe789
   ```

2. Create the deployment for orchestration using the configuration `id`:

   a. Send the following request:

   ```
   curl --request POST $AI_API_URL/v2/lm/deployments \
       --header 'content-type: application/json' \
       --header "Authorization: Bearer $TOKEN" \
       --header "AI-Resource-Group: $RESOURCE_GROUP" \
       --data-raw "{
           \"configurationId\": \"$ORCH_CONFIG_ID\"
       }"
   ```

   The response appears as follows:

   Output Code

   ```
   {
     "deploymentUrl": "",
     "id": "d4168482710c6cf9",
     "message": "Deployment scheduled.",
     "status": "UNKNOWN"
   }
   ```

   b. Take the `id` of the deployment response and set it as an environment variable.

   ```
   ORCH_DEPLOYMENT_ID=d4168482710c6cf9
   ```

3. Wait for the deployment to start.

   It takes a few minutes for the orchestration deployment to reach the status RUNNING. You can check the status of the deployment using the deployment `id`.

   ```
   curl --request GET "$AI_API_URL/v2/lm/deployments/$ORCH_DEPLOYMENT_ID" \
       --header "Authorization: Bearer $TOKEN" \
       --header "AI-Resource-Group: $RESOURCE_GROUP"
   ```

   If the output looks as follows (that is, the status is RUNNING), the deployment is ready. If not, try again after a couple of minutes.

   ```
   {
     "configurationId": "f7ac7f77-e70f-4e9c-86b5-1504b44fe789",
     "configurationName": "orchestration-config",
     "createdAt": "2024-06-20T14:40:16Z",
     "deploymentUrl": ,
     "details": {
       "resources": {
         "backend_details": {}
       },
       "scaling": {
         "backend_details": {}
       }
     },
     "id": "d4468482710c6cf9",
     "lastOperation": "CREATE",
     "latestRunningConfigurationId": "f7ac7f77-e70f-4e9c-86b5-1504b44fe789",
     "modifiedAt": "2024-06-20T14:48:03Z",
   ```

```
    "scenarioId": "llm-orchestration",
    "startTime": "2024-06-20T14:41:22Z",
    "status": "RUNNING",
    "submissionTime": "2024-06-20T14:40:18Z",
    "targetStatus": "RUNNING"
  }
```

4. Set the value of the returned `deploymentUrl` as an environment variable.

   The `deploymentUrl` is available only when the deployment has the status RUNNING.

```
ORCH_DEPLOYMENT_URL="<deployment_url>"
```

Prompt Registry

Manage the life cycle of your prompts, from design to runtime.

Prompt registry integrates prompt templates into SAP AI Core, making them discoverable across your applications and orchestration.

It reduces the complexity of dealing with prompt templates and leveraging integration capabilities.

The You can use prompt registry to manage the lifecycle of the following:

- Prompt templates. For more information, see Prompt Template Management.
- Orchestration configs. For more information, see Orchestration Config Management.

Prompt Template Management

Manage the life cycle of your prompts, from design to runtime.

Prompt registry integrates prompt templates into SAP AI Core, making them discoverable across your applications and orchestration.

It reduces the complexity of dealing with prompt templates and leveraging integration capabilities.

The Prompt Registry consists of the following interfaces:

- The imperative API supports full CRUD operations and is recommended for refining prompt templates in design time use cases. Iterations are tracked and can be viewed in the history endpoint. For more information, see Create a Prompt Template (Imperative).
- The declarative API utilizes SAP AI Core applications and is recommended for runtime application use cases and CI/CD pipelines. You can manage your prompt templates in a git repository declaratively. Your commits are automatically synchronized with the Prompt Registry For more information, see Create a Prompt Template (Declarative).

The interfaces can all be consumed either directly via the API or through Orchestration.

Create a Prompt Template (Imperative)

Context

→ Recommendation

The imperative API supports full CRUD operations and is recommended for refining prompt templates in design-time use cases. Iterations are tracked and can be viewed in the history endpoint.

You can create a reusable prompt for a specific use case, including placeholders that are filled later.

By default, prompt templates are handled on the main-tenant level. To handle them on resource-group level, add the `AI-Resource-Group-Scope` and `AI-Resource-Group` headers to your requests. For example:

```
--header 'AI-Resource-Group-Scope: true'
--header 'AI-Resource-Group: <resource group>'
```

ℹ Note

For CaaS users, the resource-group-id is extracted automatically, and doesn't need to be provided in the headers.

ℹ Note

CaaS users only have write access to prompt templates on a resource-group level.

Procedure

Create a prompt template by sending a POST request to endpoint `{{apiurl}}/v2/lm/promptTemplates`.

Sample Code

```
curl -X POST "{{apiurl}}/v2/lm/promptTemplates"\
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $TOKEN" \
--data '{
    "name": "example-prompt-template",
    "version": "0.0.1",
    "scenario": "categorization",
    "spec": {
      "template": [
        {
          "role": "system",
          "content": "You classify input text into the two following categories: {{?categories}}"
        },
        {
          "role": "user",
          "content": "{{?inputExample}}"
        }
      ],
      "defaults": {
        "categories": "Finance, Tech, Sports"
      },
      "additional_fields": {
        "modelParams": {
          "temperature": 0.7,
          "max_tokens": 100
        },
        "modelGroup": "chat"
      }
    }
  }'
```

- The `defaults` and `additional_fields` fields are optional.
- The `additional_fields` field is unstructured and can be used to store metadata or configuration objects.

Output Code

```
{
  "id": "9y02-ha9x-92b1-4255",
  "name": "example-prompt-template",
  "versionId": "0.0.1",
  "message": "Prompt created successfully."
}
```

Next Steps

You can iterate over the prompt template and make changes to it.

You can save your changes as a new version. The latest iteration is always the head version and is marked with `isVersionHead: true`.

Tip

You'll need to update the version number when you want to create a new version, and the version entry must be compliant with semantic versioning (SemVer).

You can check the change history for the template. For more information, see Get Prompt Template History.

Create a Prompt Template (Declarative)

Prerequisites

- You've created and synced a git repository. For more information, see Add a Git Repository.
- You've created an application that points to the prompt templates in your git repository. For more information, see Create an Application.

Context

Recommendation

The declarative API utilizes SAP AI Core applications and is recommended for runtime application use cases and CI/CD pipelines. You can manage your prompt templates in a git repository declaratively. Your commits are automatically synchronized with the prompt registry.

i Note

Declarative prompt templates are always handled on the main-tenant level.

Procedure

Create a prompt template and push it to your git repository. The file name must be in the format `<name>.promptemplate.ai.sap.yaml` to be recognized.

The following formats are supported:

- yaml

Sample Code

```yaml
name: simple
version: 0.0.1
scenario: my-scenario
spec:
  template:
    - role: "system"
      content: "{{ ?instruction }}"
    - role: "user"
      content: "Some more {{ ?user_input }}"
  defaults:
    instruction: "default instruction"
  additional_fields:
    isDev: true
    validations:
      required: true
    blockedModels:
      - name: "gpt-4"
        versions: "gpt-4-vision"
      - name: "gpt-4o"
        versions: "*"
```

- The `defaults` and `additional_fields` fields are optional.
- The `additional_fields` field is unstructured and can be used to store metadata or configuration objects.

Your template syncs automatically. After a few minutes, you'll be able to verify your template by sending a GET request to the endpoint `{{apiurl}}/v2/lm/promptTemplates`.

Sample Code

```jsonc
{
    "count": 3,
    "resources": [
        {
            "id": "a460d210-df38-4867-9535-7a556701a4b0",
            "name": "simple",
            "version": "0.0.1",
            "scenario": "my-scenario",
            "creationTimestamp": "2024-08-18T14:50:17.157000",
            "managedBy": "declarative",
            "isVersionHead": true
        },
        {...}
    ]
}
```

The prompt template is marked as `managedBy`: *<declarative>*. Declarative managed prompt templates are always the head version and can't be edited with the imperative API.

Next Steps

You use your prompt template at runtime. For more information, see [Use a Prompt Template](#).

You can change templates locally and push them to git.

→ Tip

For declaratively managed templates, use the history capabilities in git to track your changes.

Get a Prompt Template

Prerequisites

- You have created a prompt template. For more information, see Create a Prompt Template (Imperative) and Create a Prompt Template (Declarative).

Context

You can retrieve a prompt template by ID, or by the combination of name, scenario, and version.

Prompt templates can also be retrieved and consumed in orchestration. For more information, see Templating.

> **Note**
> Retrieval by ID offers immutability; guaranteeing that the prompt template behind the ID will not change.
>
> Retrieval by name, scenario, and version are not immutable, and the latest iteration of the prompt template is retrieved.

By default, prompt templates are handled on the main-tenant level. To handle them on resource-group level, add the `AI-Resource-Group-Scope` and `AI-Resource-Group` headers to your requests. For example:

```
--header 'AI-Resource-Group-Scope: true'
--header 'AI-Resource-Group: <resource group>'
```

> **Note**
> For CaaS users, the resource-group-id is extracted automatically, and doesn't need to be provided in the headers.

Procedure

Send a GET request to the endpoint `{{apiurl}}/v2/lm/promptTemplates`, and include the information for your chosen retrieval method.

Sample Code

```
curl -X GET "{{apiurl}}/v2/lm/promptTemplates?name=example-prompt-template&scenario=categorization&version=0.0.1" \
  -Header "Authorization: Bearer <your_auth_token>"
```

Sample Code

```
curl -X GET "{{apiurl}}/v2/lm/promptTemplates/{{promptTemplateId}}" \
-Header "Authorization: Bearer <your_auth_token>"
```

Get Prompt Template History

Context

You can list the history of edits to prompt templates, for imperatively managed prompt templates only.

By default, prompt templates are handled on the main-tenant level. To handle them on resource-group level, add the `AI-Resource-Group-Scope` and `AI-Resource-Group` headers to your requests. For example:

```
--header 'AI-Resource-Group-Scope: true'
--header 'AI-Resource-Group: <resource group>'
```

> **Note**
> For CaaS users, the resource-group-id is extracted automatically, and doesn't need to be provided in the headers.

Procedure

Send a GET request to endpoint `{{apiurl}}/v2/lm/scenarios/{{scenarioId}}/promptTemplates/{{promptTemplateName}}/versions/{{versionId}}/history` and include the name, scenario, and version of your prompt template.

Sample Code

```
curl -X GET "{{apiurl}}/v2/lm/scenarios/{{scenarioId}}/promptTemplates/{{promptTemplateName}}/versions/{{versionId}}/history" \
```

```
-Header "Authorization: Bearer <your_auth_token>"
```

Output Code

```
{
  "count": 10,
  "resources": [
    {
      "id": "8y02-ha9x-92b1-4255",
      "name": "example-prompt-template",
      "version": "0.0.1",
      "scenario": "categorization",
      "managedBy": "imperative",
      "isVersionHead": true,
      "creationTimestamp": "2021-09-29T14:00:00Z",
      "spec": {
        "template": [
          {
            "role": "system",
            "content": "You classify input text into the two following categories: {{?categories}}"
          },
          {
            "role": "user",
            "content": "{{?inputExample}}"
          }
        ],
        "defaults": {
          "categories": "Finance, Tech, Sports"
        },
        "additional_fields": {
          "modelParams": {
            "temperature": 0.7,
            "max_tokens": 100
          },
          "modelGroup": "chat"
        }
      }
    }
  ]
}
```

Use a Prompt Template

Prerequisites

- You have created a prompt template. For more information, see Create a Prompt Template (Imperative).

Context

By default, prompt templates are handled on the main-tenant level. To handle them on resource-group level, add the `AI-Resource-Group-Scope` and `AI-Resource-Group` headers to your requests. For example:

```
--header 'AI-Resource-Group-Scope: true'
--header 'AI-Resource-Group: <resource group>'
```

Note

For CaaS users, the resource-group-id is extracted automatically, and doesn't need to be provided in the headers.

Note

CaaS users only have write access to prompt templates on a resource-group level.

Procedure

You can fill a prompt template by ID, or by the combination of name, scenario, and version.

- To fill a prompt template by ID, send a POST request to the endpoint `{{apiurl}}/v2/lm/promptTemplates/{{promptTemplateId}}/substitution` and add your variable values in the body.

- **Sample Code**

```
curl -X POST "{{apiurl}}/v2/lm/promptTemplates/{{promptTemplateId}}/substitution" \
     -H "Authorization: Bearer <your_auth_token>" \
     -H "Content-Type: application/json" \
     -d '{
           "inputParams": {
             "inputExample": "Sample text"
           }
         }'
```

- To fill a prompt template by name, scenario, and version send a POST request to the endpoint `{{apiurl}}/v2/lm/scenarios/{{scenarioId}}/promptTemplates/{{promptTemplateName}}/versions/{{versionId}}/substitution` and add your variable values in the body.

  Sample Code

```
curl -X POST "{{apiurl}}/v2/lm/scenarios/{{scenarioId}}/promptTemplates/{{promptTemplateName}}/versions/{{versionId}}/subs
     -H "Authorization: Bearer <your_auth_token>" \
     -H "Content-Type: application/json" \
     -d '{
           "inputParams": {
             "inputExample": "Sample text"
           }
         }'
```

To return a response containing the full prompt template definition, include the optional query parameter `metadata = <true>`

Import a Prompt Template

Prerequisites

- You have created and exported a prompt template. For more information, see Create a Prompt Template (Imperative) and Export a Prompt Template.

Context

You can import a declarative prompt template as a single file export in yaml format.

By default, prompt templates are handled on the main-tenant level. To handle them on resource-group level, add the `AI-Resource-Group-Scope` and `AI-Resource-Group` headers to your requests. For example:

```
--header 'AI-Resource-Group-Scope: true'
--header 'AI-Resource-Group: <resource group>'
```

Note

For CaaS users, the resource-group-id is extracted automatically, and doesn't need to be provided in the headers.

Note

CaaS users only have write access to prompt templates on a resource-group level.

Procedure

Send a POST request to the endpoint `{{apiurl}}/v2/lm/promptTemplates/import`, and include the name of your file in the `file` parameter.

Sample Code

```
curl -X POST "{{apiurl}}/v2/lm/promptTemplates/import" \
     -Header "Authorization: Bearer <your_auth_token>" \
     -Header "Content-Type: multipart/form-data" \
     -File "file=@/path/to/your/file.yaml"
```

Output Code

```
{
  "message": "Prompt <prompt-name> for version <version-number> imported successfully.",
  "resource": {
    "name": "example-prompt-template",
```

```
      "version": "0.0.1",
      "scenario": "job-description",
      "spec": {
        "template": [
          {
            "role": "system",
            "content": "You classify input text into the two following categories: {{?categories}}"
          },
          {
            "role": "user",
            "content": "{{?inputExample}}"
          }
        ],
        "defaults": {
          "categories": "Finance, Tech, Sports"
        },
        "additional_fields": {
          "modelParams": {
            "temperature": 0.7,
            "max_tokens": 100
          },
          "modelGroup": "chat"
        }
      }
    }
  }
```

Export a Prompt Template

Prerequisites

- You have created a prompt template. For more information, see Create a Prompt Template (Imperative).

Context

You can export a prompt template as a single file export in declarative compatible yaml format.

By default, prompt templates are handled on the main-tenant level. To handle them on resource-group level, add the `AI-Resource-Group-Scope` and `AI-Resource-Group` headers to your requests. For example:

```
--header 'AI-Resource-Group-Scope: true'
--header 'AI-Resource-Group: <resource group>'
```

Note

For CaaS users, the resource-group-id is extracted automatically, and doesn't need to be provided in the headers.

Procedure

Send a GET request to the endpoint `{{apiurl}}/v2/lm/promptTemplates/{{promptTemplateId}}/export`, and include the name of your file in the `output` parameter.

Sample Code

```
curl -X GET "{{apiurl}}/v2/lm/promptTemplates/{{promptTemplateId}}/export" \
-Header "Authorization: Bearer <your_auth_token>" \
--output "exported_prompt_template.yaml"
```

Output Code

```
name: example-prompt-template
version: 0.0.1
scenario: job-description
spec:
    template:
        - role: 'system'
        content: 'You classify input text into the two following categories: {{?categories}}'
        - role: 'user'
        content: '{{?inputExample}}'
    defaults:
        categories: Finance, Tech, Sports
```

```
    additional_fields:
        modelParams:
            temperature: 0.7
            max_tokens: 100
        modelGroup: chat
        collection: abc
```

Delete a Prompt Template

Prerequisites

- You have created a prompt template. For more information, see Create a Prompt Template (Imperative).
- You know the template ID of the prompt template that you want to delete. For more information, see Get a Prompt Template.

Context

Delete a specific version of the prompt template, for imperatively managed prompt templates only.

By default, prompt templates are handled on the main-tenant level. To handle them on resource-group level, add the `AI-Resource-Group-Scope` and `AI-Resource-Group` headers to your requests. For example:

```
--header 'AI-Resource-Group-Scope: true'
--header 'AI-Resource-Group: <resource group>'
```

Note

For CaaS users, the resource-group-id is extracted automatically, and doesn't need to be provided in the headers.

Note

CaaS users only have write access to prompt templates on a resource-group level.

Procedure

Send a DELETE request to the endpoint `{{apiurl}}/v2/lm/promptTemplates/{{promptTemplateId}}` and include the template ID of the prompt you want to delete.

Output Code

```
{
  "message": "Prompt deleted successfully."
}
```

Orchestration Config Management

Manage the lifecycle of your orchestration configurations, from design to runtime.

You can store, version, and manage orchestration configs as reusable assets in the prompt registry. This makes them discoverable across your applications and orchestration workflows, enabling better governance and reusability.

Stored orchestration configs reduce the complexity of managing orchestration configurations by providing lifecycle management capabilities including versioning, history tracking, and import and export functionality.

The orchestration config endpoints are imperative API operations that support full CRUD functionality for both design-time and runtime use cases. Iterations are tracked and can be viewed in the history endpoint.

Orchestration configs can reference existing prompt templates by ID or by name, scenario, and version, allowing you to build reusable orchestration workflows that leverage your prompt template library.

Stored orchestration configs can be referenced in LLM Orchestration completion requests, providing an alternative to inline configuration definitions.

Note

Orchestration configs are only available through the imperative API and are managed at the tenant level.

Create an Orchestration Config

Context

The imperative API supports full CRUD operations and is recommended for both design-time and runtime orchestration configuration use cases. Iterations are tracked and can be viewed in the history endpoint.

You can create a reusable orchestration configuration for a specific use case, including module configurations for filtering, masking, grounding, and prompt templating.

Orchestration configs are handled at the tenant level and do not require resource group headers.

Orchestration configs can reference existing prompt templates in two ways:

- By template ID for immutable references
- By name, scenario, and version for flexible references that can point to the latest version

Procedure

Create an orchestration config by sending a POST request to endpoint `{{apiurl}}/v2/registry/v2/orchestrationConfigs`.

Set the following in the headers:

- `AI_API_URL`: the base URL of your SAP AI Core environment.
- `{{access_token}}`: Your access token for SAP AI Core

The following examples show only the `prompt_templating` module for simplicity. Orchestration configs can include all supported modules such as `filtering`, `masking`, `grounding`, and `translation` in addition to `prompt_templating`.

Note

Use template references by ID when you need immutable references that won't change. Use references by name, scenario, and version when you want flexibility to update the underlying template while keeping the same orchestration config.

Example With Inline Prompt Template

Sample Code

```
curl -X POST "{{apiurl}}/v2/registry/v2/orchestrationConfigs" \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $TOKEN" \
--data '{
    "name": "example-orchestration-config",
    "version": "0.0.1",
    "scenario": "customer-support",
    "spec": {
      "modules": {
        "prompt_templating": {
          "prompt": {
            "template": [
              {
                "role": "system",
                "content": "You are a helpful customer support assistant."
              },
              {
                "role": "user",
                "content": "{{?userQuery}}"
              }
            ],
            "defaults": {
              "userQuery": "How can I help you today?"
            }
          },
          "model": {
            "name": "<model>",
            "params": {
              "temperature": 0.7,
              "max_tokens": 500
            }
          }
        }
      }
    }
  }'
```

Example With Prompt Template Reference by ID

Sample Code

```
curl -X POST "{{apiurl}}/v2/registry/v2/orchestrationConfigs" \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $TOKEN" \
--data '{
    "name": "example-orchestration-config",
    "version": "0.0.1",
    "scenario": "customer-support",
    "spec": {
      "modules": {
        "prompt_templating": {
          "prompt": {
            "template_ref": {
              "id": "<templateId>"
```

```
              }
            },
            "model": {
              "name": "model",
              "params": {
                "temperature": 0.7,
                "max_tokens": 500
              }
            }
          }
        }
      }
    }'
```

Example With Scenario, Template Name, and Version Combination

Sample Code

```
curl -X POST "{{apiurl}}/v2/registry/v2/orchestrationConfigs" \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $TOKEN" \
--data '{
    "name": "example-orchestration-config",
    "version": "0.0.1",
    "scenario": "customer-support",
    "spec": {
      "modules": {
        "prompt_templating": {
          "prompt": {
            "template_ref": {
              "scenario": "categorization",
              "name": "example-prompt-template",
              "version": "0.0.1"
            }
          },
          "model": {
            "name": "<model>",
            "params": {
              "temperature": 0.7,
              "max_tokens": 500
            }
          }
        }
      }
    }
  }'
```

Results

You will receive an output JSON containing details about your configuration, including an ID and success message.

Output Code

```
{
  "id": "<orchestrationConfigId>",
  "name": "example-orchestration-config",
  "version": "0.0.1",
  "scenario": "customer-support",
  "message": "Orchestration config created successfully."
}
```

Next Steps

You can iterate over the orchestration config and make changes to it.

You can save your changes as a new version. To create a new version, update the version number. Your entry must be compliant with semantic versioning (SemVer). The latest iteration is always the head version and is marked with `is_version_head: true`.

You can check the change history for the config. For more information, see Get Orchestration Configs.

Get Orchestration Configs

Prerequisites

You have created an orchestration config. For more information, see Create an Orchestration Config.

Context

You can list all orchestration configs or retrieve a specific orchestration config by ID, or by the combination of name, scenario, and version.

Orchestration configs can also be retrieved and consumed in orchestration completion requests. For more information, see Use an Orchestration Config in LLM Orchestration.

Note

Retrieval by ID offers immutability; guaranteeing that the orchestration config behind the ID will not change.

Retrieval by name, scenario, and version are not immutable, and the latest iteration of the orchestration config is retrieved.

Orchestration configs are handled at the tenant level and do not require resource group headers.

Procedure

Send a GET request to the endpoint `{{apiurl}}/v2/registry/v2/orchestrationConfigs`, and include the information for your chosen retrieval method.

Set the following in the headers:

- `AI_API_URL`: the base URL of your SAP AI Core environment.
- `{{access_token}}`: Your access token for SAP AI Core

List All Orchestration Configs

Sample Code

```
curl -X GET "{{apiurl}}/v2/registry/v2/orchestrationConfigs" \
  -H "Authorization: Bearer <your_auth_token>"
```

Get by Name, Scenario, and Version

Sample Code

```
curl -X GET "{{apiurl}}/v2/registry/v2/orchestrationConfigs?name=example-orchestration-config&scenario=customer-support&version=0.0.1" \
  -H "Authorization: Bearer <your_auth_token>"
```

Get by ID

Sample Code

```
curl -X GET "{{apiurl}}/v2/registry/v2/orchestrationConfigs/{{orchestrationConfigId}}" \
-H "Authorization: Bearer <your_auth_token>"
```

Tip

Use the `resolve_template_ref=true` query parameter to resolve template references and include the full template definition in the response.

Results

Output Code

```
{
  "count": 3,
  "resources": [
    {
      "id": "<orchestrationConfigId1>",
      "name": "example-orchestration-config",
      "version": "0.0.1",
      "scenario": "customer-support",
      "creation_timestamp": "2024-08-18T14:50:17.157000",
      "managed_by": "imperative",
      "is_version_head": true
    },
    {
      "id": "<orchestrationConfigId2>",
      "name": "another-orchestration-config",
      "version": "1.0.0",
      "scenario": "data-analysis",
      "creation_timestamp": "2024-08-19T10:30:45.123000",
      "managed_by": "imperative",
      "is_version_head": true
    }
  ]
}
```

Get Orchestration Config History

Context

You can list the history of edits to orchestration configs for imperatively managed configurations.

Orchestration configs are handled at the tenant level and do not require resource group headers.

Procedure

Send a GET request to endpoint `{{apiurl}}/v2/registry/v2/scenarios/{{scenarioId}}/orchestrationConfigs/{{orchestrationConfigName}}/versions/{{versionId}}/history` and include the name, scenario, and version of your orchestration config.

Set the following in the headers:

- `AI_API_URL`: the base URL of your SAP AI Core environment.
- `{{access_token}}`: Your access token for SAP AI Core

Sample Code

```
curl -X GET "{{apiurl}}/v2/registry/v2/scenarios/{{scenarioId}}/orchestrationConfigs/{{orchestrationConfigName}}/versions/{{versionId}},
  -H "Authorization: Bearer <your_auth_token>"
```

Output Code

```
{
  "count": 5,
  "resources": [
    {
      "id": "<orchestrationConfigId>",
      "name": "example-orchestration-config",
      "version": "0.0.1",
      "scenario": "customer-support",
      "managed_by": "imperative",
      "is_version_head": true,
      "creation_timestamp": "2021-09-29T14:00:00Z",
      "spec": {
        "modules": {
          "prompt_templating": {
            "prompt": {
              "template_ref": {
                "id": "<promptTemplateId>"
              }
            },
            "model": {
              "name": "<model>",
              "params": {
                "temperature": 0.7,
                "max_tokens": 500
              }
            }
          }
        }
      }
    }
  ]
}
```

Use an Orchestration Config in LLM Orchestration

Prerequisites

You have created an orchestration config. For more information, see Create an Orchestration Config.

Context

Orchestration configs can be used in LLM Orchestration completion requests in the following ways:

1. **Inline configuration**: Define the configuration directly in the request
2. **Reference by ID**: Reference a stored config by its unique ID
3. **Reference by name, scenario, and version**: Reference a stored config by its metadata

Procedure

You can reference orchestration configs using the following methods:

Set the following in the headers:

- `AI_API_URL`: the base URL of your SAP AI Core environment.
- `{{access_token}}`: Your access token for SAP AI Core

Method 1: Reference Orchestration Config by ID

Send a POST request to the LLM Orchestration completion endpoint with a config reference by ID.

```
curl -X POST "{{apiurl}}/v2/completion" \
     -H "Authorization: Bearer <your_auth_token>" \
     -H "Content-Type: application/json" \
     -d '{
           "config_ref": {
             "id": "<orchestrationConfigId>"
           },
           "placeholder_values": {
             "userQuery": "What are the benefits of SAP AI Core?",
             "inputContext": "enterprise AI platform"
           }
         }'
```

Method 2: Reference Orchestration Config by Name, Scenario, and Version

Send a POST request to the LLM Orchestration completion endpoint with a config reference by metadata.

```
curl -X POST "{{apiurl}}/v2/completion" \
     -H "Authorization: Bearer <your_auth_token>" \
     -H "Content-Type: application/json" \
     -d '{
           "config_ref": {
             "scenario": "customer-support",
             "name": "example-orchestration-config",
             "version": "0.0.1"
           },
           "placeholder_values": {
             "userQuery": "How do I integrate with SAP AI Core?",
             "inputContext": "API integration"
           },
           "messages_history": [
             {
               "role": "system",
               "content": "You are helping with technical integration questions."
             }
           ]
         }'
```

Method 3: Inline Configuration (for comparison)

You can also use orchestration configurations inline without storing them first.

```
curl -X POST "{{apiurl}}/v2/completion" \
     -H "Authorization: Bearer <your_auth_token>" \
     -H "Content-Type: application/json" \
     -d '{
           "config": {
             "modules": {
               "prompt_templating": {
                 "prompt": {
                   "template": [
                     {
                       "role": "user",
                       "content": "{{?userQuery}}"
                     }
                   ]
                 },
                 "model": {
                   "name": "<model>"
                 }
               }
             }
           },
           "placeholder_values": {
             "userQuery": "What is SAP AI Core?"
           }
         }'
```

Note

Use stored orchestration configs (methods 1 and 2) when you have reusable configurations that multiple applications or workflows need to access. Use inline configurations (method 3) for one-off or experimental requests.

Note

When referencing orchestration configs in LLM Orchestration:

- Reference by ID provides immutability meaning that the config behind the ID will never change
- Reference by name, scenario, and version provides flexibility meaning that you get the latest iteration of that version

Export an Orchestration Config

Prerequisites

You have created an orchestration config. For more information, see Create an Orchestration Config.

Context

You can export an orchestration config as a single file export in YAML format.

Orchestration configs are handled at the tenant level and do not require resource group headers.

Procedure

Send a GET request to the endpoint `{{apiurl}}/v2/registry/v2/orchestrationConfigs/{{orchestrationConfigId}}/export`, and include the name of your file in the output parameter.

Set the following in the headers:

- `AI_API_URL`: the base URL of your SAP AI Core environment.
- `{{access_token}}`: Your access token for SAP AI Core

Sample Code

```
curl -X GET "{{apiurl}}/v2/registry/v2/orchestrationConfigs/{{orchestrationConfigId}}/export" \
-H "Authorization: Bearer <your_auth_token>" \
--output "exported_orchestration_config.yaml"
```

Output Code

```
name: example-orchestration-config
version: 0.0.1
scenario: customer-support
spec:
  modules:
    prompt_templating:
      prompt:
        template_ref:
          id: <promptTemplateId>
      model:
        name: <model>
        params:
          temperature: 0.7
          max_tokens: 500
```

Import an Orchestration Config

Prerequisites

You have created and exported an orchestration config. For more information, see Create an Orchestration Config and Export an Orchestration Config.

Context

You can import an orchestration config as a single file export in YAML format.

Orchestration configs are handled at the tenant level and do not require resource group headers.

Procedure

Send a POST request to the endpoint `{{apiurl}}/v2/registry/v2/orchestrationConfigs/import`, and include the name of your file in the file parameter.

Set the following in the headers:

- `AI_API_URL`: the base URL of your SAP AI Core environment.
- `{{access_token}}`: Your access token for SAP AI Core

Sample Code

```
curl -X POST "{{apiurl}}/v2/registry/v2/orchestrationConfigs/import" \
     -H "Authorization: Bearer <your_auth_token>" \
     -H "Content-Type: multipart/form-data" \
     -F "file=@/path/to/your/orchestration-config.yaml"
```

Output Code

```
{
  "message": "Orchestration config example-orchestration-config for version 0.0.1 imported successfully.",
  "id": "<orchestrationConfigId>",
  "name": "example-orchestration-config",
  "version": "0.0.1",
  "scenario": "customer-support"
}
```

Delete an Orchestration Config

Prerequisites

You have created an orchestration config. For more information, see Create an Orchestration Config.

You know the config ID of the orchestration config that you want to delete. For more information, see Get Orchestration Configs.

Context

Delete a specific version of the orchestration config for imperatively managed configurations.

Orchestration configs are handled at the tenant level and do not require resource group headers.

Procedure

Send a DELETE request to the endpoint `{{apiurl}}/v2/registry/v2/orchestrationConfigs/{{orchestrationConfigId}}` and include the config ID of the orchestration config you want to delete.

Set the following in the headers:

- `AI_API_URL`: the base URL of your SAP AI Core environment.
- `{{access_token}}`: Your access token for SAP AI Core

Sample Code

```
curl -X DELETE "{{apiurl}}/v2/registry/v2/orchestrationConfigs/{{orchestrationConfigId}}" \
-H "Authorization: Bearer <your_auth_token>"
```

Output Code

```
{
  "message": "Orchestration config deleted successfully."
}
```

Optimizations

Optimizations lets you evaluate and refine your generative AI workflows and prompts.

Key features of optimizations include:

- **Evaluations:** Systematically assess and validate AI model performance using evaluation workflows and metrics. Manage the complete lifecycle of your evaluation processes and analyze results to ensure your AI solutions meet quality standards before deployment. For more information, see Evaluations.
- **Prompt Optimization:** Enhance prompt effectiveness by optimizing templates against target datasets to maximize specified performance metrics. Leverage prompt registry integration and systematic testing to improve model outputs and achieve better alignment with business objectives. For more information, see Prompt Optimization.
- **Metrics:** Monitor model performance using built-in and custom indicators to ensure your AI applications meet business requirements. For more information, see Metrics.

Register an Object Store for Optimizations

Context

Restriction

You must create an **object store secret** named **default** to store optimization output artifacts.

For **input artifacts** only, you may create multiple object store secrets with different names as needed.

Procedure

To create an object store secret for your object store, send a POST request to the endpoint `$AI_API_URL/v2/admin/objectStoreSecrets` and include your credentials in the body of your request.

Note

For all storage types **except** Azure Blob Storage, all *<data>* fields are required. For Azure, required fields are specified.

- For Amazon S3:

```
curl --location --request POST "$AI_API_URL/v2/admin/objectStoreSecrets" \
--header "Authorization: Bearer $TOKEN" \
--header 'Content-Type: application/json' \
--header 'AI-Resource-Group: <Resource group>' \
--data-raw '{
        "name": "default",
        "type": "S3",
        "bucket": "<S3 bucket name>",
        "endpoint": "<S3 end point>",
        "pathPrefix": "<A path prefix that follows the bucket name>",
        "region": "<S3 region>",
        "data": {
              "AWS_ACCESS_KEY_ID": "<AWS access key ID>",
              "AWS_SECRET_ACCESS_KEY": "<AWS secret access key>"
        }
    }'
```

- For OSS (Alicloud Object Storage Service):

```
curl --location --request POST "$AI_API_URL/v2/admin/objectStoreSecrets" \
--header "Authorization: Bearer $TOKEN" \
--header 'Content-Type: application/json' \
--header 'AI-Resource-Group: <Resource group>' \
--data-raw '{
        "name": "default",
        "type": "oss",
        "pathPrefix": "<path prefix to be appended with bucketname>",
        "data": {
             "BUCKET": "<bucket-name>",
             "ENDPOINT": "oss-cn-shanghai.aliyuncs.com",
             "REGION": "",
             "ACCESS_KEY_ID": "xxxxx",
             "SECRET_ACCESS_KEY": "xxxxx"
         }
}
```

- For Azure Blob Storage:

```
curl --location --request POST "$AI_API_URL/v2/admin/objectStoreSecrets" \
--header "Authorization: Bearer $TOKEN" \
--header 'Content-Type: application/json' \
--header 'AI-Resource-Group: <Resource group>' \
--data-raw '{
                "name": "default",
                "type": "azure",
                "pathPrefix": "<path prefix to be appended>",
                "data": {
                        "CONTAINER_URI": "https://account_name.blob.core.windows.net/container_name",  //required
                        "REGION": "<region name>",                  //optional
                        "CLIENT_ID": "<azure client id>",           //optional
                        "CLIENT_SECRET": "<azure client secret>",   //optional
                        "STORAGE_ACCESS_KEY": "sas_token",          //required
                        "TENANT_ID": "azure tenant id",             //optional
                        "SUBSCRIPTION_ID": "subscription id",       //optional
        }
}
```

→ Tip

The `pathPrefix` is useful if you share the same bucket for different projects. You can set the name of your project folder to `my-ml-project1`, for example. All data is then stored in that folder.

ℹ Note

If the `AI-Resource-Group` header isn't specified, the *<Resource Group>* is assigned the value `"default"` automatically.

Evaluations

Evaluations provides tools for benchmarking large language models and prompts via orchestration configurations.

Evaluations can be used for the following use cases:

- Evaluating prompt templates and models as orchestration configurations to determine the most effective combination for a use case.

- Use industry-standard predefined metrics for model and prompt comparison with your use-case-specific dataset.
- Use your own custom-defined metrics for your prompt and model evaluations.

Evaluations Overview

Variable Mapping

Variable mapping enables you to align variable names between orchestration configurations, metric evaluators, and test datasets, ensuring correct data flow even when attribute names differ. Use this feature to resolve naming mismatches and maintain consistency in automated testing and evaluation workflows.

Variable mappings map properties in the orchestration configuration or response and metric evaluators to attributes in the test data set. By default, variable names in the prompt templates are matched against the attribute names in the test dataset. Where variable names and test data attribute names are mismatched, mapping is required. For example, the prompt template in your orchestration configuration contains an attribute called “query” containing your prompt message, whereas the corresponding attribute in your ground truth dataset is called “questions”. In this case, you have to map the orchestration configuration variable “query” to the dataset variable “questions”.

Variable mappings are provided as a JSON dictionary in the following format:

Sample Code

```
{
  "target_prefix/field_name": "source_prefix/field_name"
}
```

Case 1

Mapping a variable in the orchestration workflows to an attribute in the test dataset

In the following example, “prompt” and “data” are target and source prefixes for the orchestration workflow prompt template and the test dataset respectively and “query” and “question” are the name mismatched prompt template variable and dataset attribute.

Sample Code

```
{
  "prompt/query": "data/question"
}
```

Case 2

Mapping a variable of a metric evaluator to an attribute in the test dataset.

For variables that are commonly used in judge prompts, the following system-defined variable names are defined, and their values are populated automatically.

Variable Name	Description
aicore_llm_prompt	The prompt sent to the LLM
aicore_llm_completion	The completion string returned by the LLM

You must define the following common values for variables.

Variable Name	Description
reference	The ground truth reference string provided by the user.

The variable names starting with "aicore" are reserved and must not be used for other variables or dataset attributes.

In the following example, the target prefix is the ID of the metric evaluator and the source prefix is “data”.

Sample Code

```
{
  ...
  "<metric_id>/reference": "data/ground_truth"
  ...
}
```

To map a dataset attribute to a variable being used in all metric evaluators, the target prefix `all_metrics` can be used as the target prefix. For example:

Sample Code

```
{
  ...
  "all_metrics/reference": "data/ground_truth"
  ...
}
```

Register Evaluation Artifacts

You can evaluate different orchestration configurations and prompt variable values against the shortlisted models to find the best orchestration configuration for your use case.

Prerequisites

- You have an orchestration deployment running. For more information, see Get Your Orchestration Deployment URL or Create a Deployment for Orchestration.
- You've registered an object store for Amazon S3, OSS, or Azure Blob Storage. For more information, see Register an Object Store for Optimizations.

Context

To run an evaluation, you must provide the following artifacts:

- A test dataset containing data needed for the evaluation, such as values for the prompt variables and ground truth references. The following file formats are supported:
  - `CSV`
  - `JSON`
  - `JSONL`

The following folder structure is recommended:

```
root-directory
└── testdata
    ├── dataset1.jsonl
    └── dataset2.csv
```

Procedure

Send a request to the endpoint `{{apiurl}}/v2/lm/artifacts`, and include the prompt evaluation label and the path to your artifact within your object store in your request.

Sample Code

```
curl --location --request POST "{{apiurl}}/v2/lm/artifacts" \
--header "Authorization: Bearer $TOKEN" \
--header 'Content-Type: application/json' \
--header 'AI-Resource-Group: <resource group>' \
--data-raw '{
        "name": "yourTestData",
        "kind": "other",
```

```
        "url": "ai://<name of object store secret>/<relative path to evaluations testData folder>"
        "description": "demo artifacts for evaluation flow.",
        "scenarioId": "genai-evaluations"
    }'
```

- The `url` must begin with `ai://`, followed by the name of your object store secret and the relative path to the directory that contains your evaluation files. The directory containing all files for evaluation is registered as a single artifact.
- The `scenarioId` must be `"genai-evaluations"`.

The data path specified in the URL field uses the name of the OSS you created as the path prefix. This replaces the originally assigned path prefix, as shown in the image below.

Create an Evaluation

Prerequisites

- You have prepared your dataset and registered it as an artifact. For more information, see Register Evaluation Artifacts.
- You have chosen your metric types. If you're using custom metrics, you've prepared your custom metric. For more information, see System-Defined Evaluation Metrics and Custom Metrics.
- If your prompt template variable names and test data attribute names are mismatched, you have prepared your variable mapping. For more information, see Variable Mapping.
- You have registered an object store with the name `default`. For more information, see Register an Object Store for Optimizations.
- You have created a prompt for your evaluation, and your prompt is available in the prompt registry for your evaluation. For more information, see Prompt Registry.

Create a Configuration

To create a configuration, send a POST request to endpoint `$AI_API_URL/v2/lm/configurations`.

Include the `artifactId` for your registered dataset as `inputArtifactBindings`.

Include the `executableId` `"genai-evaluations-simplified"`

Include the following in your request as `parameterBindings`:

Input Parameter	Description
`datasetFolder`	An AI Core artifact whose path points to the root folder in the object store containing the test dataset for evaluation.
`testDataset`	A JSON containing the path to a test dataset relative to the rootFolder and a file type.
`metrics`	The IDs of metrics to be evaluated. A string containing comma-separated names of system-defined metrics, or IDs of custom metrics.
`promptTemplateScope` (Optional)	Defines the scope of the prompt template. Accepted values are `tenant` (default) or `resourcegroup`. You can omit this field for tenant-level templates. You must set it to resource group when the template is created with the `AI-Resource-Group-Scope: true` header.

Input Parameter		Description
`variableMapping` (Optional)		A JSON containing column name mappings. For more information, see Variable Mapping.
`tags` (Optional)		A JSON containing name-value pairs containing user-defined metadata.
`orchestrationDeploymentURL`		The orchestration deployment to use for orchestration completion and LLM-as-a-judge completion calls.
`repetitions` (Optional)		The number of times the same input is submitted to the LLM to evaluate the consistency of the LLM outputs. Should be greater than 1 if specified. Default is 1.
`testRowCount` (Optional)		Specifies the number of rows to be selected from the testDataset for evaluation.
A reference to your prompt template and models or to your orchestration config	`promptTemplate` and `models` combined.	• The ID of the prompt template stored in the prompt registry. • The models that correspond to the prompt template. A string containing a comma-separated list of models and their versions. For example, "`gpt-4.1:latest, gpt-4o-mini:latest`".
	`orchestrationRegistryIds`	The ID of the orchestration config stored in the orchestration registry.

Example with **`promptTemplate`** and **`models`** combined

Sample Code

```
curl --request POST "$AI_API_URL/v2/lm/configurations" \
--header "Authorization: Bearer $TOKEN" \
--header "AI-Resource-Group: $RESOURCE_GROUP" \
--header "Content-Type: application/json" \
--data '{
    "name": "genai-eval-conf",
    "scenarioId": "genai-evaluations",
    "executableId": "genai-evaluations-simplified",
    "inputArtifactBindings": [
        {
            "key": "datasetFolder",
            "artifactId": "<artifactId>"
        }
    ],
    "parameterBindings": [
        {
            "key": "repetitions",
            "value": "1"
        },
        {
            "key": "orchestrationDeploymentURL",
            "value": "https://<ai-api-base-url.com>/v2/inference/deployments/<deploymentId>"
        },
        {
            "key": "tags",
            "value": "{}"
        },
        {
            "key": "variableMapping",
            "value": "{"json_schema_match/json_schema":"data/json_schema_value"}"
        },
        {
            "key": "metrics",
            "value": "BERT Score,BLEU,ROUGE,"
        },
        {
            "key": "testDataset",
            "value": "{"path":"data/template_variables.csv", "type": "csv"}"
        },
        {
            "key": "promptTemplate",
            "value": "<promptTemplateId>"
        },
        {
            "key": "promptTemplateScope",
            "value": "resourcegroup"
        },
        {
            "key": "models",
            "value": "<model>:latest, <model>:latest"
        },
    ]
}'
```

Example with **`orchestrationRegistryIds`**

Sample Code

```
curl --request POST "$AI_API_URL/v2/lm/configurations" \
--header "Authorization: Bearer $TOKEN" \
--header "AI-Resource-Group: $RESOURCE_GROUP" \
```

```
--header "Content-Type: application/json" \
--data '{
    "name": "genai-eval-conf",
    "scenarioId": "genai-evaluations",
    "executableId": "genai-evaluations-simplified",
    "inputArtifactBindings": [
        {
            "key": "datasetFolder",
            "artifactId": "<artifactId>"
        }
    ],
    "parameterBindings": [
        {
            "key": "repetitions",
            "value": "1"
        },
        {
            "key": "orchestrationDeploymentURL",
            "value": "https://<ai-api-base-url.com>/v2/inference/deployments/<deploymentId>"
        },
        {
            "key": "tags",
            "value": "{}"
        },
        {
            "key": "variableMapping",
            "value": "{"json_schema_match/json_schema":"data/json_schema_value"}"
        },
        {
            "key": "metrics",
            "value": "BERT Score,BLEU,ROUGE,"
        },
        {
            "key": "testDataset",
            "value": "{"path":"data/template_variables.csv", "type": "csv"}"
        },
          { "key": "orchestrationRegistryIds",
            "value": "<orchestrationConfigId>" },
    ]
}'
```

You will receive a configuration ID. Note the ID for the next step.

Start an Evaluation

To start an evaluation, send a POST request to endpoint `$AI_API_URL/v2/lm/executions`: Include your configuration ID in your request.

For example:

Sample Code

```
curl --request POST "$AI_API_URL/v2/lm/executions"
--header "Authorization: Bearer $TOKEN"
--header "AI-Resource-Group: $RESOURCE_GROUP"
--header "Content-Type: application/json" \
--data '{
"configurationId": "<configurationID>"
}'
```

Results

Your evaluation will take some time to run. To check the status of an evaluation, send a GET request to endpoint `{{apiurl}}/v2/lm/executions/{{executionid}}`.

Next Steps

The evaluation outputs the following:

- An SQLite DB file which stores the orchestration input, orchestration output, values for all the metrics calculated for this orchestration output and statistics such as latency for this orchestration output. These metric values are called raw metric values. This SQLite DB file is stored in the object store as an AI Core output artifact.
- A set of metrics whose values are aggregated from the raw metric values. The aggregate metrics are stored in the tracking service. The user-defined tags along with the run names are stored with the metrics. Post evaluation completion the runs generated by the workload along with the aggregate metrics can be seen by calling the tracking api.

When the evaluation is complete, you can view and analyze your results. For more information, see View Evaluation Results.

After and evaluation is complete, there may be residual output artifacts such as logs or instance level metrics, that can be safely deleted by users.

View Evaluation Results

Prerequisites

You have successfully run an evaluation execution. For more information, see Create an Evaluation (Simplified).

Context

The evaluation job will calculate and report the aggregate job statistics for each completion.

Aggregate job statistics include the following:

Statistic	Description
average_latency	The average time taken in seconds to get a completion from the orchestration service
completion_count	Number of completions evaluated
total_prompt_tokens	Sum of prompt_tokens of completions
total_completion_tokens	Sum of completion_tokens of completions

The following aggregate result statistics are included for all metrics:

Statistic	Description
mean	The center point of a collection of values, also known as arithmitic mean
stddev	The amount of deviation from the mean
median	The middle value or 50th percentile of a collection of values
p90	The value at the 90th percentile of a collection of values
p95	The value at the 95th percentile of a collection of values

In addition, BERT `Score` includes these statistics for the sub metrics F1, precision, and recall.

Recall is the fraction of correctly labelled positive instances, in the context of all available positive instances. It can be computed using the equation: Recall = TP/(TP + FN), where TP is the number of true positives and FN is the number of false negatives.

Precision is the fraction of correctly labeled positive instances, in the context of all positively labelled instances. It is computed using the equation: Precision = TP/(TP + FP), where TP is the number of true positives and FP is the number of false positives.

The **F1 score** is the harmonic mean of the precision and recall. It can be computed with the equation: F1 = 2 (precision*recall)/(precision + recall)

Procedure

Retrieve your results by sending a GET request to one of the following endpoints:

- Using your `executionId`: `{{apiurl}}/v2/lm/metrics?tagFilters=evaluation.ai.sap.com/child-of={execution_id}`
- Using your `runName`: `{{apiurl}}/v2/lm/metrics?tagFilters=evaluation.ai.sap.com/run-name={run_name}`

ℹ Note

After and evaluation is complete, there may be residual output artifacts such as logs or instance level metrics, that can be safely deleted by users.

Prompt Optimization

Prompt Optimization utilizes an input prompt template from the prompt registry and a JSON dataset in the object store containing desirable responses. It optimizes the input prompt to maximize a specified metric, with metrics tracked in the ML Tracking Service. The resulting optimized prompt is saved back to the prompt registry, and additional result information is stored in the object store. This optimization process is model-specific to accommodate different model behaviors and is executed as a distinct run.

ℹ Note

Prompt optimization executions can take from minutes to multiple hours to run, and will submit a large number of prompt requests to the target models.

! Restriction

Mistral and DeepSeek models are not supported for prompt optimization.

Prompt Optimization Workflow

The workflow for prompt optimization is as follows:

1. Register your object store secrets. You need one for output artifacts with the name `default`. If you want separate object stores for input and output artifacts, register another object store secret with a name of your choice. For more information, see Register an Object Store for Optimizations.
2. Prepare a prompt template. For more information, see Template Preparation.
3. Prepare an optimization dataset. For more information, see Dataset Preparation.
4. Register your optimization dataset as a prompt optimization artifact. For more information, see Register Optimization Artifacts.
5. Create a configuration for your prompt optimization. For more information, see Create a Configuration for a Prompt Optimization

   Note

   Optimization calls often require multiple repetitions to an LLM, with each repetition incurring costs. Additionally, some calls may involve extra internal processes, leading to actual costs that differ from those predicted by cost calculators.

   For more information, see the Cost Calculator.

6. Create an execution for your configuration. For more information, see Create an Execution for your Prompt Optimization.

Template Preparation

You prepare a prompt template and make it available for optimization by storing it in the prompt registry. For more information, see Prompt Registry.

Your prompt template must meet the following conditions:

- It is available in the prompt registry
- The use prompt must contain one or more placeholders
- The system prompt must not contain variables.

For example:

Sample Code

```
---
{{?input}}
---
Extract and return a json with the following keys and values: - "urgency" as one of `high`, `medium`, `low` -
"sentiment" as one of `negative`, `neutral`, `positive` - "categories" Create a dictionary with categories as
keys and boolean values (True/False), where the value indicates whether the category is one of the best
matching support category tags from: `emergency_repair_services`, `routine_maintenance_requests`,
```

```
`quality_and_safety_concerns`, `specialized_cleaning_services`, `general_inquiries`,
`sustainability_and_environmental_practices`, `training_and_support_requests`, `cleaning_services_scheduling`,
`customer_feedback_and_complaints`, `facility_management_issues` Your complete message should be a valid json
string that can be read directly and only contain the keys mentioned in the list above. Never enclose it in
```json...```, no newlines, no unnecessary whitespaces.
```

Dataset Preparation

The dataset shows the desirable outcome of an input prompt and corresponding values for placeholders in the input prompt.

Your dataset must meet the following conditions:

- It has a minimum of 25 and maximum of 200 samples
- It contains the exact placeholder names that are used in the template in the `fields` object.

Caution

Do not include confidential of personally identifiable information in the dataset.

For example:

Sample Code

```
[
    {
        "fields": {
            "input": "Subject: Urgent Assistance Required for Specialized Cleaning Services\n\nDear ProCare
                        Facility Solutions Support Team. Could you please arrange for a specialized cleaning team to
                        visit our home at the earliest convenience? We would greatly appreciate it if this could be
                        prioritized since we want to host a large party this week.\n\nThank you for your prompt
                        attention to this matter. We look forward to your swift response and assistance.\n\nBest
                        regards,\n[Sender]"
        },
        "answer": "{\"categories\": {\"routine_maintenance_requests\": false,
                \"customer_feedback_and_complaints\": false, \"training_and_support_requests\": false,
                \"quality_and_safety_concerns\": false, \"sustainability_and_environmental_practices\": false,
                \"cleaning_services_scheduling\": false, \"specialized_cleaning_services\": true,
                \"emergency_repair_services\": false, \"facility_management_issues\": false,
                \"general_inquiries\": false}, \"sentiment\": \"neutral\", \"urgency\": \"high\"}"
    },
  {...}
]
```

Register Optimization Artifacts

Prerequisites

- You've registered an object store for Amazon S3, OSS, or Azure Blob Storage. For more information, see Register an Object Store for Optimizations.
- You've prepared a dataset in JSON format and stored it in your registered object store. For more information, see Dataset Preparation.

Context

To run an optimization, you must provide a test dataset in JSON format. Your test dataset must contain data needed for the optimization, such as values for the prompt variables and ground truth references.

Use the following folder structure:

```
root-directory
└── testdata
    ├── dataset1.json
```

If you want to use train and test datasets, use the following folder structure.

```
root-directory
└── data
    ├── train-dataset.json
    ├── test-dataset.json
```

All optimization-relevant files must be stored under file paths relative to the URL of the registered artifact.
```

Procedure

Send a POST request to the endpoint `$AI_API_URL/v2/lm/artifacts`, and include the prompt optimization label and the path to your artifact within your object store in your request.

Sample Code

```
curl --location --request POST "$AI_API_URL/v2/lm/artifacts" \
--header "Authorization: Bearer $TOKEN" \
--header 'Content-Type: application/json' \
--header 'AI-Resource-Group: <resource group>' \
--data-raw '{
        "name": "yourTestData",
        "kind": "dataset",
        "url": "ai://<name of object store secret>/<relative path to optimizations testData folder>"
        "description": "demo artifacts for optimization flow.",
        "scenarioId": "genai-optimizations"
    }'
```

- The `url` must begin with `ai://`, followed by the name of your object store secret and the relative path to the directory that contains your optimization files. The directory containing all files for optimization is registered as a single artifact.
- The `scenarioId` must be `"genai-optimizations"`.

The data path specified in the URL field uses the name of the OSS you created as the path prefix. This replaces the originally assigned path prefix, as shown in the image below.

Next Steps

Create a configuration for your optimization. For more information, see Create a Configuration for a Prompt Optimization.

Create a Configuration for a Prompt Optimization

Configuration for a prompt optimization defines the parameters, artifacts, and model settings that SAP AI Core uses to execute a prompt optimization.

Prerequisites

- You're using the `extended` service plan. For more information, see Service Plans.
- You've registered an object store secret with the name `default` for output artifacts. For more information, see Register an Object Store for Optimizations.
- If you want to separate your input and output object stores, you've registered an object store secret for input artifacts with a name of your choice. For more information, see Register an Object Store for Optimizations.
- You've prepared an optimization dataset and registered it as an optimization artifact. For more information, see Dataset Preparation and Register Optimization Artifacts.
- You've prepared a prompt template, and your template is available in the prompt registry. For more information, see Template Preparation.

Procedure

Create a configuration by sending a POST request to endpoint `$AI_API_URL/v2/lm/configurations`.

Include the following in your request:

Field	Value	
$AI_API_URL	The base URL of your SAP AI Core environment. This can also be set as an environment variable.	
$RESOURCE_GROUP	The AI resource group assigned to your account	
$TOKEN	Your access token for SAP AI Core	
`inputArtifactBindings`	The `artifactId` for your registered dataset	
`scenarioId`	`genai-optimizations`	
`executableId`	`genai-optimizations`	
`parameterBindings`	`basePrompt` (mandatory)	Reference to the prompt in the prompt registry that should be optimized in the format "`<scenario>/<promptName>:<promptVersion>`"
	`baseModel` (optional)	In case an evaluation of the selected metric should be run against a baseline model without optimization, to determine the pre-optimization metric. Default: `none`.
	`dataset` (optional)	The dataset that is used as basis for optimization. The dataset contains desired responses from the target model. The optimization will maximize the optimization metric by varying the input prompt. This parameter specifies the path to the JSON contained in the input artifact that is submitted as part of the configuration. The dataset must contain a minimum of 25 samples and a maximum of 200 samples.
	`trainDataset` (optional)	The train dataset used for optimization. The file must be in JSON format and stored in the input artifact. Use this field when you want to provide a dedicated dataset for training instead of using a single dataset. The dataset must contain a minimum of 25 samples and a maximum of 200 samples.
	`testDataset` (optional)	The test dataset used to evaluate optimization performance. The file must be in JSON format and stored in the input artifact. Use this field when you provide train and test datasets. The dataset must contain a minimum of 1 sample and a maximum of 200 samples.
	`promptTemplateScope` (Optional)	Defines the scope of the prompt template. Accepted values are `tenant` (default) or `resourcegroup`. You can omit this field for tenant-level templates. You must set it to resource group when the template is created with the `AI-Resource-Group-Scope: true` header.
	`optimizationMetric` (mandatory)	The metric to be used for optimizing. The possible choices are: • `LLMaaJ:Sem_Sim_10` This metric uses an LLM-as-a-judge to evaluate the semantic similarity of the model response relative target dataset on a scale of 1-10. • `JSON_Match` This metric determines if the LLM's JSON output matches the dataset output. The precision, recall, and f1 scores are computed based on individual fields in the JSON and averaged across all samples in the dataset. This is useful for applications that require structured outputs.
	`targetModels` (mandatory)	A comma separated list of models with version for which to optimize the prompt. A prompt can be optimized for a maximum of four models. The format is `<modelName>:<modelVersion>`. The target models must be available in the corresponding region. For more information, see SAP Note 3437766.
	`targetPromptMapping` (mandatory)	Provides a mapping to the optimized prompt for storage in the prompt registry in the format `<modelName>:<modelVersion>=<promptName>:<promptVersion>`.

Field	Value	
	`includeFewShotExamples` (optional)	Whether to include some parts of the provided dataset in the input prompt to improve the output. Default: `false`.

For example:

Sample Code

```
curl --request POST "$AI_API_URL/v2/lm/configurations" \
--header "Authorization: Bearer $TOKEN" \
--header "AI-Resource-Group: $RESOURCE_GROUP" \
--header "Content-Type: application/json" \
--data '{
    "name": "genai-eval-conf",
    "scenarioId": "genai-optimizations ",
    "executableId": "genai-optimizations ",
    "inputArtifactBindings": [
        {
            "key": "datasetFolder",
            "artifactId": "1f50fbf8-18d6-4e72-ae10-04fdbc087815"
        }
    ],
    "parameterBindings": [
        {
            "key": "basePrompt",
            "value": "<scenario>/<promptName>:<promptVersion>"
        },
        {
            "key": "dataset",
            "value": "data.json"
        {
            "key": "optimizationMetric",
            "value": "JSON_Match”
        },

        {
            "key": "promptTemplateScope",
            "value": "resourcegroup"
        },
        {

            "key": "targetModels",
            "value": "<modelName>:<modelVersion>”
        },
        {
            "key": "targetPromptMapping",
            "value": “<modelName>:<modelversion>=<promptName>:<promptVersion>”
        },
    ]
}‘
```

Note

To use `trainDataset` and `testDataset`, replace the key and value attributes in the `dataset` as shown below.

```
{
            "key": "trainDataset",
            "value": "train-dataset.json",
          }
          {
            "key": "testDataset",
            "value": "test-dataset.json",
          }
```

Create an Execution for your Prompt Optimization

Prerequisites

You've created a configuration for your optimization. For more information, see Create a Configuration for a Prompt Optimization.

Procedure

To create an execution, send a POST request to endpoint `$AI_API_URL/v2/lm/executions`. Include your configuration ID in your request.

Field	Value
$AI_API_URL	The base URL of your SAP AI Core environment. This can also be set as an environment variable.
$RESOURCE_GROUP	The AI resource group assigned to your account
$TOKEN	Your access token for SAP AI Core
`configurationId`	Your `configurationId` from the previous step

For example:

Sample Code

```
curl --request POST "$AI_API_URL/v2/lm/executions"
--header "Authorization: Bearer $TOKEN"
--header "AI-Resource-Group: $RESOURCE_GROUP"
--header "Content-Type: application/json" \
--data '{"configurationId": "<YourConfigurationId>"}'
```

Results

Your execution will take some time to run.

You can check the status of the execution, by sending a GET request to endpoint `$AI_API_URL/v2/lm/executions/<executableId>`.

After the execution has reaches `COMPLETED` status, you can check your results.

The prompt optimization execution outputs the following:

- A set or metrics that are stored in the tracking service. Post execution completion the metrics can be seen by calling the tracking api. For more information, see Get Metrics.
- A set of optimized prompt templates in the prompt registry, one per model. In case the optimization didn't give better results than the original prompt, no prompt template is stored. You can review the optimized template in the prompt registry. For more information, see Get a Prompt Template.
- A log of results contained in an output artifact that is created in the execution. This gets created in the `default` object store.

Metrics

Metrics are quantitative measures used to evaluate the performance of machine generative AI models within SAP AI Core. By tracking and analyzing metrics, you can assess model quality, compare different model versions, and ensure that your prompts generate responses that meet your standards and criteria.

Key features include:

- **System-Defined Metrics:** SAP AI Core offers a set of predefined evaluation methods and metrics tailored for common LLM tasks, such as text generation, classification, and summarization. System-defined metrics ensure consistency and comparability across different models and experiments.

  For more information, see System-Defined Evaluation Metrics.

- **Custom Metrics:** In addition to system-defined metrics, you can define and register your own custom metrics to address specific evaluation needs. This flexibility allows you to tailor performance measurement to your unique use cases and business requirements.

  For more information, see Custom Metrics.

- **Metric Management:** SAP AI Core provides tools for managing the entire lifecycle of metrics, including registration, versioning, and deprecation. You can organize metrics, track their usage, and ensure that the most relevant metrics are applied to your LLM evaluations.

  For more information, see Metric Management.

- **Integration with Evaluations:** Metrics are seamlessly integrated into the evaluation workflows. When you run an evaluation job, you can specify which metrics to compute, and the results are automatically stored and made available for analysis.

  For more information, see Evaluations.

- **Analysis:** Metric results can be compared across different model runs, making it easy to identify trends, detect regressions, and select the best-performing models.

Next Steps

Complete the prerequisites. For more information, see Register an Object Store for Optimizations.

You can create a custom metric. For more information, see Custom Metrics.

System-Defined Evaluation Metrics

System defined metrics are available when using the `genai-evaluation` global scenario.

The following types of metrics are supported:

- Computed
- LLM-as-a-judge

Computed Metrics

The following system-defined computed metrics are supported, depending on whether you use the simplified or standard evaluation workflow:

Metric Used in Simplified Workflow	Metric Used in Standard Workflow	Description	Reference Required
`BERT Score`	`bert_score`	BERTScore is a metric for evaluating the quality of text generation by comparing candidate text to reference text. It uses a pre-trained BERT model to generate contextual embeddings for each token, then measures similarity between these embeddings to assess how closely the candidate aligns with the references.	Yes
`BLEU`	`bleu`	BLEU (Bilingual Evaluation Understudy) evaluates machine-translated text quality by calculating n-gram precision between candidate and reference translations. Scores range from 0 to 1, with higher values indicating greater similarity.	Yes
`ROUGE`	`rouge`	ROUGE (Recall-Oriented Understudy for Gisting Evaluation) is a set of metrics for evaluating summarization and machine translation by measuring overlap in n-grams, word sequences, and word pairs between generated and reference texts. This implementation is case-insensitive.	Yes
`JSON Schema Match`	`json_schema_match`	Validates an LLM-generated response against a predefined JSON Schema, returning a boolean indicating whether the response matches the schema. For more information, see JSON Schema Match.	No
`Content Filter on Input`	`content_filter_on_input`	Boolean indicating whether the input content filter was invoked.	No
`Content Filter on Output`	`content_filter_on_output`	Boolean indicating whether the output content filter was invoked.	No
`Exact Match`	`exact_match`	Boolean indicating whether the output exactly matches the reference.	Yes
`Language Match`	`language_match`	Indicates whether the output matches the expected language. Results may be inaccurate when the text is very short. For more information, see Language Match.	No

LLM-as-a-Judge Metrics

The following system-defined LLM-as-a-judge metrics are supported, depending on whether you use the simplified or standard evaluation workflow:

Metric Used in Simplified Workflow	Metric Used in Standard Workflow	Description	Reference Required
`Pointwise Instruction Following`	`pointwise_instruction_following`	Evaluates the model's ability to follow the instructions provided in the user prompt. Scores range from 1 to 5, with 1 indicating no fulfillment and 5 indicating complete fulfillment.	No

Metric Used in Simplified Workflow	Metric Used in Standard Workflow	Description	Reference Required
`Pointwise Correctness`	`pointwise_correctness`	Evaluates whether an LLM response is correct, accurate, and factual using a user-provided reference, for both general and retrieval-augmented (RAG) use cases. Scores range from 1 to 5, with 1 indicating completely incorrect and 5 indicating fully correct	No
`Pointwise Answer Relevance`	`pointwise_answer_relevance`	Measures how closely the model's response relates to the user prompt, for both general and RAG use cases. Scores range from 1 to 5, with higher values indicating greater relevance.	No
`Pointwise Conciseness`	`pointwise_conciseness`	Measures how short and concise the model's response is. Scores range from 1 to 5, with higher values indicating a more concise answer.	No
`Pointwise RAG Groundedness`	`pointwise_rag_groundedness`	Evaluates whether the model's response is supported by the provided context, measuring factual consistency in RAG use cases. Scores range from 1 to 3, with higher values indicating stronger grounding in the context.	No
`Pointwise RAG Context Relevance`	`pointwise_rag_context_relevance`	Evaluates how relevant the retrieved context is to the user prompt, indicating the quality of the context retrieval stage in RAG systems. Scores range from 1 to 3, with higher values indicating that the context directly addresses the user prompt.	No
`Pointwise RAG Context Precision*`	`pointwise_rag_context_precision`	Measures how much of the provided context is relevant and useful relative to the user prompt. Scores range from 1 to 3, with higher values indicating a greater proportion of relevant context.	No
`Pointwise RAG Completeness*`	`pointwise_rag_completeness`	Evaluates how effectively the model's response incorporates all relevant information from the provided context that is necessary to fully address the user prompt. Scores range from 1 to 5, with higher values indicating minimal relevant omissions.	No

Entries marked with an asterisk (*) are experimental metrics.

Related Information

Evaluations

Language Match

Context

This metric determines whether LLM generated content is in the language required.

For more information about supported languages and their corresponding codes, see Supported Languages.

The following external libraries are used:

- lingua-py (Apache-2.0 License). For more information, see Lingua-py
- hanzidentifier (MIT) Hanzidentifier

Language match uses the following evaluation flow:

1. Use lingua-py to check the language
2. If lang_code not Chinese (simplified/traditional), match with user provided language code
3. Else run with hanzidentifier (zh_TW/zh_CN) and match with user provided language code

Procedure

Provide the code for the language you want to match.

- Provide an entry on a single row that will be applied to all rows.
- Provide an entry for each row individually.

For more information on available languages and their corresponding codes, see Supported Languages.

The metric returns `true` or `false` to indicate LLM output matches the given language. For example:

Sample Code

```
{
    "text": "LLM response, preferably long text",
    "language": "en"
}
```

Output Code

```
True
```

Supported Languages

Language	iso_code_639_1
AFRIKAANS	'AF'
ALBANIAN	'SQ'
ARABIC	'AR'
ARMENIAN	'HY'
AZERBAIJANI	'AZ'
BASQUE	'EU'
BELARUSIAN	'BE'
BENGALI	'BN'
BOKMAL	'NB'
BULGARIAN	'BG'
CATALAN	'CA'
CHINESE	'ZH'
SIMPLIFIED CHINESE	'ZH_CN'
TRADITIONAL CHINESE	'ZH_TW'
CROATIAN	'HR'
CZECH	'CS'
DANISH	'DA'
DUTCH	'NL'
ENGLISH	'EN'
ESPERANTO	'EO'
ESTONIAN	'ET'
FINNISH	'FI'
FRENCH	'FR'
GANDA	'LG'
GEORGIAN	'KA'
GERMAN	'DE'
GREEK	'EL'
GUJARATI	'GU'
HEBREW	'HE'
HINDI	'HI'

Language	iso_code_639_1
HUNGARIAN	'HU'
ICELANDIC	'IS'
INDONESIAN	'ID'
IRISH	'GA'
ITALIAN	'IT'
JAPANESE	'JA'
KAZAKH	'KK'
KOREAN	'KO'
LATIN	'LA'
LATVIAN	'LV'
LITHUANIAN	'LT'
MACEDONIAN	'MK'
MAORI	'MI'
MARATHI	'MR'
MONGOLIAN	'MN'
NYNORSK	'NN'
PERSIAN	'FA'
POLISH	'PL'
PORTUGUESE	'PT'
PUNJABI	'PA'
ROMANIAN	'RO'
RUSSIAN	'RU'
SERBIAN	'SR'
SHONA	'SN'
SLOVAK	'SK'
SLOVENE	'SL'
SOMALI	'SO'
SOTHO	'ST'
SPANISH	'ES'
SWAHILI	'SW'
SWEDISH	'SV'
TAGALOG	'TL'
TAMIL	'TA'
TELUGU	'TE'
THAI	'TH'
TSONGA	'TS'
TSWANA	'TN'
TURKISH	'TR'
UKRAINIAN	'UK'
URDU	'UR'
VIETNAMESE	'VI'
WELSH	'CY'
XHOSA	'XH'
YORUBA	'YO'

Language	iso_code_639_1
ZULU	'ZU'

JSON Schema Match

The `json_schema_match` metric validates whether the LLM's generated output conforms to a specified JSON schema. This is essential for applications that depend on structured data, requiring reliability and consistency for downstream tasks like API calls or database entries.

Dataset formatting

This metric requires a column or mapped variable "json_schema" in the dataset. This column must contain a valid JSON Schema object that defines the expected structure of the LLM's output.

A JSON Schema is a powerful tool for validating the structure of JSON data. For more information on how to construct a schema, see the official JSON Schema documentation https://json-schema.org/ .

The json_schema column entries can be provided in the following ways:

- Provide a schema for each row individually.
- Provide a schema in a single row that will be automatically applied to all rows.

The following example datasets show identical schemas provided in different ways:

Example 1: A single row is used to define all rows

Prompt	json_schema
Extract info from: 'User is Jane Doe, age 29.'	{"type": "object", "properties": {"name": {"type": "string"}, "age": {"type": "integer"}}, "required": ["name", "age"]}
Extract info from: 'User is John Smith, age 42.'	
Extract info from: 'User is Sam, age 35.'	

Example 2: Each row is defined individually

Prompt	json_schema
Extract info from: 'User is Jane Doe, age 29.'	{"type": "object", "properties": {"name": {"type": "string"}, "age": {"type": "integer"}}, "required": ["name", "age"]}
Extract info from: 'User is John Smith, age 42.'	{"type": "object", "properties": {"name": {"type": "string"}, "age": {"type": "integer"}}, "required": ["name", "age"]}
Extract info from: 'User is Sam, age 35.'	{"type": "object", "properties": {"name": {"type": "string"}, "age": {"type": "integer"}}, "required": ["name", "age"]}

Example Evaluation

This example uses the following JSON schema to assess LLM outputs for a task that extracts user contact and priority information. The schema is defined in the `json_schema` column of your dataset.

```
{
  "type": "object",
  "properties": {
    "customer_name": {
        "type": "string"
    },
    "customer_email": {
        "type": "string",
        "format": "email"
    },
    "priority": {
        "type": "string",
        "enum": ["High", "Medium", "Low"]
    }
  },
  "required": ["customer_name", "customer_email", "priority"]
}
```

Case 1: Valid Output (Pass)

This LLM output generates a well-structured JSON object that conforms perfectly to the schema:

Output Code

```
{
  "customer_name": "Alex",
  "customer_email": "alex@example.com",
  "priority": "High"
}
```

Metric Score: 1 (Pass).

The reasoning for this result is as follows:

- All required fields, for example `customer_name`, `customer_email` and `priority`, are present
- Their data types are correct. For example, `priority` is a string, and `High` is a valid enum option.

Case 2: Invalid Output (Fail)

This LLM output generates a JSON object that violates the schema in several ways:

Output Code

```
{
  "name": "Alex",
  "priority": 1
}
```

Metric Score: 0 (Fail).

The reasoning for this result is as follows:

- Missing Required Field: the required field `customer_email` is missing.
- Incorrect Field Name: the required field `customer_name` is missing; the LLM used name instead.
- Incorrect Data Type: the value for priority is the integer `1`, but the schema requires a string.
- Invalid Enum Value: the value 1 is not one of the allowed enum values: [`High`, `Medium`, `Low`].

Grounding Metrics

To use grounding related metrics, at least one of your runs provided should have `grounding_module_config` field present in your run configuration file. For more information, see Grounding.

Grounding integrates external, contextually relevant, domain-specific, or real-time data into AI processes. User provided context documents enhance the natural language processing capabilities of pretrained models, which are trained on general material.

This process involves an information retrieval component, where contextually relevant information is identified and retrieved and a generative component, where contextually relevant information is rewritten in line with the user query.

Evaluating a grounding system involves assessing both the retrieval and generation components to ensure they work together effectively.

You can evaluate your grounding pipelines using the metrics in this section.

Validated Metrics

Following are llm-as-a-judge metrics, which use another LLM to assess the response for specific criteria.

Groundedness

Groundedness measures whether the response relies solely on the provided context documents and avoids introducing external information or unsupported claims. A response is considered fully grounded if all factual claims are verifiable against the provided context.

Groundedness depends on the user query, context, and LLM output.

Note

While the definition of groundedness only depends on context and LLM output, we found that giving the user query as additional background information increased the correlation score with the ground truth from 67% to 79%.

Context Relevance

Context relevance measures how well the retrieved information from context documents matches the query.

Context relevance depends on the user query and context.

Validation Methodology

The above metrics were validated on the following datasets:

- Ragbench
- Two SAP internal datasets

Most datasets have ground truth scores, which were used to measure whether or not the metrics accurately captured performance. Ground truth scores are calculated based on a model's assessment of the ground truth data, and are verified by humans.

For datasets without ground truth scores, `gemini-2.5-pro` was used to generate synthetic scores, which were then verified by humans.

Tests for groundedness and context relevance using GPT-4o as a judge achieved a correlation of 79% and 83% with the ground truth scores, respectively.

Tests for groundedness and context relevance using GPT-4.1 as a judge achieved a correlation of 83.74% and 87.07% with the ground truth scores, respectively.

Computed Metrics vs LLM-as-a-Judge Metrics

Computed metrics such as BERT Score, BLEU, and cosine similarity performed poorly compared to llm-as-a-judge metrics and were not selected for grounding evaluation.

Instead, we recommend these corresponding metrics:

Computed Metric	LLM-as-a-Judge Metric
BERT Score with `user query` and `context`	Pointwise RAG Context Relevance
BERT Score with `context` and `llm output`	Pointwise RAG Groundedness

Custom Metrics

In addition to system-defined evaluation metrics, you can define custom metrics to evaluate a specific use-case.

LLM-as-a-judge-type metrics are supported. A custom LLM-as-a-judge metric uses a judge LLM along with a rubric to compute a metric rating. The output can be in numeric or text format.

You can define custom LLM-as-a-judge metrics using either structured prompts or free form prompts.

Structured Prompts

Structured prompts are the recommended way to define custom metrics. They use a predefined format that allows you to define the metrics.

Sample Code

```
{
    "spec": {
        "promptType": "structured",
        "configuration": {
            "modelConfiguration": {
                "name": "gpt-4",
                "version": "2024-02-15",
                #optional
                "parameters": [
                    {
                        "key": "maxlength",
                        "value": "120000"
                    }
                ]
            },
            "promptConfiguration": {
                "definition": "Measures how well the response aligns with verifiable facts",
                "evaluationTask": "Rate the factual accuracy of the response compared to the reference",
                "ratingRubric": [
                    {
                        "rating": 5,
                        "rule": "Response is completely factual with no unsupported claims"
                    },
                    {
                        "rating": 3,
                        "rule": "Response has minor inaccuracies but no major contradictions"
                    },
                    {
                        "rating": 1,
                        "rule": "Response contains significant factual errors or hallucinations"
                    }
                ],
                "criteria": "Evaluate based on facts, absence of hallucinations, and alignment with reference",
                "evaluationSteps": [
                    "Compare response against reference material",
```

```
                        "Identify any factual claims in the response",
                        "Verify each claim against known facts",
                        "Rate based on accuracy level"
                    ],
                    "examples": [
                        {
                            "prompt": "What is the capital of France?",
                            "groundingInput": "Paris is the capital and most populous city of France.",
                            "groundingOutput": "Paris is the capital and most populous city of France.",
                            "response": "The capital of France is Paris.",
                            "reference": "Paris is the capital and most populous city of France.",
                            "rating": 5,
                            "explanation": "Response is completely accurate and matches the reference"
                        }
                    ]
                }
            }
        }
    }
```

Metric Management

You can manage the lifecycle of your custom metrics, outside of your evaluation deployments.

Perquisites

You're using the simplified workflow. For more information, see Custom Metrics

You have prepared your custom metric for prompt management. For more information, see Register Evaluation Artifacts, and Create an Evaluation.

Context

You can:

- Define and create metrics in a structured way.
- Manage multiple versions of each metric definition as your use cases evolve.
- Track immutable revisions within each version, ensuring full reproducibility and auditability.

By separating metric definitions from your application logic, the service enables safe experimentation and updates without disrupting ongoing workflows. This ensures that new ideas can be tested, validated, and rolled out confidently while preserving the integrity of existing metrics.

PUBLIC
2026-01-29

SAP AI Launchpad

© 2026 SAP SE or an SAP affiliate company. All rights reserved.

Content

1 What Is SAP AI Launchpad? . . . 4

2 What's New for SAP AI Launchpad . . . 7

3 Concepts. . . . 36

3.1 SAP AI Launchpad Overview. . . . 43

3.2 Custom Runtime Capabilities Using the Meta API. . . . 43

3.3 Resource Groups. . . . 44

4 Service Plans. . . . 45

Changing Service Plans. . . . 46

4.1 Free Plan. . . . 46

Enabling Free Plan. . . . 47

4.2 Metering and Pricing. . . . 47

4.3 Update from Free Plan to Standard Plan. . . . 48

5 Initial Setup. . . . 50

5.1 Create a Subaccount. . . . 50

5.2 Enable Cloud Foundry. . . . 52

5.3 Add a Service Plan. . . . 53

5.4 Subscribe to SAP AI Launchpad. . . . 55

5.5 Allow Access to SAP AI Launchpad. . . . 56

5.6 Launch SAP AI Launchpad. . . . 63

5.7 Change Language Setting. . . . 65

6 Using SAP AI Launchpad. . . . 67

6.1 Generative AI Hub. . . . 68

Models and Scenarios in the Generative AI Hub. . . . 69

Tutorial. . . . 70

Create a Deployment for a Generative AI Model. . . . 71

Model Library. . . . 75

Grounding Management. . . . 80

Chat. . . . 84

Prompt Editor. . . . 89

Orchestration. . . . 100

Optimizations. . . . 125

Prompt Management. . . . 147

Administration. . . . 154

6.2 AI Scenario Producer. 158
Prerequisites. 158
Workspaces. 158
Administration. 167
ML Operations. 196
6.3 AI Scenario Consumer. 266
Workspaces. 266
Functions Explorer (Deprecated). 270

7 Example End-to-End Use Case. 318

8 Security. 319
8.1 Data Protection and Privacy. 319
8.2 Security and Customer Data Protection. 320
8.3 Encryption in Transit. 320
8.4 User Authentication and Authorization. 321
Advanced Role Management. 321
8.5 Auditing and Logging Information. 337

9 Accessibility Features in SAP AI Launchpad. 341

10 Monitoring and Troubleshooting. 342

1 What Is SAP AI Launchpad?

SAP AI Launchpad is a multitenant software as a service (SaaS) application on SAP Business Technology Platform (SAP BTP). Customers and partners can use SAP AI Launchpad to manage AI use cases (scenarios) across multiple instances of AI runtimes (such as SAP AI Core). SAP AI Launchpad also provides generative AI capabilities via the Generative AI Hub.

> **Note**
>
> Throughout this document, SAP AI Core is used as an example of an AI runtime. AI runtimes are not included in your SAP AI Launchpad subscription.

> **→ Tip**
>
> The English version of this guide is open for contributions and feedback using GitHub. This allows you to get in contact with responsible authors of SAP Help Portal pages and the development team to discuss documentation-related issues. To contribute to this guide, or to provide feedback, choose the corresponding option on SAP Help Portal:
>
> - *Feedback* > *Create issue*: Provide feedback about a documentation page. This option opens an issue on GitHub.
> - *Feedback* > *Edit page*: Contribute to a documentation page. This option opens a pull request on GitHub.
>
> You need a GitHub account to use these options.
>
> More information:
>
> - Contribution Guidelines
> - Introduction Video
> - Introduction Blog Post

Environment

This service is available in the SAP BTP, Cloud Foundry environment.

Features

Integrate with AI runtime instances	Add connections to one or more AI runtime instances and switch between them to carry out further actions.

Work with resource groups of SAP AI Core	Access your resource groups from your SAP AI Core runtime and perform actions only on the AI assets that they contain (for example, models, executions, and deployments).
Manage the lifecycle of Gen AI prompts	Access the Generative AI Hub and manage the lifecycle of your prompts. The Generative AI Hub includes prompt experimentation, prompt lifecycle management, and administrative tools.
Manage the lifecycle of AI use cases	Explore and manage the lifecycle of all your AI use cases. The lifecycle includes training and deploying AI models to generate endpoints for online predictions
Access statistics on your use cases	View statistics about your AI use cases (scenarios) and how they are consumed. By analyzing these statistics, you can better estimate your computing needs for your AI runtimes.
Manage administration for SAP AI Core	Manage administration tasks for your SAP AI Core runtime, such as the authentications required in your AI workflows.

Overview of SAP AI Launchpad and SAP AI Core (as the AI Runtime)

SAP AI Launchpad Apps and Roles

For more information about resource groups in SAP AI Core, see Resource Groups.

Related Information

Using SAP AI Launchpad [page 67]

2 What's New for SAP AI Launchpad

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Generative AI hub	The orchestration workflow has been optimized, and protected material detection is available in output filtering. For more information, see Output Filtering [page 117].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2026-01-19	2026-01-19
SAP AI Launchpad	Cloud Foundry	Generative AI hub	The data repository details mode of *Grounding Management* shows the history and status details of your synchronizations. You can also request a sync manually from this mode. For more information, see Grounding Management [page 80].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-12-22	2025-12-22
SAP AI Launchpad	Cloud Foundry	Generative AI hub	When defining custom metrics, you can define categorical rubrics. Existing metrics containing numerical rubrics can be updated. For more information, see Create a Custom Metric Evaluator [page 144].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-12-22	2025-12-22
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Prompt storage is not enabled in some data centers, meaning that some users are not be able to save prompts. Users who can't save prompts should save a prompt template instead. For more information, see Save a Template [page 99].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-12-08	2025-12-08

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Evaluations added to Optimizations Evaluations provides tools for benchmarking large language models and prompts via orchestration configurations. Evaluations can be used for the following use cases: • Evaluating prompt templates and models as orchestration configurations to determine the most effective combination for a use case. • Use industry standard predefined metrics for model and prompt comparison with your use case specific dataset. • Use your own custom defined metrics for your prompt and model evaluation. For more information, see Evaluations [page 129].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-12-08	2025-12-08
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Metrics added to Optimizations You can choose from a selection of system defined metrics, or define your own custom llm-as-a-judge metrics, including rating criteria. You can incorporate system defined and custom metrics into your evaluation workflows. For more information, see Metrics [page 144].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-12-08	2025-12-08

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Generative AI hub	New models are supported, including Anthropic Claude 4.5 Haiku. For more information, see SAP Note 3437766	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-12-08	2025-12-08
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Change of the default model from Mistral Large. For users with an active Mistral Large deployment, Mistral Medium is the default model. For users without an active Mistral Medium deployment, another available model will be selected as the default model. Users without an active deployment for any other model, should create a new deployment for a model of their choice. For more information, see Create a Deployment for a Generative AI Model [page 71].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-12-08	2025-12-08
SAP AI Launchpad	Cloud Foundry	ML Ops app	You can refresh table list contents using the *(refresh)* icon. For example, when viewing scenarios, configurations and so on.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-12-08	2025-12-08
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Orchestration registry You can manage the lifecycle of your orchestration workflow configurations, including saving, versioning and deleting orchestration configurations. For more information, see Save an Orchestration Config [page 121].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-11-16	2025-11-16

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Generative AI hub	You can choose an access level when saving prompt templates to the prompt registry. Choose from tenant level and resource group level. For more information, see Save a Template [page 99].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-11-16	2025-11-16
SAP AI Launchpad	Cloud Foundry	Help Documentation	You can access the Help Documentation for SAP AI Launchpad through the interface, using the ⓘ *(help)* icon.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-11-16	2025-11-16
SAP AI Launchpad	Cloud Foundry	Generative AI hub	The *Functions Explorer* app is deprecated and will be decommissioned in November 2026. Users of the *Functions Explorer* app can access their scenarios, runs, and deployments through the *ML Operations* app. For more information, see ML Operations [page 196].	Info only	Deprecated	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-11-03	2025-11-03
SAP AI Launchpad	Cloud Foundry	Generative AI hub	New models are supported, including Anthropic Claude 4.5 Sonnet. For more information, see SAP Note 3437766.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-11-03	2025-11-03
SAP AI Launchpad	Cloud Foundry	Generative AI hub	New models are supported, including Cohere Command A Reasoning, Perplexity Sonar and Sonar Pro. For more information, see SAP Note 3437766.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-10-20	2025-10-20

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Generative AI hub	SAP Document Management Service is supported as a repository type for the orchestration grounding module. For more information, see Create a Generic Secret for AI Data Management [page 106].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-09-15	2025-09-15
SAP AI Launchpad	Cloud Foundry	Generative AI hub	SAP Build WorkZone is supported as a repository type for the orchestration grounding module. For more information, see Create a Generic Secret for AI Data Management [page 106].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-09-01	2025-09-01
SAP AI Launchpad	Cloud Foundry	Generative AI hub	The orchestration workflows have been improved and updated to consume the v2 endpoint. Users uploading orchestration workflows downloaded before 2025-09-01 will need to adapt their workflow. . For more information, see SAP AI Core Orchestration V2.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-09-01	2025-09-01
SAP AI Launchpad	Cloud Foundry	Generative AI hub	New models are supported, including GPT 5, GPT 5 Mini, and GPT 5 Nano. For more information, see SAP Note 3437766.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-08-18	2025-08-18
SAP AI Launchpad	Cloud Foundry	Generative AI hub	New models are supported, including Nova Premier from AWS Bedrock. For more information, see SAP Note 3437766.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-06-21	2025-06-21

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Prompt shields are available as part of input filtering with Azure Content Safety. Prompt shields detect and mitigate prompt attacks, such as prompts designed to bypass safety mechanisms or override previous instruction. For more information, see Harm categories in Azure AI Content Safety and .	Info only	Deprecated	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-06-02	2025-06-02
SAP AI Launchpad	Cloud Foundry	Generative AI hub	GPT-4 versions 0613 and turbo-2024-04-09 and gpt-4-32k version 0613 have been deprecated. It's recommended that users of these models migrate to GPT-4.1 as a replacement. For more information, see SAP Note 3437766.	Info only	Deprecated	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-06-02	2025-06-02
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Gemini-1.5-pro and gemini-1.5-flash have been deprecated. It's recommended that users of these models migrate to gemini-2.0-flash, or gemini-2.0-flash-lite as replacements. For more information, see SAP Note3437766.	Info only	Deprecated	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-06-02	2025-06-02
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Mistralai--mixtral08x7b-v01 has been deprecated. It's recommended that users of this model migrate to mistralai--mistral-small-instruct as a replacement. For more information, see SAP Note3437766.	Info only	Deprecated	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-06-02	2025-06-02

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Generative AI hub	An additional module has been added to orchestration. The translation module translates text, and can be configured for input and output text. The input translation module helps improve answer quality when the configured model performs better when input is provided in a specific language, for example English. For more information, see Input Translation [page 112] and Output Translation [page 118].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-06-02	2025-06-02
SAP AI Launchpad	Cloud Foundry	Generative AI hub	The retirement of Chat and Prompt editor via LLM deployments is complete. Going forward, users will need an orchestration deployment to use the Chat and Prompt Editor apps. You likely already have an orchestration deployment running. You can check by viewing your deployments in the *ML Ops* app. For more information, see View a Deployment [page 232]. To create a new orchestration deployment, see Create a Deployment for Orchestration [page 101]. Alternatively, administrators can use the *Enable Orchestration* button available through the interface.	Required	Deprecated	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-05-27	2025-05-27

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Going forward, users will need an orchestration deployment to use the Prompt Editor and Chat apps. You likely already have an orchestration deployment running. You can check by viewing your deployments in the *ML Ops* app. For more information, see View a Deployment [page 232]. To create a new orchestration deployment, see Create a Deployment for Orchestration [page 101]. Alternatively, administrators can use the *Enable Orchestration* button available through the interface.	Required	Deprecated	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-05-19	2025-05-19
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Image as input is supported in orchestration. For more information, see Templating [page 111]. To see which models support image input, consult the model library. For more information, see Model Library [page 75].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-05-19	2025-05-19
SAP AI Launchpad	Cloud Foundry	Generative AI hub	New models are supported, including OpenAI GPT, o3, o4-mini, 4.1, 4.1-mini, 4.1-nano and Mistral small instruct. For more information, see SAP Note 3437766.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-05-19	2025-05-19
SAP AI Launchpad	Cloud Foundry	Generative AI hub	You can view the list of available prompt templates from the prompt repository.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-04-14	2025-04-14

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	SAP AI Launchpad	We'd like to know what you think of the SAP AI Launchpad interface. To leave your feedback, select the *(Feedback)* icon.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-04-14	2025-04-14
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Responses in markdown format from chat will be rendered in the response window, where supported by the chosen model.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-04-14	2025-04-14
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Going forward, users will need an orchestration deployment to use the Prompt Editor app. Users that currently access Chat through model deployments should create an orchestration deployment. For more information, see Create a Deployment for Orchestration [page 101]. Alternatively, administrators can use the *Enable Orchestration* button available through the chat interface.	Required	Deprecated	Changed	Technology	Not applicable	SAP Business Technology Platform	2025-04-14	2025-04-14
SAP AI Launchpad	Cloud Foundry	Generative AI hub	New models are supported, including Anthropic Claude 3.7 sonnet, GCP Vertex AI Gemini 2.0-flash and flash-lite, NVIDIA Llama 3.2 nv embedqa 1b, and OpenAI GPT 4o version 2024-11-20. For more information, see SAP Note 3437766.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-04-14	2025-04-14
SAP AI Launchpad	Cloud Foundry	Generative AI hub	We now support the Anthropic models Claude Opus 4 and Claude Sonnet 4. For more information, see SAP Note 3437766.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-06-23	2025-06-23

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Additional selected models and model versions from Open AI are supported. For more information, see 3437766.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-04-01	2025-04-01
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Responses in markdown format from orchestration and prompt editor apps will be rendered in the response window, where supported by the chosen model.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-03-03	2025-03-03
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Going forward, users will need an orchestration deployment to use the Chat app. Users that currently access Chat through model deployments should create an orchestration deployment. For more information, see Create a Deployment for Orchestration [page 101]. Alternatively, administrators can use the *Enable Orchestration* button available through the chat interface.	Required	Deprecated	Changed	Technology	Not applicable	SAP Business Technology Platform	2025-02-12	2025-02-12
SAP AI Launchpad	Cloud Foundry	Generative AI hub	An additional selected model from Aleph Alpha is supported in orchestration. For more information, see SAP Note 3437766.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-02-03	2025-02-03
SAP AI Launchpad	Cloud Foundry	Generative AI hub	The grounding management app lets you manage the lifecycle of your data pipelines. For more information, see Grounding Management [page 80].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-02-03	2025-02-03

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Chart mode is available as part of the model library. For more information, see Model Library [page 75].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2025-02-03	2025-02-03
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Model library Explore models available in generative AI hub and their metadata, using the catalog mode. Inform your decision making using bench marking data, using the leaderboard mode. View detailed information for a single model such as data input types, cost information and metrics where available on its model card. For more information, see Model Library [page 75].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2024-12-16	2024-12-16
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Additional selected models and model versions from AWS Bedrock are supported. For more information, see SAP Note 3437766.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2024-12-16	2024-12-16
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Additional selected models and model versions from Anthropic via AWS Bedrock are supported. For more information, see SAP Note 3437766.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2024-12-01	2024-12-01
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Additional selected models and model versions from Open AI are supported. For more information, see SAP Note 3437766.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2024-12-01	2024-12-01

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Additional selected model versions fromVertex AI are supported. For more information, see SAP Note 3437766.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2024-12-01	2024-12-01
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Additional module added to orchestration. The grounding module integrates external, contextually relevant, domain-specific, or real-time data into AI processes. This data supplements the natural language processing capabilities of pre-trained models, which are trained on general material. For more information, see Grounding [page 110] .	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2024-12-01	2024-12-01
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Prompt registry Prompt registry integrates prompt templates into SAP AI Core, making them discoverable across your applications and orchestration It reduces the complexity of dealing with prompt templates and leveraging integration capabilities. For more information, see .	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2024-12-01	2024-12-01

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Change of the default model from Chat GPT 3.5. For users with an active Mistral Large deployment, Mistral Large is the default model. For users without an active Mistral Large deployment, another available model will be selected as the default model. Users without an active deployment for any other model, should create a new deployment for a model of their choice. For more information, see Create a Deployment for a Generative AI Model [page 71].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2024-11-04	2024-11-04
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Additional selected models from IBM are supported. For more information, see Models and Scenarios in the Generative AI Hub [page 69].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2024-10-07	2024-10-07
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Additional selected models from Mistral AI are supported. For more information, see Models and Scenarios in the Generative AI Hub [page 69].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2024-10-07	2024-10-07
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Additional module added to orchestration. The data masking module anonymizes or pseudonymizes personally identifiable information from input. For more information, see Build Your Orchestration Workflow [page 108].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2024-10-07	2024-10-07

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Additional selected models from Meta are supported. For more information, see Models and Scenarios in the Generative AI Hub [page 69].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2024-09-16	2024-09-16
SAP AI Launchpad	Cloud Foundry	Superdomain URL for SAP AI Launchpad	SAP AI Launchpad has moved to new superdomain URL. SAP is migrating its products to a common cloud.sap superdomain to address browser isolation and privacy issues caused by current domain-specific URLs. This change aims to improve integration scenarios using iFrames or cross-origin requests, which are affected by modern browser security features For existing customers with domain specific URLs, both the original URL and the new superDomain URL will be supported for all clusters.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2024-09-16	2024-09-16
SAP AI Launchpad	Cloud Foundry	Generative AI hub	`tiiuae--falcon-40b-instruct` is deprecated and is unavailable from 2024-09-01. Users should choose another model. For more information, see Models and Scenarios in the Generative AI Hub [page 69].	Info only	Deprecated	Changed	Technology	Not applicable	SAP Business Technology Platform	2024-09-01	2024-09-01
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Image as input is supported for selected models. For more information, see Chat [page 84] and Prompt Experimentation [page 89].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2024-09-02	2024-09-02

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Additional selected models from OpenAI are supported via Azure. For more information, see Models and Scenarios in the Generative AI Hub [page 69].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2024-09-02	2024-09-02
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Disclaimers created by administrators now appear in the generative AI hub chat app. For more information, see Create a Disclaimer [page 156].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2024-08-19	2024-08-19
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Additional selected models from Anthropic are supported via AWS Bedrock. For more information, see Models and Scenarios in the Generative AI Hub [page 69].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2024-08-19	2024-08-19
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Orchestration combines content generation with a set of functions that are often required in business AI scenarios. Functions include templating, which lets you compose a prompt with placeholders that are filled during inference, and content filtering, which lets you restrict the type of content that is passed to and received from a generative AI model.	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2024-08-05	2024-08-05
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Additional selected models from OpenAI are supported via Azure. For more information, see Models and Scenarios in the Generative AI Hub [page 69].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2024-06-24	2024-06-24

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Additional selected models from GCP Vertex AI are supported. For more information, see Models and Scenarios in the Generative AI Hub [page 69].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2024-07-08	2024-07-08
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Selected models from Amazon and Anthropic are supported via AWS Bedrock. For more information, see SAP Note 3437766.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2024-06-03	2024-06-03
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Selected models from Meta are supported. For more information, see SAP Note 3437766.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2024-06-03	2024-06-03
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Chat conversations are supported, including streaming responses for selected models, including chat context, system message and prompt examples. For more information, see Chat [page 84].	Info only	General Availability	New	Technology	Not applicable		2024-05-20	2024-05-20
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Monitoring of prompt storage is supported. For more information, see Administration [page 154].	Info only	General Availability	New	Technology	Not applicable		2024-05-20	2024-05-20
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Streaming responses is supported for selected models. For more information, see Prompt Experimentation [page 89] and Chat [page 84].	Info only	General Availability	New	Technology	Not applicable		2024-05-20	2024-05-20

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Selected models from Mistal AI are supported. For more information, see Models and Scenarios in the Generative AI Hub [page 69].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2024-05-20	2024-05-20
SAP AI Launchpad	Cloud Foundry	Generative AI hub	Selected models from GCP Vertex AI are supported. For more information, see Models and Scenarios in the Generative AI Hub [page 69].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2024-04-08	2024-04-08
SAP AI Launchpad	Cloud Foundry	Generative AI hub	For generative models that support it, a 32k context window is available.	Info only	General Availability	Changed	Technology	Not applicable	SAP Business Technology Platform	2024-04-08	2024-04-08
SAP AI Launchpad	Cloud Foundry	Workspaces	Connection creation in Workspaces through x.509 certificates is supported. For more information, see Create a Service Key in SAP AI Core and Add Connection to SAP AI Core [page 160].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2024-04-08	2024-04-08
SAP AI Launchpad	Cloud Foundry	Disclaimer in generative AI hub	Administrators can create a custom disclaimer message for a selected SAP AI Core runtime. The message appears in SAP AI Launchpad for all generative AI hub users of that connection. For more information, see Create a Disclaimer [page 156].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2024-02-15	2024-02-15

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Generative AI hub	You can now access the generative AI hub in SAP AI Core. The generative AI hub includes prompt experimentation, prompt management, and administrative tools. Prompt experimentation includes creating and running natural language prompts with a choice of models and parameters. Prompt management includes saving prompts with collections, metadata in the form of tags and notes, versioning, and deletion. For more information, see Generative AI Hub [page 68].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2023-12-20	2023-12-20
SAP AI Launchpad	Cloud Foundry	Metadata in *Configurations* view	The *Configurations* view now contains metadata information, including the description for parameters and artifacts, as well as the default values associated with the parameters.	Info only	General Availability	Changed	Technology	Not applicable	SAP Business Technology Platform	2023-05-30	2023-05-30
SAP AI Launchpad	Cloud Foundry	Enum values in *Create Configuration* view	The *Create Configuration* view now includes a list of accepted enum values as a dropdown and specifies whether a parameter or input artifact is mandatory or optional. Where possible, input artifacts are filtered by type.	Info only	General Availability	Changed	Technology	Not applicable	SAP Business Technology Platform	2023-05-30	2023-05-30
SAP AI Launchpad	Cloud Foundry	Object Storage	Management of Azure object storage is supported through the *ML Operations* app.	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2023-03-20	2023-03-20

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Artifacts	Artifact details and listings pages include the artifact URL.	Info only	General Availability	Changed	Technology	Not applicable	SAP Business Technology Platform	2023-03-20	2023-03-20
SAP AI Launchpad	Cloud Foundry	Contribute to Our Documentation	At SAP, we endeavor to make sure that our documentation works for you. If you feel that something is missing or that something doesn't quite hit the mark, you can now provide feedback and suggest changes directly from SAP Help Portal. You can do so in one of two ways: • Click *Edit* in the toolbar to open the document in GitHub. There, you can suggest a change and submit a pull request for us to review. • Click *Feedback* in the toolbar to create a GitHub issue and tell us how we can improve the documentation for you. For more information, including guidelines on how to contribute, see Open Documentation Initiative.	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2023-03-13	2023-03-13
SAP AI Launchpad	Cloud Foundry	Stop or Delete Multiple Executions or Deployments	Multiple executions or deployments can be stopped or deleted at the same time. See Stop Multiple Executions [page 219], Delete Multiple Executions [page 221], Stop Multiple Deployments [page 238], or Delete Multiple Deployments [page 240].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2023-03-10	2023-03-10

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Manual GitOps Sync	The sync between GitOps and applications can be triggered manually, in addition to automatic syncs. See Create an Application [page 178].	Info only	General Availability	New	Technology	Not applicable		2023-03-10	2023-03-10
SAP AI Launchpad	Cloud Foundry	Generic Secrets	Generic secrets for the SAP AI Core runtime can be created and managed through the *SAP AI Core Administration* app. See Manage Generic Secrets in SAP AI Core [page 193].	Info only	General Availability	New	Technology	Not applicable		2023-03-10	2023-03-10
SAP AI Launchpad	Cloud Foundry	Scheduling	Main tenants can create and manage schedules for the automation of workflow executions. See Schedules [page 242].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2023-03-10	2023-03-10
SAP AI Launchpad	Cloud Foundry	Deployments - Duration	If your runtime supports deployment duration, you'll be able to set durations for new deployments. Deployments run until this time and are then automatically stopped and deleted. See Create a Deployment [page 230].	Info only	General Availability	Changed	Technology	Not applicable	SAP Business Technology Platform	2022-11-28	2022-11-28
SAP AI Launchpad	Cloud Foundry	Resource Groups	Users can use the *SAP AI Core Administration* app to manage resource groups. See Manage Resource Groups [page 187].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-11-28	2022-11-28

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Deployments - Model Replicas and Resource Plan	*ML Operations* users can investigate model replica and resource plan details, provided SAP AI Core or other compatible runtime is used. See View a Deployment [page 232].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-10-18	2022-10-18
SAP AI Launchpad	Cloud Foundry	Free Service Plan	To try SAP AI Launchpad for free, you can use a free service plan. A free service plan can be easily updated to a standard plan, retaining your users and data. See Service Plans [page 45] and Free Plan [page 46].	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2022-10-18	2022-10-18
SAP AI Launchpad	Cloud Foundry	Search Behavior for Configurations and Artifacts	Case-sensitivity has changed when searching for configurations and artifacts (such as models and datasets). If your runtime is SAP AI Core, then these searches are no longer case-sensitive. For other runtimes, search entries may be case-sensitive. See Create an Execution [page 213] or Find a Configuration [page 276]	Info only	General Availability	Changed	Technology	Not applicable	SAP Business Technology Platform	2022-10-18	2022-10-18
SAP AI Launchpad	Cloud Foundry	Object Store Secrets	Users can use the *SAP AI Core Administration* app to manage object store secrets. See Manage Object Store Secrets [page 173].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-10-04	2022-10-04

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Artifact Registration	Users can use the *ML Operations* app to register artifacts (such as models or datasets) stored in hyperscale object storage. See Register a Dataset [page 250], Register a Model [page 255], Register a Result Set [page 261], and Register an Other Artifact [page 264].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-10-04	2022-10-04
SAP AI Launchpad	Cloud Foundry	Executions and Deployments - Filter Enhancements	*ML Operations* users can filter executions and deployments. Users can apply a filter using execution ID or deployment ID, or by specifying a configuration ID or status, or by selecting a scenario and further choosing scenario attributes, such as version and executable. See View an Execution [page 215] and View a Deployment [page 232].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-09-19	2022-09-19
SAP AI Launchpad	Cloud Foundry	Configurations - Filter Enhancements	Users can filter configurations by entering a configuration ID, or by selecting a scenario and further choosing scenario attributes, such as version and executable. In the *ML Operations* app, see View a Configuration [page 207].	Info only	General Availability	Changed	Technology	Not applicable	SAP Business Technology Platform	2022-09-19	2022-09-19
SAP AI Launchpad	Cloud Foundry	Preferred Language Setting	Users can choose a preferred language when using SAP AI Launchpad apps. The preferred language can be changed at any time and from within any SAP AI Launchpad app. See Change Language Setting [page 65].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-09-02	2022-09-02

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Update a Deployment - Pending Status	Deployments can be updated if their status is *pending*, *running* or *dead*. In the *Functions Explorer* app, see Update a Deployment [page 314]. In the *ML Operations* app, see Update a Deployment [page 235].	Info only	General Availability	Changed	Technology	Not applicable	SAP Business Technology Platform	2022-08-20	2022-08-20
SAP AI Launchpad	Cloud Foundry	Custom Connections - *Workspaces* app	Connections in the *Workspaces* app can be customized, so that only applicable connections are available to nominated users. The custom setting is provided by creating a role collection with the `allow_connections` role. See Create Role Collection [page 329] and Custom Access for Connections [page 331].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-07-30	2022-07-30
SAP AI Launchpad	Cloud Foundry	Configurations - AI API Alignment	Configurations in SAP AI Launchpad are aligned with the AI API specification. Configurations can contain up to 1000 parameters. Input parameter values can't exceed 5000 characters. In the *ML Operations* app, see Create a Configuration [page 209]. In the *Functions Explorer* app, see Create a Configuration [page 209] and Create a Configuration [page 288].	Info only	General Availability	Changed	Technology	Not applicable	SAP Business Technology Platform	2022-07-30	2022-07-30

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Docker Registry Secrets - Enhancements	The *SAP AI Core Administration* app now supports managing Docker registry secrets with `Base64` encoding. Administrators can copy or upload secrets to a single *Secret* field. See Add a Secret [page 183] and Edit a Secret [page 185].	Info only	General Availability	Changed	Technology	Not applicable	SAP Business Technology Platform	2022-07-30	2022-07-30
SAP AI Launchpad	Cloud Foundry	Metrics Visualization Charts	Metrics visualization is enabled in the Functions Explorer and Ops Manager apps. You can create charts to visualize metrics associated with a single model, or to compare metric data for multiple models. You can also create charts to visualize metrics for an execution (run) or for multiple executions, and compare data for timestamp, step or configuration parameters. See Create Chart to Compare Models [page 258] and Create Chart to Compare Executions [page 224]. See Create Chart to Compare Models [page 293] and Create Chart to Compare Runs [page 303].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-07-04	2022-07-04

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Functions Explorer Enhancements	Based on user feedback, the Functions Explorer app has been enhanced. App navigation has been streamlined, and terms for key objects have been simplified. For example: • Executables are now called templates • Jobs are now called runs See Functions Explorer (Deprecated) [page 270].	Info only	General Availability	Changed	Technology	Not applicable	SAP Business Technology Platform	2022-07-04	2022-07-04
SAP AI Launchpad	Cloud Foundry	Service Key Upload	Service key data can be uploaded as a `.TXT` or `JSON` file to simplify connection creation. See Add Connection to SAP AI Core [page 160] and Edit a Connection [page 162].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-06-04	2022-06-04
SAP AI Launchpad	Cloud Foundry	Resource Groups	Resource group information has been enhanced to simplify connection creation. Resource groups are displayed by resource ID, and with other identifying labels. See Set Resource Group [page 164].	Info only	General Availability	Changed		Not applicable	SAP Business Technology Platform	2022-06-04	2022-06-04

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Meta API	SAP AI Launchpad now uses the `Meta API` (from AI API), to identify the capabilities implemented in the underlying runtime/ service. SAP AI Launchpad uses this information to enable or disable features in the user interface as applicable to the service/ runtime. Metadata for capabilities can be refreshed manually by users. See Custom Runtime Capabilities Using the Meta API [page 43] and AI API Runtime Implementations.	Info only	General Availability	New		Not applicable	SAP Business Technology Platform	2022-06-04	2022-06-04
SAP AI Launchpad	Cloud Foundry	*ML Operations* App Usability Improvements	In response to user feedback, various overview screens for operations have been enhanced in the *ML Operations* app. Additional details are included, such as changed on timestamps, as well as IDs for related operations. For example, see View a Scenario [page 202].	Info only	General Availability	Changed	Technology	Not applicable	SAP Business Technology Platform	2022-05-21	2022-05-21

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	*SAP AI Core Administration* app	An SAP AI Launchpad system administrator can manage administration tasks for their SAP AI Core runtime, such as the authentications required in their AI workflows. See Manage Repositories [page 168], Manage Applications [page 177], Manage Docker Registry Secrets [page 182] New roles and role collections are available for SAP AI Core system administrators. See Roles and Authorizations [page 321]	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-04-09	2022-04-09
SAP AI Launchpad	Cloud Foundry	Compare Metrics	SAP AI Launchpad supports metrics comparison for up to five executions or models. You can compare metrics in both the *ML Operations* and *Functions Explorer* apps. See Compare Model Metrics [page 259], and Compare Model Metrics [page 295]	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-04-09	2022-04-09

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Role Collection and Role Changes	Two role collections have been deprecated and replaced: • `ailaunchpad_functions_explorer_viewer_v2` replaces `ailaunchpad_functions_explorer_viewer` • is replaced with `ailaunchpad_functions_explorer_editor_v2` replaces `ailaunchpad_functions_explorer_editor` Two new roles are now available: • `mlfunctions_viewer` • `mlfunctions_editor` See Roles and Authorizations [page 321]	Info only	General Availability	Changed	Technology	Not applicable	SAP Business Technology Platform	2022-04-09	2022-04-09
SAP AI Launchpad	Cloud Foundry	Deployments in Functions Explorer	You can manage deployments within the *Functions Explorer* app. See Deployments [page 310].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-04-09	2022-04-09
SAP AI Launchpad	Cloud Foundry	Update a Deployment with a Chosen Configuration	You can update a deployment with your choice of configuration. You can choose to apply a new configuration, or re-use an older or previously used configuration. See Update a Deployment [page 235].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-02-17	2022-02-17

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Launchpad	Cloud Foundry	Execution and Deployment Logs	Execution and deployment logs can be downloaded in the *ML Operations* app. You can access the downloaded log files in the **Downloads** folder of your local device. See View Execution Logs [page 227] and View Deployment Logs [page 241].	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-02-17	2022-02-17
SAP AI Launchpad	Cloud Foundry	Filter and Search	Filter and search have been enabled for more pages in the *Functions Explorer* app. Users can search for the required model, dataset, or configuration using the ID or description. They can also filter a job list by specifying attribute values or conditions.	Info only	General Availability	Changed	Technology	Not applicable	SAP Business Technology Platform	2022-01-20	2022-01-20
SAP AI Launchpad	Cloud Foundry	Additional Column Listings	In the *ML Operations* app, you can now view the scenario details and artifact description as additional columns. This information is provided for the models, datasets, result sets, and other artifacts. In the *Functions Explorer* app, you can now view the scenario details and description as additional columns. This information is provided for the models and datasets.	Info only	General Availability	Changed	Technology	Not applicable	SAP Business Technology Platform	2022-01-20	2022-01-20

3 Concepts

In this section, we'll explore some of the concepts surrounding both the SAP AI Core and SAP AI Launchpad services.

Term	Definition
AI API	An application programming interface that manages and organizes AI artifacts and workflows (such as training scripts, datasets, models, and model servers) across multiple AI runtimes (environments where AI models are executed, such as cloud platforms or edge devices) to facilitate the development, deployment, and monitoring of AI applications.
AI API connection	A link between SAP AI Launchpad and a runtime. This connection is established through the AI API. For more information, see About the AI API.
AI scenario consumer	A user, either paying or non-paying, who utilizes an AI-powered software application, API, or platform for tasks such as automation, prediction, or decision-making. The consumer may have varying levels of technical expertise, ranging from non-technical end-users to experienced developers.
AI scenario producer	The developer or provider responsible for creating, maintaining, and updating an AI scenario. AI scenarios can be managed in three ways: • SAP-managed: Developed and maintained by SAP • Partner-managed: Developed and maintained by SAP partners • In-house: Developed and maintained by the organization using the AI scenario
AI service	A Software as a Service (SaaS) offering provided by SAP Business Technology Platform that enables users to leverage artificial intelligence capabilities for specific use cases. It is typically accessed through an AI API and can be used to expose and deploy AI scenarios as consumable services.
AI use case	A specific application of AI technology designed to generate business value by improving efficiency, reducing costs, or enhancing customer satisfaction. Examples include invoice matching in finance, product recommendations in e-commerce, product review classification in customer service, fraud detection in banking, and predictive maintenance in manufacturing.
artifact	Any tangible by-product produced or consumed by an execution or deployment. Artifacts can include data, files, binaries, libraries, packages, or other resources. They serve as inputs or outputs for various stages of the software lifecycle and are often versioned to track changes and maintain a history of the development process.
configuration	A collection of parameters, artifact references (such as datasets or models), and environment settings that are used to instantiate and run an execution or deployment of an executable or template. A configuration binds specific values to input parameters and specifies the versions of artifacts to be used. It is required to run an execution or deployment, and multiple configurations can be associated with a single executable or template (a 1:n relationship). Configurations are used in both training and serving processes to train models or serve predictions, respectively. They provide a way to customize and control the behavior of the executable or template for different scenarios or use cases.

Term	Definition
dataset	A collection of data used for training, testing, or analysis in artificial intelligence and machine learning applications. A dataset has the following characteristics: • References a specific data source, such as a file, database, or API • Includes metadata describing the structure, format, and content of the data • May require specific tools, libraries, or credentials to access and process the data • Can be used as input to machine learning models, algorithms, or data processing pipelines • Is uniquely identified by an ID, which is used to reference the dataset in configurations and bind it to specific components or tasks • May be versioned to track changes and ensure reproducibility of results • Can be divided into subsets for training, validation, and testing purposes • May undergo preprocessing, cleaning, or augmentation steps to prepare the data for use in machine learning tasks
deployment	An instance of a model serving template (a serving executable or deployment template) that is configured to use a model artifact and apply it to data passed in a serving request (for example, for prediction). A successful deployment creates a model server and generates a deployment URL for inference. As input, a deployment takes one or more models and parameters from a configuration. The serving executable or deployment template defines the expected parameters and input dataset required for the serving process. Values for these parameters and input models are provided by a configuration. Deployments are implemented on a runtime that produces HTTPS endpoints, enabling secure access to the deployed models for inference.
deployment template	A template that defines the parameters and inputs required to serve or deploy one or more trained models. The template is instantiated with a specific configuration, which provides the necessary parameters and inputs. The resulting instance of the template is then used to serve the trained models. Deployment templates are used in the application to facilitate the deployment and serving of machine learning models. A deployment template is also known as a “serving executable”.
embedding	A dense, low-dimensional vector representation that captures the semantic and syntactic information of a discrete input item, such as a word, phrase, or other entity, learned from a large dataset. Embeddings are typically used as input features in machine learning models to represent the underlying meaning and relationships of the items in a continuous space.

Term	Definition
executable	A reusable template that defines a workflow or pipeline for tasks such as training a machine learning model or creating a deployment. It contains placeholders for input artifacts (datasets or models) and parameters (custom key-pair values) that enable the template to be reused in different scenarios. There are two types of executables: • Non-deployable executables: When instantiated, they result in executions that may produce output artifacts. • Deployable executables: When instantiated, they result in deployments that generate URLs for inference. Executables are referred to as "templates" within the application. They can have user-defined labels applied to them, which are listed with the executable's details. The code within an executable defines the training pipeline or model deployment pipeline, leveraging the placeholders for input artifacts and parameters to make the template reusable.
execution	A workflow execution, also known as a "run" in the app, is an instance of a non-deployable executable that represents a single run of a pipeline. There are two types of executions: • Training execution: A run of a training pipeline that takes input artifacts and parameters, and produces trained AI models as output artifacts. • Batch-inferencing execution: A run of a batch-inferencing pipeline that takes input artifacts and parameters, and produces result sets as output artifacts. Each execution is associated with metadata, including metrics, labels, tags, and custom information, which can be queried using the execution ID. For example, a training execution might have metrics like accuracy and loss, while a batch-inferencing execution might have metrics like processing time and number of processed records.
function	A modular, reusable piece of functionality that can be utilized across different scenarios or workflows. Functions are designed to perform specific tasks or computations and can be combined or integrated into larger scenarios to achieve complex business objectives. For example, a data preprocessing function can be used to clean and transform input data before feeding it into a machine learning model, while a model evaluation function can be used to assess the performance of a trained model. Functions promote code reusability, maintainability, and efficiency in building AI solutions.
input artifact	Placeholder in an executable or template that enables the attachment of datasets or models required for the execution of an AI workflow or pipeline. These artifacts can include training data, validation data, pre-trained models, or any other data or model assets needed for the AI process.
job executable	A simplified representation of a workflow executable. A job executable encapsulates the essential components and logic of a workflow, allowing users to easily understand and manage the execution of AI and machine learning tasks within the Functions Explorer interface of SAP AI Launchpad.
job template	A predefined configuration that specifies the parameters and resources required to execute a long-running AI process, such as model training, batch inference, or data preprocessing. It defines the Docker image, compute resources, environment variables, and other settings needed to run the job. Job templates act as blueprints that can be instantiated to create and execute specific job instances with customized parameters.

Term	Definition
knowledge graph	A structured representation of entities, their attributes, and the relationships between them, which can be inferred from various data sources, including structured, semi-structured, and unstructured data. Knowledge graphs enable machines to understand and reason about complex real-world concepts and their interconnections, facilitating tasks such as semantic search, recommendation systems, and question answering. Examples of knowledge graphs include ontologies, taxonomies, and knowledge bases.
label	A key-value pair attached to a metric to provide additional context and metadata. Labels are used to classify and categorize metrics, enabling users to filter, group, and aggregate data for better insights and analysis. A set of labels can be applied to each instance of a metric record, allowing for more granular and targeted monitoring.
metrics	A key/value pair where the value is numeric. Metrics are used to measure and monitor the performance, progress, and quality of a model during the training process. Common examples of metrics include accuracy, precision, recall, F1 score, and mean squared error. Metrics can have optional step, timestamp, and label fields, which provide additional information about the metric's context. Every metric (and associated labels), tag, and custom info must be associated with a specific training execution. This association allows for proper organization, tracking, and comparison of different training runs. Once the metric, tag, and custom info are saved, they can be queried using an execution ID. SAP AI Launchpad provides a user-friendly interface to visualize and analyze the captured metrics, enabling users to monitor the model's performance and make informed decisions based on the data.
model	An artifact that is the output of a machine learning training process, representing the learned patterns, parameters, and structure of a trained system. • A model is generated by a training process, which optimizes the model's parameters based on training data. • A model consists of one or more files stored in a hyperscaler storage system (such as SAP AI Core's connected data storage) or the data lake for SAP HANA Cloud. • Each model is uniquely identified by a model ID, which is used in a configuration to bind the model as an input artifact to the serving executable for deployment. • Models can be manually uploaded to the connected data storage or automatically generated and uploaded as the output of a training execution in SAP AI Launchpad. • The model encapsulates the learned knowledge of the AI system and is used for making predictions or decisions on new, unseen data during inference. • Different types of models exist, such as neural networks, decision trees, or clustering models, each suited for different AI tasks and data types. • In SAP AI Core, models are managed and versioned artifacts that can be deployed to serving endpoints for production inference.
model serving template	A predefined configuration that specifies how a trained machine learning model is to be deployed and served for inference in a production environment, typically including details such as the model framework, runtime environment, resources required, and API endpoints.
operations	All activities within the AI lifecycle, including data preparation, model training, model deployment, application integration, model monitoring, and continuous improvement.

Term	Definition
output artifact	Generated results, typically AI models, produced by executing or running a training process or pipeline. When AI models are generated from an execution, they are automatically uploaded and registered to the connected data lake for SAP HANA Cloud or to the hyperscaler data storage, which is a scalable cloud storage solution provided by major cloud service providers such as Amazon Web Services (AWS), Microsoft Azure, or Google Cloud Platform (GCP).
parameter	A placeholder in an executable (script, program, or model) to which a value of a specific data type (String, Integer, Float, Boolean, List, or Dictionary) is assigned during runtime. The values are provided by configurations, which are settings or files that define the parameter values. Parameters are used to provide input data, settings, or options to the executable for customizing its behavior in executions or deployments.
prompt	A natural language instruction or query given to a generative AI model to elicit a response. An AI platform like SAP AI Launchpad includes capabilities for prompt experimentation, management, and administration: • Prompt experimentation: Creating and running natural language prompts with a choice of large language models and adjustable parameters to test and optimize prompts. • Prompt management: Saving, organizing, and managing prompts using collections, metadata (tags and notes), versioning to track changes over time, and deletion when no longer needed. • Prompt administration: Tools for governing prompt usage, monitoring prompt performance, controlling access and permissions, and ensuring responsible AI practices.
resource group	A dedicated workspace for a specific AI scenario or use case. It allows users to organize, manage, and collaborate on related AI entities such as configurations, executions, deployments, and artifacts within a defined scope. Resource Groups ensure isolation and access control, enabling users to work on their AI projects independently while sharing common resources and artifacts as needed. Each Resource Group is associated with a unique subscription to an SAP AI Launchpad tenant.
result set	A data artifact or dataset that contains the results of a batch inference run (execution). A batch inference run uses a trained machine learning model to process an input dataset containing data points or instances and generates predictions or inferences for each data point. The result set is an output artifact that stores these predictions or inferences.
run	A training process that generates a model or models based on a run template. Also known as an "execution".
run template	A template that defines the components required for an AI pipeline within the ML Operations application. It specifies placeholders for parameters, input artifacts (such as datasets), and output artifacts (such as trained models) that are necessary to instantiate and run the executable successfully. The actual values for these placeholders are provided by a configuration, which is a set of settings and parameters specific to a particular use case. In SAP AI Launchpad, a run template is also referred to as a "workflow executable".
runtime	The environment or platform that provides the necessary processing resources to execute AI and machine learning workloads, such as training and inference. It enables businesses to make their applications intelligent by leveraging AI and machine learning technologies, and allows them to train AI services using their data to automate tasks and processes. SAP AI Core is an example of a runtime.

Term	Definition
SAP AI Core	A service in the SAP Business Technology Platform designed to handle the execution and operations of your AI assets (machine learning models, datasets, and other AI-related resources) in a standardized, scalable, and hyperscaler-agnostic way, meaning it can work with different cloud service providers such as AWS, Azure, and Google Cloud Platform. SAP AI Core provides seamless integration with your SAP solutions, enabling you to easily incorporate AI capabilities into applications like SAP S/4HANA, SAP SuccessFactors, or SAP Customer Experience. Any AI function can be realized using popular open-source frameworks such as TensorFlow, PyTorch, or scikit-learn. SAP AI Core supports the full lifecycle management of AI scenarios, including data preparation, model training, deployment, monitoring, and retraining.
SAP AI Launchpad	A multitenant software as a service (SaaS) application on SAP Business Technology Platform (SAP BTP). Customers and partners can use SAP AI Launchpad as a centralized platform to manage AI use cases (scenarios), which are specific applications or problems that can be solved using AI, across multiple instances of AI runtimes (such as SAP AI Core). SAP AI Launchpad also provides access to the generative AI hub, which offers a set of pre-built generative AI models and tools that users can leverage to create and deploy their own generative AI applications.
scenario	An implementation of a specific AI use case within a user's tenant. It consists of a pre-defined set of AI capabilities in the form of executables (pre-built AI models or services that can be directly deployed) and templates (outlines of AI models or services that need to be trained with customer data). A scenario can have multiple versions that correspond to different versions of its contained executables and templates. Placeholders within the executables and templates are populated with customer-specific values using a configuration. The purpose of a scenario is to group executables and templates related to an AI use case and make them available to all AI consumers in the tenant, while also enabling version tracking as the AI models and services evolve over time.
serving executable	A predefined AI pipeline that encapsulates the necessary components and logic to deploy an AI model or application. It serves as a blueprint for creating a specific deployment instance. A deployment template defines placeholders for parameters, input artifacts (such as datasets), and output artifacts (such as trained models). These placeholders are filled with actual values provided by a configuration when the template is instantiated. Instantiating a deployment template involves providing the required configuration, which specifies the concrete values for the parameters and input artifacts. This process creates a deployment instance that can be executed to train, evaluate, or serve an AI model or application. In the SAP AI Launchpad, deployment templates are referred to as “serving executables”.
tag	A name/value pair that is used to categorize and organize test executions. Tags allow for the segregation and grouping of test executions based on specific criteria, making it easier to manage, analyze, and report on the results. For example, you can assign tags to a group of selected test executions to indicate their purpose, priority, or other relevant characteristics. A set of tags can be associated with a MetricResource, which is an entity that represents a specific metric or measurement. The MetricResource, in turn, is linked to an execution, establishing a connection between the tags and the corresponding test run. This relationship enables the effective tracking, monitoring, and evaluation of test executions based on their assigned tags.

Term	Definition
tenant	A logically separated customer instance that represents an organization or a company. Each tenant has its own isolated collection of customized content and services, which are available only to that specific tenant, its users, and the service provider managing the multi-tenant environment.
training	The process of running a machine learning algorithm on a dataset to produce a trained model. The trained model is an artifact that captures the patterns and relationships learned from the data, which can then be used to make predictions or decisions on new, unseen data.
workflow executable	A template that defines the components required for an AI pipeline within the ML Operations application. It specifies placeholders for parameters, input artifacts (such as datasets), and output artifacts (such as trained models) that are necessary to instantiate and run the executable successfully. The actual values for these placeholders are provided by a configuration, which is a set of settings and parameters specific to a particular use case. In SAP AI Launchpad, a workflow executable is also referred to as a "run template".

You can see how these concepts interact from the following diagram:

3.1 SAP AI Launchpad Overview

The SAP AI Launchpad service is intended to be used together with the SAP AI Core and the `AI API`.

The key components are SAP AI Core, SAP AI Launchpad, and the `AI API`.

- **SAP AI Core** provides an engine that lets you run AI workflows and model serving workloads.
- **SAP AI Launchpad** manages a number of AI runtimes. It allows various user groups to access and manage their AI scenarios.
- **AI API** provides a standard way of managing the AI scenario lifecycle on different runtimes, regardless of whether they are provided on SAP technology (such as SAP S/4HANA) or on partner technology (such as Amazon Web Services). When the AI API is deployed on runtimes other than SAP AI Core, the runtimes have to provide a runtime adapter.

3.2 Custom Runtime Capabilities Using the Meta API

The AI API `Meta API` is used to identify what capabilities are applicable for a given AI service or runtime. The SAP AI Launchpad then shows the capabilities in the user interface.

The AI API offers rich capabilities for AI use cases on your workload engine. However, not all services require all capabilities. Capabilities are enabled or disabled for your service based on the specific requirements of your AI use case, and the needs of users of your service. The `Meta API` is used to hide unnecessary capabilities in the underlying service and consequently, only those that are applicable are present in the user interface.

The `Meta API` reduces the impact of API changes, ensuring that users are not confronted with capabilities they do not use, resulting in a streamlined and robust launchpad experience.

The SAP Runtime team disables or enables capabilities in the `Meta API`.

To learn more about the capabilities provided by `Meta API`, see AI API Runtime Implementations.

> **Example**
>
> You have service **A**. This service has the capability to let users create their own deployments but there is no capability to run executions. The capability required to support service **A** is `userDeployments`, and not `userExecutions`. SAP AI Launchpad uses the metadata associated with the capability settings to hide the corresponding user interface for executions.

Metadata for capabilities can be refreshed on-demand by users. Capability metadata is also refreshed periodically according to a predefined schedule.

3.3 Resource Groups

Tenants use resource groups to isolate related ML resources and workloads. Scenarios, executables, and Docker registry secrets are shared across all resource groups.

Resource groups represent a virtual collection of related resources within the scope of one tenant. When a tenant is onboarded, the system immediately creates a default resource group. Tenant administrators can create or delete additional resource groups using the AI API. Tenants can map resource groups based on corresponding usage scenarios.

If a tenant uses resource groups to isolate scenario consumer tenants and those resource groups are deleted, the scenario consumers are deprovisioned. The service does not recognize the scenario consumer of the tenant. The standard XSUAA multitenancy model is followed.

4 Service Plans

Learn more about the different types of service plans for SAP AI Launchpad.

The service plan you choose determines pricing, conditions of use, resources, available services, and hosts.

Your use case will determine whether you choose a free plan option for SAP AI Launchpad or a paid service plan.

If you plan to use your global account in productive mode, you must purchase a paid enterprise account. It's important that you're aware of this when you're planning and setting up your account model.

The following service plans are available:

- Free plan
- Standard

Service Plan	Details	Account Type

Free plan	• Get to know SAP AI Launchpad using a free service, with limitations.
	• Only community support is available for free service plans and these are not subject to SLAs.
	• Limitations:
	  • You can have one free plan option per enterprise account
	  • Limited to one connection only, irrespective of AI runtime
	  • Generative AI hub not included
	• For region information, see SAP Discovery Centre.
	
	See Free Plan [page 46] and the tutorial Get an Account on SAP BTP to Try Out Free Plan Service Plans.
		Enterprise

Service Plan	Details	Account Type
Standard	• Service plan intended for productive usage • A fixed price is charged each month for every subaccount (tenant) • Access to the generative AI hub and generative AI hub support. • For region information, see SAP Discovery Centre. See Metering and Pricing [page 47].	Enterprise

Changing Service Plans

If you first subscribe to the free plan option, you can migrate the same service instance to a standard service plan (for enterprise accounts). If you migrate from the free plan option to standard, your metadata and transaction data, including trained models, are also migrated (for enterprise accounts). It is not possible to migrate from a standard service plan to the free plan option.

4.1 Free Plan

The free plan is an instance of SAP AI Launchpad that lets you explore the available features and capabilities without cost.

You can use the free plan option and benefit from the following advantages:

- Explore SAP AI Launchpad features and capabilities without cost.
- Migrate from the free plan to a standard service plan whilst retaining your work.

Free Plan Scope

- For region information, see SAP Discovery Center.
- Support for the free plan option is provided by the SAP Community.
- The free plan option is subject to usage limits. See Service Plans [page 45].

> **Note**
>
> The generative AI hub is a feature of the standard plan only, and is **not** available through free plan. For more information, see Service Plans [page 45].

Enabling Free Plan

Enable the free plan option to use SAP AI Launchpad with usage limits, to familiarize yourself with the service.

Prerequisites

> ⚠ Restriction
>
> SAP AI Launchpad free plan is only available in SAP BTP free tier, not SAP BTP trial.

Procedure

1. Open your global account in the SAP BTP cockpit.
2. Go to your subaccount.
3. In the navigation area, choose *Instances and Subscriptions*.
4. Click *New Instance or Subscription*
5. Search for SAP AI Launchpad in the *Service* field.
6. Choose *Free* in the *Plan* field.

Related Information

Using Free Service Plans
Service Plans [page 45]
Getting a Global Account

4.2 Metering and Pricing

Metric

SAP AI Launchpad is metered based on the billable metric “tenant”. A fixed price is charged each month for every subaccount that is subscribed to SAP AI Launchpad (that is, for every tenant). This is irrespective of how many activities are performed within a tenant.

> **Example**
>
> This example uses fictitious values. For the current conversion rates, see the SAP BTP Service Description Guide.
>
> Let's assume that you set up three SAP AI Launchpad instances and that each instance costs EUR 1,000 per month. The costs incurred are therefore:
>
> 3 instances = 3 * EUR 1,000 = EUR 3,000 per month

Related Information

SAP Discovery Center
Description Guides for Cloud Services
SAP BTP Service Description Guide

4.3 Update from Free Plan to Standard Plan

Learn how to update from a free plan option to a standard plan for SAP AI Launchpad.

Context

If you first subscribe to the free plan option, you can migrate the same service instance to a standard service plan (for enterprise accounts).

> **→ Tip**
>
> - If you update from the free plan option to standard, your metadata and transaction data, including trained models, are also migrated (for enterprise accounts).
> - If you don't want data from the free plan option and standard service plans to be combined, you can keep them separate by subscribing to the service plans in separate subaccounts.
> - It is not possible to migrate from a standard service plan to a free plan option.

For more information about updating from a free plan to paid plan on the SAP BTP cockpit, see Update to a Paid Service Plan.

Procedure

1. Open your global account in the SAP BTP cockpit.

2. Go to your subaccount.
3. In the navigation area, choose *Instances and Subscriptions*.

   A list of the applications to which your subaccount is subscribed in the Cloud Foundry environment is displayed.
4. Search for SAP AI Launchpad.
5. Select the ellipsis at the end of the subscription row, and from the menu, select *Update*.
6. In the wizard that opens, select *default* and click *Update*.

Results

After you have updated to a standard plan, free plan usage limits no longer apply. The update automatically migrates your data from the free plan to the standard plan.

> **Note**
>
> Updating to a standard plan does not change user permissions.

Related Information

Metering and Pricing [page 47]

5 Initial Setup

You can get started with SAP AI Launchpad using the standard procedures for SAP BTP, Cloud Foundry environment.

Prerequisites

- You have an enterprise global account and subaccount on SAP Business Technology Platform. For an overview of the required steps, see Getting Started in the Cloud Foundry Environment.
- Your SAP BTP administrator has set the entitlement to a subaccount so that you can provision SAP AI Launchpad.
- You have one of the following browsers to access SAP AI Launchpad:

Supported Browser Versions

Browser	Versions
Google Chrome	Latest version
Mozilla Firefox	Extended Support Release (ESR) and latest version
Microsoft Edge (chromium-based)	Latest Current Branch for Business
Safari	Latest two versions (for macOS only)

5.1 Create a Subaccount

Create a subaccount in your global account using the SAP BTP cockpit.

Prerequisites

You are an administrator of the global account in SAP Business Technology Platform.

Context

You create a subaccount within your global account. The subaccount is added as a tile to the global account view, and you are automatically assigned to it as an administrator.

We improve the product incrementally, therefore screenshots may differ from the current interface

Procedure

1. In the SAP BTP cockpit, choose *Account Explorer* and then choose *Create* > *Subaccount*.

2. Enter the details for your subaccount as follows:
   a. Enter a meaningful name and description for your subaccount.
   b. Enter a subdomain.

      This will become part of the URL for accessing the applications that you subscribe to from this subaccount.

      > **Note**
      >
      > The subdomain can contain only letters (uppercase and lowercase), digits, and hyphens (-). Hyphens are not permitted at the start or end of the name. The subdomain must be unique across all subaccounts in the same region. You cannot differentiate subdomains by uppercase and lowercase characters alone. For example, `SUBDOMAIN` and `subdomain` are considered to be the same.

   c. Enter the desired region for your subaccount.
   d. Select the *Used for production* checkbox unless your account is used for non-production purposes such as development or testing.

      This setting does not change the configuration of your subaccount. The checkbox helps you to operate your production subaccounts and systems more efficiently. Your cloud operator may also use this checkbox to take appropriate action when handling incidents related to mission-critical accounts in production systems. You can change your selection at any time by editing the subaccount properties.
   e. **Optional:** Click *Enable beta features* if you want to use beta services and applications in your subaccount.

      You cannot change this setting after you have enabled it.

> ⚠ Caution
>
> You shouldn't use SAP BTP beta features in subaccounts that belong to productive enterprise accounts. For more information, see Important Disclaimers and Legal Information.

f. Click *Create*.

3. Save your changes.

5.2 Enable Cloud Foundry

Procedure

1. Click your subaccount and on the *Overview* page, choose *Enable Cloud Foundry*.

2. Enter the basic information for your Cloud Foundry environment instance and click *Create*.

5.3 Add a Service Plan

Configure the required entitlements to make SAP AI Launchpad accessible in your subaccount.

Prerequisites

- You are an administrator of the global account in SAP Business Technology Platform.
- The subaccount is in the SAP Business Technology Platform, Cloud Foundry environment with enabled Cloud Foundry capabilities. For more information, see Enable Cloud Foundry [page 52].
- Your SAP BTP administrator has set the entitlement to a subaccount so that you can provision SAP AI Launchpad. For more information, see Managing Entitlements and Quotas Using the Cockpit.

Context

An entitlement is the service plan that you're entitled to use.

Procedure

1. In the navigation area for your global account, choose *Entitlements* > *Entity Assignments*.
2. Select your subaccount from the *Subaccounts* dropdown menu and choose *Go*.
3. Choose *Edit*.

4. Choose *Add Service Plans*.

5. Select the SAP AI Launchpad application, set the checkbox for the standard plan, and click *Add Service Plans*.

6. Save your changes.

5.4 Subscribe to SAP AI Launchpad

You subscribe to SAP AI Launchpad from the Service Marketplace in the SAP BTP cockpit.

Procedure

1. In the SAP BTP cockpit, choose *Account Explorer*.
2. Go to your global account and choose your subaccount.

3. Choose *Service Marketplace* and click the tile for SAP AI Launchpad.

4. Click *Create* to subscribe to SAP AI Launchpad.

5. Choose *Instances and Subscriptions* to see the created instances. The SAP AI Launchpad application appears with the status *Subscribed*.

5.5 Allow Access to SAP AI Launchpad

You can assign role collections to specific users to allow access to SAP AI Launchpad.

Procedure

1. Open the SAP BTP cockpit.
2. Go to your global account and subaccount.
3. Choose *Security* > *Users* and click *Create*.

4. Choose the user by clicking the corresponding row in the table.

5. In the *Role Collections* section, choose *Assign Role Collection* (you may have to click the *More* icon (•••) to see this).

6. Choose the role collections as required.

SAP AI Launchpad provides standard or default role collections. The default role collections bundle the roles required for likely SAP AI Launchpad tasks and app functions. However, if these role collections do not suit your organizational needs, you can create your own role collections.

> **Note**
>
> Generative AI hub is available through the default `viewer` role.
>
> Access to the generative AI hub can be revoked by assigning the equivalent role `without_genai`. For example, the `viewer` and `viewer_without_genai` are equivalent roles, with and without generative AI hub capabilities respectively.

Default Role Collections

Relevance	Default Role Collection	Description	Includes Roles
SAP AI Launchpad	`ailaunchpad_connections_editor`	Provides roles to view, create, edit, and delete connections to your AI runtime (for example, SAP AI Core)	`viewer` `connections_editor`
SAP AI Launchpad	`ailaunchpad_allow_all_resourcegroups`	Provides access to all resource groups	`allow_all_resourcegroups`
ML Operations	`ailaunchpad_mloperations_viewer`	Provides roles to view all contents of scenarios and resource groups	`viewer` `mloperations_viewer`
ML Operations	`ailaunchpad_mloperations_editor`	Provides roles to view all contents of scenarios, and to view and edit contents of resource groups	`viewer` `mloperations_editor`
SAP AI Core Administration	`ailaunchpad_aicore_admin_viewer`	Provides roles to view authentications required for AI workflows involving SAP AI Core (AI runtime)	`viewer` `aicore_admin_viewer_all`
SAP AI Core Administration	`ailaunchpad_aicore_admin_editor`	Provides roles to edit authentications required for AI workflows involving SAP AI Core (AI runtime)	`viewer` `aicore_admin_editor_all`

Relevance	Default Role Collection	Description	Includes Roles

Functions Explorer	`ailaunchpad_functions_explorer_viewer_v2`	Provides roles to view scenarios and all ML resources of a scenario	`viewer`
			`connections_viewer`
			`functions_explorer`
			`mlfunctions_viewer`
			`scenario_executable_viewer`
			`scenario_metadata_viewer`
			`scenario_configuration_viewer`
			`scenario_job_viewer`
			`scenario_artifact_viewer`
			`scenario_metric_viewer`

Functions Explorer	`ailaunchpad_functions_explorer_editor_v2`	Edit scenarios and all ML resources of a scenario	`viewer`
			`connections_viewer`
			`functions_explorer`
			`mlfunctions_editor`
			`scenario_executable_viewer`
			`scenario_metadata_viewer`
			`scenario_metric_viewer`
			`scenario_configuration_editor`
			`scenario_job_editor`
			`scenario_artifact_editor`

SAP AI Launchpad	`ailaunchpad_connections_editor_without_genai`	Provides roles to view, create, edit, and delete connections to your AI runtime (for example, SAP AI Core) without generative AI hub	`viewer_without_genai` `connections_editor`

Relevance	Default Role Collection	Description	Includes Roles
ML Operations	`ailaunchpad_mloperations_viewer_without_genai`	Provides roles to view all contents of scenarios and resource groups without generative AI hub	`viewer_without_genai` `mloperations_viewer`
ML Operations	`ailaunchpad_mloperations_editor_without_genai`	Provides roles to view and edit all contents of scenarios, and to view and edit contents of resource groups without generative AI hub	`viewer_without_genai` `mloperations_editor`

Functions Explorer	`ailaunchpad_functions_explorer_viewer_v2_without_genai`	Provides roles to view scenarios and all ML resources of a scenario without generative AI hub	`viewer_without_genai`
			`connections_viewer`
			`functions_explorer`
			`mlfunctions_viewer`
			`scenario_executable_viewer`
			`scenario_metadata_viewer`
			`scenario_configuration_viewer`
			`scenario_job_viewer`
			`scenario_artifact_viewer`
			`scenario_metric_viewer`

Relevance	Default Role Collection	Description	Includes Roles

Functions Explorer	`ailaunchpad_functions_explorer_editor_v2_without_genai`	Edit scenarios and all ML resources of a scenario without generative AI hub	`viewer_without_genai`
			`connections_viewer`
			`functions_explorer`
			`mlfunctions_editor`
			`scenario_executable_viewer`
			`scenario_metadata_viewer`
			`scenario_metric_viewer`
			`scenario_configuration_editor`
			`scenario_job_editor`
			`scenario_artifact_editor`

SAP AI Core Administration	`ailaunchpad_aicore_admin_viewer_without_genai`	Administrator (SAP AI Core) viewer, without access to generative AI hub	`viewer_without_genai` `aicore_admin_viewer_all`
SAP AI Core Administration	`ailaunchpad_aicore_admin_editor_without_genai`	Administrator (SAP AI Core) editor, without access to generative AI hub	`viewer_without_genai` `aicore_admin_editor_all`
Generative AI Hub	`ailaunchpad_genai_experimenter`	Prompt experimentation, viewing disclaimers and orchestration experimentation in generative AI hub.	`viewer_without_genai` `genai_experimenter`

Generative AI Hub	`ailaunchpad_genai_manager`	Prompt experimentation, managing prompts, viewing disclaimers and orchestration experimentation in generative AI hub.	`viewer_without_genai`
			`genai_experimenter`
			`genai_manager`

Generative AI Hub	`ailaunchpad_genai_administrator`	Administration user data deletion and reading, writing and deletion of disclaimers in generative AI hub.	`viewer_without_genai` `genai_administrator`

7. Choose *Assign Role Collection*.

Results

You've now assigned the role collection to the user, who has all of the authorizations provided by the role collection.

For more information, see Roles and Authorizations [page 321].

Related Information

Working with Role Collections
Roles and Authorizations [page 321]

5.6 Launch SAP AI Launchpad

You access SAP AI Launchpad from your subaccount.

Prerequisites

You have been assigned a role or role collection that grants you access to SAP AI Launchpad as described at Allow Access to SAP AI Launchpad [page 56].

Procedure

1. Open the SAP BTP cockpit.
2. Go to your global account and subaccount.
3. Choose *Instances and Subscriptions*.

4. On the *Subscriptions* tab, click the row that contains your SAP AI Launchpad application.

5. In the SAP AI Launchpad details area, choose *Go to Application*.

You will be redirected to the initial screen of SAP AI Launchpad.

5.7 Change Language Setting

Context

You can choose the language in which you work with SAP AI Launchpad apps. SAP AI Launchpad apps initially default to your browser language, however, you can change this to any of the currently supported languages.

You can change your preferred language setting at any time and from within any SAP AI Launchpad app.

Procedure

1. Open SAP AI Launchpad. The *Workspaces* app appears.
2. Choose your profile icon in the header.
3. Choose ⚙ *(Settings)* to show available languages.

4. Select your preferred language from the list, and then choose *Apply*.

Results

The screen refreshes and your preferred language appears throughout SAP AI Launchpad apps.

6 Using SAP AI Launchpad

SAP AI Launchpad can be used by both AI scenario producers and AI scenario consumers. AI scenario producers, such as AI operations engineers or AI engineers, are responsible for developing and productizing AI scenarios. AI scenario consumers, such as business analysts, subscribe to a service that offers an AI scenario and consume it. The generative AI hub within SAP AI Launchpad can be used to interact with generative AI models.

Generative AI Hub

The generative AI hub can be used for prompt engineering and prompt lifecycle management. SAP AI Launchpad provides support in the following ways:

- Prompts can be created and run in the generative AI hub *Prompt Editor*.
- Prompt lifecycle can be managed via generative AI hub *Prompt Management*.
- User data can be deleted in the generative AI hub via *Administration*.

AI Lifecycle Management

Scenarios can be managed by either the AI scenario producer or the AI scenario consumer.

Scenarios Managed by AI Scenario Producers

When the AI scenario producer is responsible for managing the lifecycle of the AI scenario, SAP AI Launchpad provides support in the following ways:

- AI scenario producers can manage the operations and lifecycle tasks of the AI scenarios for all of their AI scenario consumers using the *ML Operations* app.
- AI scenario producers can access logs to provide support to AI scenario consumers using the *ML Operations* app.
- AI scenario consumers can use the *Functions Explorer* app to explore the AI scenarios that are maintained by their scenario producer.

Scenarios Managed by AI Scenario Consumers

When the AI scenario consumer is responsible for managing the lifecycle of the AI scenario, SAP AI Launchpad provides supports in the following ways:

- AI scenario consumers can use the *Functions Explorer* app to maintain their AI scenarios.
- AI scenario producers can access logs to provide support to AI scenario consumers using the *ML Operations* app.

Related Information

What Is SAP AI Launchpad? [page 4]
Concepts [page 36]

6.1 Generative AI Hub

The generative AI hub incorporates generative AI into your AI activities in SAP AI Core and SAP AI Launchpad.

Scenarios

Access to generative AI models is provided under the global AI scenarios `foundation-models` and `orchestration`. These scenarios are managed by SAP AI Core. Individual models are provided as executables in the form of serving templates, and accessed by choosing the corresponding template for the desired model.

The following scenarios are available:

Global Scenario	Executable ID	Description
foundation-models	azure-openai	The Azure OpenAI Service provides REST API access to OpenAI's LLMs.
foundation-models	aicore-opensource	Open-source models hosted and accessed through SAP AI Core.
foundation-models	gcp-vertexai	GCP Vertex AI provides access to PaLM 2 and Gemini models from Google.
foundation-models	aws-bedrock	AWS Bedrock provides access to foundation models from Anthropic, Amazon, and other providers.
foundation-models	aicore-mistralai	Models from Mistral AI hosted and accessed through SAP AI Core.
foundation-models	aicore-ibm	Models from IBM hosted and accessed through SAP AI Core.
orchestration	orchestration	Access to orchestration of generative AI models is provided under the global AI scenario `orchestration`, which is managed by SAP AI Core.

Models

For more information about available models, including conversion rates for tokens, rate limits and deprecation dates, see SAP Note 3437766.

For more information about models and their parameters, see the documentation from the model provider.

Models from Azure OpenAI are accessed through a private instance of the `chat-completions API.` The access points aren't publicly accessible, and are accessed through SAP AI Core. For more information, see Azure OpenAI Chat Completions API .

Open Source models are hosted by SAP AI Core and can be accessed via an OpenAI-compatible API schema.

For more information on the generative AI hub in SAP AI Core, see SAP AI Core documentation.

Related Information

Azure Chat Completions Documentation
Tiiuae Falcon 40b Instruct Documentation

6.1.1 Models and Scenarios in the Generative AI Hub

The generative AI hub is available in SAP AI Core only with the `extended` service plan.

To use the generative AI hub, you must have one of the following roles or a role collection that contains one of the roles:

- `genai_manager`
- `prompt_manager`
- `genai_experimenter`
- `prompt_experimenter`

Note that users with only the `genai_experimenter` or `prompt_experimenter` roles cannot save prompts.

For more information, see Roles and Authorizations [page 321].

Scenarios

Access to generative AI models is provided under the global AI scenarios `foundation-models` and `orchestration`. These scenarios are managed by SAP AI Core. Individual models are provided as executables in the form of serving templates, and accessed by choosing the corresponding template for the desired model.

The following scenarios are available:

Global Scenario	Executable ID	Description
foundation-models	azure-openai	The Azure OpenAI Service provides REST API access to OpenAI's LLMs.
foundation-models	aicore-opensource	Open-source models hosted and accessed through SAP AI Core.

Global Scenario	Executable ID	Description
foundation-models	gcp-vertexai	GCP Vertex AI provides access to PaLM 2 and Gemini models from Google.
foundation-models	aws-bedrock	AWS Bedrock provides access to foundation models from Anthropic, Amazon, and other providers.
foundation-models	aicore-mistralai	Models from Mistral AI hosted and accessed through SAP AI Core.
foundation-models	aicore-ibm	Models from IBM hosted and accessed through SAP AI Core.
orchestration	orchestration	Access to orchestration of generative AI models is provided under the global AI scenario `orchestration`, which is managed by SAP AI Core.

Models

For more information about available models, including conversion rates for tokens, rate limits and deprecation dates, see SAP Note 3437766.

For more information about models and their parameters, see the documentation from the model provider.

Models from Azure OpenAI are accessed through a private instance of the `chat-completions API.` The access points aren't publicly accessible, and are accessed through SAP AI Core. For more information, see Azure OpenAI Chat Completions API .

Open Source models are hosted by SAP AI Core and can be accessed via an OpenAI-compatible API schema.

> **Note**
>
> The following topics are out of the scope of this document:
>
> - Advanced consumption patterns such as working with a textual knowledge base (such as embeddings)
> - Complex orchestration of LLM calls
> - Training your own models

6.1.2 Tutorial

The activation and consumption steps are also available online in a step by step tutorial.

Generative AI Hub Starter Tutorial	Activate the generative AI hub and learn the basics of composing prompts.

6.1.3 Create a Deployment for a Generative AI Model

You make a model available for use by creating a deployment. You can do so one time for each model and model version, and for each resource group that you want to use with generative AI hub.

Prerequisites

- You have an SAP AI Core service instance and service key. For more information, see SAP AI Core Initial Setup Documentation.
- You're using the SAP AI Core `extended` service plan.
- You have either the `mloperations_editor` or `scenario_deployment_editor` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations.
- You have completed the client authorization for your preferred user interface. For more information, see Use a Service Key in SAP AI Core.

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app and choose a resource group.

The *Generative AI Hub* app is now clickable in your side navigation panel and resource groups are listed.

2. In the *ML Ops* app, choose *Scenarios* and check that a scenario called *foundation-models* scenario exists.

If the *foundation-models* scenario is not available, check your SAP AI Core tenant service plan.

3. In the *ML Ops* app, choose *Configurations* and click *Create*.

Alternatively. you can make deployment through model cards in the *Model Library* app. For more information, see Model Library [page 75].

4. Enter a name for your configuration, choose the *foundation-models* scenario, enter a version number, and select the executable for your chosen model provider.

5. Enter the name and version (if applicable) of the model that you want to use.

For more information about available models, including conversion rates for tokens, rate limits and deprecation dates, see SAP Note 3437766.

> **Note**
>
> - Instead of specifying a model version, using “latest” will use the latest version of the model available in SAP AI Core.
> - Where the model version is not listed, it is not applicable.

6. After you’ve created your configuration, select *Create Deployment*.

> **Note**
>
> You must use the same resource group for all of your generative AI activities. To use a different resource group, these steps must be repeated for each resource group.

Next Steps

When the deployment is running, the model can be accessed using the *Generative AI Hub* app. For more information, see Prompt Experimentation [page 89].

If you want to remove a model, delete its deployment. For more information, see Delete a Deployment [page 239].

6.1.3.1 Model Lifecycle

Model versions have deprecation dates. Where a model version is specified in a deployment, the deployment stops working on the deprecation date of that model version.

Implement one of the following model upgrade options:

- **Auto Upgrade:** Create a generative AI configuration and deployment or patch a deployment with a new configuration, specifying `modelVersion latest`. When a new `modelVersion` is supported by SAP AI Core, existing generative AI deployments will automatically use the latest version of the given model.
- **Manual Upgrade:** Create a generative AI configuration with your chosen replacement `modelVersion` and use it to patch your deployment. This model version is used in generative AI deployments irrespective of updates to the models supported by SAP AI Core.

> ⓘ Note
>
> If `modelVersion` isn't specified, it will be `latest` by default. To upgrade manually, you **must** specify a `modelVersion`.

6.1.4 Model Library

Explore available models and their specifications. Inform your model choice using benchmarking data.

Prerequisites

You have the `genai_experimenter`, `genai_manager`, `genai_administrator` or `orchestration_executor` role, or a role collection that contains one. For more information, see Roles and Authorizations [page 321].

Context

Model library contains information on models available in the generative AI hub to aid decision making.

To explore models available in generative AI hub and their metadata, use the catalog mode.

To inform your decision making using benchmarking data, use the leaderboard mode.

To view detailed information for a single model such as data input types, cost information and metrics where available, view its model card.

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app.
2. You can use model library without selecting a resource group. However, if you select a resource group you will be able to use models directly through the model library later.
   Optional: Choose the resource group that was used for your generative AI hub deployment.
3. In the side navigation, expand the *Generative AI Hub* and choose *Model Library*.

You can view model specifications and benchmarking data through the following modes:

Catalog

The catalog mode shows all available models.

To refine the model selection, apply the filters.

To search for a model by name, use the search bar.

Choose a model to see it's model card. The model card provides detailed information on the model, such as data input types, cost information and metrics where available.

To view a deprecation notice, hover over the deprecation icon.

Leaderboard

The leaderboard mode shows model scores across a range of benchmarks.

To refine the model selection, apply the filters.

To search for a model by name, use the search bar.

To reorder the model list based on a benchmark, choose the corresponding header and make your selection.

To view a deprecation notice, hover over the deprecation icon.

Choose a model from the list to see it's model card. The model card provides detailed information on the model, such as data input types, cost information and metrics where available.

For models with tiered pricing, select the pricing entry to view pricing details.

Chart

The chart mode shows model scores across a range of benchmarks in chart format.

To refine the model selection, apply the filters.

To search for a model by name, use the search bar.

To zoom, use the *Zoom in* and *Zoom Out* icons.

To specify which variable is on which axis, use the respective drop down menus.

To switch the axes, use the ⟲ *(Rotate)* icon.

To view the chart in a separate window, use the *Full Screen* icon.

Choose a data point to see a small summary. Choose *View Details* to view the model card. The model card provides detailed information on the model, such as data input types, cost information and metrics where available.

Model Cards

To see more information, navigate through the tabs.

To see the status of deployments containing the model, refresh and choose *(Cloud)*.

To deploy the model, choose *Deploy*.

> **Restriction**
>
> Only for users with `mloperations_editor` or `scenario_deployment_editor` roles, and who have selected a resource group.

To use the model, select *Use in* `<App>` where available. For example, *Use in Chat* may be available where applicable.

> **Restriction**
>
> Only users with `genai_experimenter` or `genai_manager` roles, and who have selected a resource group, and who have a running deployment for the model can select *Use in Prompt*.
>
> Only users with `genai_experimenter` or `genai_manager` roles, and who have selected a resource group, and who have a running orchestration deployment can select *Use in Chat*.

6.1.5 Grounding Management

The grounding management app lets you manage the lifecycle of your data pipelines.

Prerequisites

You have the `genai_viewer`, `genai_experimenter`, `genai_manager`, `grounding_manager` or `grounding_viewer` role, or a role collection that contains one. For more information, see Roles and Authorizations [page 321].

You have resource group with the document grounding label enabled. For more information, see Create a Resource Group for AI Data Management [page 104].

For users who want to create a data repository pipeline: you have a generic secret for AI data management with the grounding label enabled. For more information, see Create a Generic Secret for AI Data Management [page 106].

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app.
2. In the side navigation, expand the *Generative AI Hub* and choose *Grounding Management*.

Data Repositories

Your data repositories are listed in the *Grounding Management* app.

Users with the `grounding_manager` or `genai_manager` can create a new repository.

To create a new repository, choose *Create*. Enter the repository details in the dialog box.

To include metadata, enable the switch.

For data repositories created without including metadata, you can specify which sub paths to include in the *Include paths* field.

> → Remember
>
> You must choose a grounding generic secret that is associated with the document store type that you have selected.

Users with the `grounding_manager` or `genai_manager` roles can duplicate a repository. To duplicate a repository, use the *Duplicate* icon, and update the repository details in the dialog box.

Users with the `grounding_manager` or `genai_manager` can delete a repository. To delete a repository, use the *Delete* icon.

Choose a repository to view the data repository details.

Data Repository Details Mode

You can view your documents and their upload status in the *Documents* panel.

You can view the status of the synchronization of your pipeline by choosing the *(History)* icon. You can select a synchronization to see more details about its documents, where available.

You can trigger a manual sync using the *(Sync)* icon.

To see the chunk preview, choose a document. Use the *Expand* icon to expand the chunks from preview to full.

To view the metadata for a document, use the *Metadata* tab.

To find a term in a document, use the search bar in the *Document Details* panel.

To find a document, enter a document ID into the search bar in the *Resources* panel.

Users with the `grounding_manager` or `genai_manager` roles can duplicate a repository. To duplicate a repository, use the *Duplicate* icon, and update the repository details in the dialog box.

Users with the `grounding_manager` or `genai_manager` can delete a repository. To delete a repository, use the *Delete* icon.

To configure a search over a pipeline, choose *Run Search*.

Run Documents Search Mode

Create a filter configuration using the *Create* icon and enter the following:

1. Enter your natural language query in the *Query* field.
2. Configure your filter by selecting repositories from the drop down list.
3. Optionally, you can set the Maximum number of *Chunks* or *Documents* parameter.
4. Optionally, you can include meta data filters using the *Create* icon and adding the following:
   - A key as a string
   - values as a comma separated list
   - A target type by using the corresponding icon
   - A match mode by using the corresponding icon
5. To run the search, choose *Run*.

To delete a filter configuration, use the *Delete* icon.

To pass your search results to orchestration, choose *Use in Orchestration*

6.1.6 Chat

Prerequisites

- You have an orchestration deployment running. For more information, see Create a Deployment for Orchestration [page 101]
- You've selected the AI API connection and resource group that you used in the activation steps.
- For image upload only: you have the `prompt_media_executor` role, or you are assigned a role collection that contains it. For more information, see Roles and Authorizations [page 321].
- You have the `genai_manager`, `prompt_manager`, `genai_experimenter` or `prompt_experimenter` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations [page 321].

Context

> ⚠ Caution
>
> SAP does not take any responsibility for the quality of the content in the input to or output of the underlying generative AI models. This includes but is not limited to bias, hallucinations, or inaccuracies. The user is responsible for verifying the content.

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app and choose the resource group that was used for your generative AI hub deployment.

2. In the side navigation, expand the *Generative AI Hub* and choose *Chat*.
   a. **Optional:** Adjust the model settings by choosing the ⚙ *(configure)* icon, then the *Model Settings* tab.

      Settings include:

      - **Model:** If you do not choose a model, the default model will be used.
        When choosing a model, you can filter by input type.

      - **Parameters:** Different models support different parameters and values. For more information, see the documentation from the model provider. Also see Models and Scenarios in the Generative AI Hub [page 69].
      - **Streaming Response:** Where supported, chat responses are output in real time, as they are generated.

b. **Optional:** Adjust the chat context by choosing *Configure* , then the *Chat Context* tab.

Settings include:

- **Select Template:** Users with an orchestration deployment and exisiting templates can select a template using the *(select template)* icon.
- **Context History:** The number of previous interactions that form the context for the chat. A single interaction consists of a user input and model response.
- **System Message:** Instructions or context to guide the behavior of the model. Available for selected models.
- **Examples:** Example of prompts and desired responses, to set a precedent for the model. Available for selected models.
  Examples can be removed using the *Decline* icon.

> → Tip
>
> Chat contexts are passed to the model with each chat message, and are included in the token count. Smaller contexts consume fewer tokens.
>
> If you get an exceeded token count error, reduce the context history.

3. Enter your chat input and press the ^ *(send)* icon.

   For selected models, image inputs are supported, and can be added using the *(add image)* icon or copy and paste.

   Prompt messages are limited to 5.00mb across all inputs, including the context history and system message.

   If you change your model configuration from a model that supports images to one that doesn't, you will start a new chat.

Results

The response to your chat input will be generated.

You can clear the current chat, including the context history, by clicking the *Clear* button.

You can download your chat using the *Download* icon. Text data will be automatically downloaded in JSON format, and can be saved locally. Images will not be downloaded.

You can copy text data from an individual chat message or response using the *copy* icon. Images will not be copied.

Next Steps

- You can continue your chat by sending more messages.
- You can repeat your chat with changes to the messages, model, and parameters to change the outcome.

6.1.7 Prompt Editor

6.1.7.1 Prompt Experimentation

Prerequisites

- You have an orchestration deployment running. For more information, see Create a Deployment for Orchestration [page 101]
- You've selected the AI API connection and resource group that you used in the activation steps.
- You have the `genai_manager`, `prompt_manager`, `genai_experimenter` or `prompt_experimenter` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations [page 321].
- For image upload only: you have the `prompt_media_executor` role, or you are assigned a role collection that contains it. For more information, see Roles and Authorizations [page 321].
- Users with only the `genai_experimenter` or `prompt_experimenter` roles are not able to save prompts.

Context

> ⚠ Caution
>
> SAP does not take any responsibility for the quality of the content in the input to or output of the underlying generative AI models. This includes but is not limited to bias, hallucinations, or inaccuracies. The user is responsible for verifying the content.

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app and choose the resource group that was used for your generative AI hub deployment.
2. In the side navigation, expand the *Generative AI Hub* and choose *Prompt Editor*.
3. Input your prompt:
   a. **Optional:** Enter a name for your prompt.

      Not available to the `genai_experimenter` or `prompt_experimenter` roles.
   b. **Optional:** Enter a collection name. Collection names are case sensitive.

      Not available to the `genai_experimenter` or `prompt_experimenter` roles.
   c. You can open a saved template using the *(select template)* icon, or create a new template by entering your input data in the *Message* box and assign a role to your messge using the tabs. You can add more message blocks using the + *(add)*.

For selected models, image inputs are supported, and can be added using the *(add image)* icon or copy and paste.

Prompt messages are limited to 5.00mb across all inputs.

> → Tip
>
> If the result of paste (`ctrl` + `v` or `cmd` + `v`) is not as expected, use `ctrl` + `shift` + `v` or `cmd` + `shift` + `v`.

d. Use the *(Syntax)* icon to add variables to your prompt, and define them in the *Variable Definitions* section.

e. **Optional:** Choose a model.

If you do not choose a model, the default model will be used.

When choosing a model, you can filter by input type.

f. **Optional:** Adjust the parameters to refine the generated response.

> → Tip
>
> Different models support different parameters and values. For more information, see the documentation from the model provider. Also see Models and Scenarios in the Generative AI Hub [page 69].

g. **Optional:** Add meaningful tags and notes to the metadata.

Not available to the `genai_experimenter` or `prompt_experimenter` roles.

4. Choose *Run*.

   Selected models support streaming for response generation. When the streaming switch is available, you can turn streaming on and off as needed.

   You will see your response as it generates.

Results

The response to your prompt will be generated.

Next Steps

- You can run your prompt again, make changes to the prompt, model, and parameters to change the outcome.
- You can save your prompt. For more information, see Save a Prompt [page 98].
  Not available to the `genai_experimenter` or `prompt_experimenter` roles.
- You can save your template. For more information, see Save a Template [page 99].
  Not available to the `genai_experimenter` or `prompt_experimenter` roles.
- You can copy text data from an individual chat message or response using the *copy* icon. Images will not be copied.
- You can expand the *Message* field using the expand button.

Sample Prompts

Question Answering

You can ask the LLM a question and receive a response written in natural language.

> Example
>
> Prompt:
>
> ```
> What is python in the context of programing?
> ```

Summarizing

You can provide the LLM with a text and ask for a summary of it.

> Example
>
> Prompt:
>
> ```
> Your task is to generate a short summary of a product review from an ecommerce site. Summarize the review below, delimited by triple backticks, in at most 30 words. Review: Got this panda plush toy for my daughter's birthday, who loves it and takes it everywhere. It's soft and super cute, and its face has a friendly look. It's a bit small for what I paid though. I think there might be other options that are bigger for the same price. It arrived a day earlier than expected, so I got to play with it myself before I gave it to her.
> ```

> Example
>
> This example specifies topic focus points.
>
> Prompt:
>
> ```
> Your task is to generate a short summary of a product review from an ecommerce site to give feedback to the pricing department, responsible for determining the price of the product. Summarize the review below, delimited by triple backticks, in at most 30 words, and focusing on any aspects that are relevant to the price and perceived value. Review: ```Got this panda plush toy for my daughter's birthday, who loves it and takes it everywhere. It's soft and super cute, and its face has a friendly look. It's a bit small for what I paid though. I think there might be other options that are bigger for the same price. It arrived a day earlier than expected, so I got to play with it myself before I gave it to her.
> ```

> Example
>
> This example uses “extract” instead of “summarize”.
>
> Prompt:
>
> ```
> Your task is to extract relevant information from a product review from an ecommerce site to give feedback to the Shipping department. From the review below, delimited by triple quotes extract the information relevant to shipping and delivery. Limit to 30 words. Review: ```Got this panda plush toy for my daughter's birthday,who loves it and takes it everywhere. It's soft and super cute, and its face has a friendly look. It's a bit small for what I paid though. I think there might be other options that are bigger for the same price. It arrived a day earlier than expected, so I got to play with it myself before I gave it to her.```
> ```

Inferencing

Inferencing uses the information in a given text to draw a conclusion.

Example

This example performs a sentiment analysis on a product review.

Prompt:

```
What is the sentiment of the following product review, which is delimited with
triple backticks? Review text: ```Needed a nice lamp for my bedroom, and this
one had additional storage and not too high of a price point. Got it fast. The
string to our lamp broke during the transit and the company happily sent over
a new one. Came within a few days as well. It was easy to put together. I had
a missing part, so I contacted their support, and they very quickly got me the
missing piece! Lumina seems to me to be a great company that cares about their
customers and products!!```
```

Example

This example generates the sentiment as a one word response.

Prompt:

```
What is the sentiment of the following product review, which is delimited with
triple backticks? Give your answer as a single word, either 'positive' or
'negative' Review text: ```Needed a nice lamp for my bedroom, and this one had
additional storage and not too high of a price point. Got it fast. The string to
our lamp broke during the transit and the company happily sent over a new one.
Came within a few days as well. It was easy to put together. I had a missing
part, so I contacted their support and they very quickly got me the missing
piece! Lumina seems to me to be a great company that cares about their customers
and products!!```
```

Example

This example analyzes the emotions expressed in the review.

Prompt:

```
Identify a list of emotions that the writer of the following review is
expressing. Include no more than five items in the list. Format your answer as
a list of lower-case words separated by commas. Review text: ```Needed a nice
lamp for my bedroom, and this one had additional storage and not too high of a
price point. Got it fast. The string to our lamp broke during the transit and the
company happily sent over a new one. Came within a few days as well. It was easy
to put together. I had a missing part, so I contacted their support and they very
quickly got me the missing piece! Lumina seems to me to be a great company that
cares about their customers and products!!
```

Example

This example detects whether anger is present.

Prompt:

```
Is the writer of the following review expressing anger? The review is delimited
with triple backticks. Give your answer as either yes or no. Review text:
```Needed a nice lamp for my bedroom, and this one had additional storage and
not too high of a price point. Got it fast. The string to our lamp broke during
the transit and the company happily sent over a new one. Came within a few days
as well. It was easy to put together. I had a missing part, so I contacted their
support and they very quickly got me the missing piece! Lumina seems to me to be
a great company that cares about their customers and products!!```
```

Example

This example detects product and company names from the customer review.

Prompt:

```
Identify the following items from the review text: - Item purchased by reviewer
- Company that made the item The review is delimited with triple backticks.
Format your response as a JSON object with 'Item' and 'Brand' as the keys. If
the information isn't present, use 'unknown' as the value. Make your response as
short as possible. Review text: ```Needed a nice lamp for my bedroom, and this
one had additional storage and not too high of a price point. Got it fast. The
string to our lamp broke during the transit and the company happily sent over
a new one. Came within a few days as well. It was easy to put together. I had
a missing part, so I contacted their support and they very quickly got me the
missing piece! Lumina seems to me to be a great company that cares about their
customers and products!!``
```

Example

This example performs multiple tasks in a single query.

Prompt:

```
Identify the following items from the review text: - Sentiment (positive or
negative) - Is the reviewer expressing anger? (true or false) - Item purchased
by reviewer - Company that made the item The review is delimited with triple
backticks. Format your response as a JSON object with 'Sentiment', 'Anger',
'Item' and 'Brand' as the keys. If the information isn't present, use 'unknown'
as the value. Make your response as short as possible. Review text: ```Needed a
nice lamp for my bedroom, and this one had additional storage and not too high of
a price point. Got it fast. The string to our lamp broke during the transit and
the company happily sent over a new one. Came within a few days as well. It was
easy to put together. I had a missing part, so I contacted their support and they
very quickly got me the missing piece! Lumina seems to me to be a great company
that cares about their customers and products!!```
```

Example

This example identifies the five topics discussed in a story.
```

Prompt:

```
Determine five topics that are being discussed in the following text, which is
delimited by triple backticks. Make each item one or two words long. Format
your response as a list of items separated by commas. Text sample: '''In a
recent survey conducted by the government, public sector employees were asked to
rate their level of satisfaction with the department they work at. The results
revealed that NASA was the most popular department with a satisfaction rating of
95%. One NASA employee, John Smith, commented on the findings, stating, 'I'm not
surprised that NASA came out on top. It's a great place to work with amazing
people and incredible opportunities. I'm proud to be a part of such an innovative
organization.' The results were also welcomed by NASA's management team, with
Director Tom Johnson stating, 'We are thrilled to hear that our employees are
satisfied with their work at NASA. We have a talented and dedicated team who work
tirelessly to achieve our goals, and it's fantastic to see that their hard work
is paying off.' The survey also revealed that the Social Security Administration
had the lowest satisfaction rating, with only 45% of employees indicating they
were satisfied with their job. The government has pledged to address the concerns
raised by employees in the survey and work towards improving job satisfaction
across all departments.
```

Transformations

Transformations transform a given text into another language or register.

Example

This example translates text from English to Spanish.

Prompt:

```
Translate the following English text to Spanish: ```Hi, I would like to order a
blender```
```

Example

This example detects the language that the text is written in.

Prompt:

```
Tell me which language this is: ```Combien coûte le lampadaire?```
```

Example

This example translates the given text into multiple languages.

Prompt:

```
Translate the following text to French and Spanish and English pirate: ```I want
to order a basketball```
```

Example

These examples translate both the language and register of the text.

Prompt:

```
Translate the following text to Spanish in both the formal and informal forms:
'Would you like to order a pillow?
```

Prompt:

```
Translate the following from slang to a business letter: 'Dude, This is Joe,
check out this spec on this standing lamp.
```

Example

This example translates between output formats. The prompt describes both the input and output format.

Prompt:

```
Translate the following python dictionary from JSON to an HTML table with column
headers and title:
```

```
{ "restaurant employees" :[
{"name":"Shyam", "email":"shyamjaiswal@gmail.com"},
{"name":"Bob", "email":"bob32@gmail.com"},
{"name":"Jai", "email":"jai87@gmail.com"}
]}
```

Example

In these examples, a text is proofread. The text can be proofread and corrected, or simply proofread.

Prompt:

```
Proofread and correct the following text and rewrite the corrected version.
If you don't find and errors, just say \"No errors found\". Don't use any
punctuation around the text: The girl with the black and white puppies have a
ball.
```

Example

Prompt:

```
Proofread and correct the following text and rewrite the corrected version.
If you don't find and errors, just say \"No errors found\". Don't use any
punctuation around the text: Yolanda has her notebook.
```

Example

Prompt:

```
Proofread and correct this review: ```Got this for my daughter for her birthday
cuz she keeps taking mine from my room. Yes, adults also like pandas too. She
takes ears is a bit lower than the other, and I don't think that was designed
to be asymmetrical. It's a bit small for what I paid for it though. I think
```

```
there might be other options that are bigger for the same price. It arrived a day
earlier than expected, so I got to play with it myself before I gave it to my
daughter.```
```

Expansions

Expansions generate text based on a prompt.

Example

This example generates an automated reply to a customer email.

Prompt:

```
You are a customer service AI assistant. Your task is to send an email reply to
a valued customer. Given the customer email delimited by ```, Generate a reply
to thank the customer for their review. If the sentiment is positive or neutral,
thank them for their review. If the sentiment is negative, apologize and suggest
that they can reach out to customer service. Make sure to use specific details
from the review. Write in a concise and professional tone. Sign the email as
`AI customer agent`. Customer review: ```So, they still had the 17 piece system
on seasonal sale for around $49 in the month of November, about half off, but
for some reason (call it price gouging) around the second week of December the
prices all went up to about anywhere from between $70-$89 for the same system.
And the 11 piece system went up around $10 or so in price also from the earlier
sale price of $29. So it looks okay, but if you look at the base, the part where
the blade locks into place doesn’t look as good as in previous editions from a
few years ago, but I plan to be very gentle with it (example, I crush very hard
items like beans, ice, rice, etc. in the blender first then pulverize them in the
serving size I want in the blender then switch to the whipping blade for a finer
flour, and use the cross cutting blade first when making smoothies, then use the
flat blade if I need them finer/less pulpy). Special tip when making smoothies,
finely cut and freeze the fruits and vegetables (if using spinach-lightly stew
soften the spinach then freeze until ready for use-and if making sorbet, use
a small to medium sized food processor) that you plan to use that way you can
avoid adding so much ice if at all-when making your smoothie. After about a
year, the motor was making a funny noise. I called customer service but the
warranty expired already, so I had to buy another one. FYI: The overall quality
has gone done in these types of products, so they are kind of counting on brand
recognition and consumer loyalty to maintain sales. Got it in about two days.```
Review sentiment: negative.
```

6.1.7.2 Save a Prompt

Prerequisites

- You have run a prompt successfully.
- You have either the `genai_manager` or `prompt_manager` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations [page 321].
- Users with only the `genai_experimenter` or `prompt_experimenter` roles are not able to save prompts.

> **Restriction**
>
> Prompt storage is not enabled in some data centers, meaning that some users can't save prompts.
>
> Users who are not able to save prompts should save a prompt template instead.
>
> For more information, see Save a Template [page 99].

Context

Prompts and templates are saved in one region only and can only be retrieved or deleted by an instance of AI launchpad in that region.

There is a storage limit applied at tenant level. If you reach this limit, you will receive an error message. Delete saved prompts to make space, or contact your administrator. Your prompt will not be saved and you will have to run it again to save it. You can use the copy function to paste it elsewhere for your reference. You can check your storage consumption in the generative AI hub *Administration* section.

Prompts containing images can't be saved.

Procedure

1. **Optional:** Update your prompt as follows:
   - Give your prompt a descriptive name in place of the automatically assigned name.
   - Assign your prompt to a collection.
   - Assign meaningful tags and notes as metadata.
2. Choose *Save*.

   > **Caution**
   >
   > Don't store personal data in prompts when using the generative AI hub. Personal data is any data that can be used alone, or in combination, to identify the person that the data refers to.

6.1.7.3 Save a Template

Prerequisites

- Users with only the `genai_experimenter` or `prompt_experimenter` roles are not able to save prompts. For more information, see Roles and Authorizations [page 321].

Context

Prompts and templates are saved in one region only and can only be retrieved or deleted by an instance of AI launchpad in that region.

There is a storage limit applied at tenant level. If you reach this limit, you will receive an error message. Delete saved templates to make space, or contact your administrator. Your template will not be saved and you will have to run it again to save it. You can use the copy function to paste it elsewhere for your reference. You can check your storage consumption in the generative AI hub *Administration* section.

Templates containing images can't be saved.

Procedure

1. Choose *Save Template* and add the following to the dialogue box:

- Choose an access level
- Enter or select a scenario name
- Enter or select a template name
- Enter or select a version

2. Choose *Save*.

> ⚠ Caution
>
> Don't store personal data in prompts when using the generative AI hub. Personal data is any data that can be used alone, or in combination, to identify the person that the data refers to.

6.1.8 Orchestration

The orchestration service runs on SAP AI Core under the global AI scenario `orchestration`. It provides unified access to multiple generative AI models through consistent code, configuration, and deployment.

Orchestration offers a harmonized API that allows you to use different foundation models without changing the client code. To use different foundation models and versions, you need to create at least one orchestration deployment, or use the orchestration deployment in your default resource group.

Key features of orchestration include:

- **Templating:** This feature lets you compose prompts with placeholders that are populated during inference.
- **Content filtering:** This feature allows you to restrict the type of content passed to and received from generative AI models.
- **Data masking:** This feature enables anonymization or pseudonymization of data before passing it to a generative AI model. With pseudonymization, masked data in the model's response is automatically restored.
- **Grounding:** This feature lets you integrate external, domain-specific, or real-time data to enhance pretrained models with contextually relevant information beyond their general training material.
- **Translation:** This feature lets you add translation capabilities for both input and output in your orchestration workflow.

The following scenarios are available:

Global Scenario	Executable ID	Description
`orchestration`	`orchestration`	Access to orchestration of generative AI models is provided under the global AI scenario `orchestration`, which is managed by SAP AI Core.

6.1.8.1 Create a Deployment for Orchestration

Prerequisites

> → Tip
>
> It is likely that you already have an orchestration deployment running in your default resource group. These steps are only required for users who do not have an orchestration deployment or who would like to make additional deployments for orchestration.

- You have an SAP AI Launchpad service instance and service key. For more information, see Initial Setup.
- You're using the `extended` service plan.
- You have either the `mloperations_editor` or `scenario_deployment_editor` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations.
- You have completed the client authorization for your preferred user interface. For more information, see Use a Service Key in SAP AI Core.
- You have at least one orchestration compatible deployment for a generative AI model running. For more information, see Models and Scenarios in the Generative AI Hub [page 69] and Create a Deployment for a Generative AI Model [page 71].

Context

You create a deployment to make orchestration capabilities available for use. After the deployment is complete, you get a `deploymentUrl`. You can use this URL across your organization to access orchestration in the generative AI hub.

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app and choose a resource group.

The *Generative AI Hub* app is now clickable in your side navigation panel and resource groups are listed.

2. In the *ML Ops* app, choose *Scenarios* and check that a scenario called *orchestration* exists.

   If the *orchestration* scenario is not available, check your SAP AI Core tenant service plan.

3. In the *ML Ops* app, choose *Configurations* and click *Create*.

4. Enter a name for your configuration, choose the *orchestration* scenario, enter a version number, and select the *orchestration* executable.

5. **Optional:** Add your model to a *modelFilterList*.

   The following example uses Claude Sonnet version 1:

   Sample Code

   ```
   [{"modelName": "anthropic--claude-3-sonnet","modelVersion":"1"}]
   ```

   When creating a deployment for orchestration, you can restrict the choice of models using an allow or deny list. You can use this to implement internal standards, for example where only certain LLMs are approved for use.

   modelFilterListType has the following options:

   - `deny`: only denies the models and defined versions in the modelFilterList to be used within orchestration.
   - `allow`: only allows the models and defined versions in the modelFilterList to be used within orchestration.

6. After you've created your configuration, select *Create Deployment*.

   Note

   You must use the same resource group for all of your generative AI activities. If you want to use a different resource group, repeat these steps for each resource group.

Next Steps

When the orchestration deployment is running, orchestration can be accessed using the *Generative AI Hub* app.

Related Information

Delete a Deployment [page 239]

6.1.8.2 AI Data Management

6.1.8.2.1 Create a Resource Group for AI Data Management

Prerequisites

You have the `aicore_admin_resourcegroup_editor` role or a role collection that contains it. For more information, see Roles and Authorizations.

Your tenant has been onboarded and a default resource group automatically created.

Context

> **Note**
>
> Resource group Ids must be of length minimum: 3, maximum: 253. The first and last characters must be either a lowercase letter, an uppercase letter, or a number. Character entries from the second to penultimate can include a lower case letter, an upper case letter, a number, a period (.), or a hyphen (-). No other special characters are permitted.

Procedure

1. In the *Workspaces* app, choose the AI API connection.
2. Open the *SAP AI Core Administration* app and choose *Resource Groups*.

   The *Resource Groups* screen appears with a tile for each existing resource group.
3. Choose *Create* to create reference details for a new resource group.
4. Complete the fields in the *Create Resource Group* dialog box.

a. Enter a resource group ID.

> **Note**
>
> Ensure that the resource group ID is unique. If the ID is not unique and is currently in use, then the new resource group and its details will overwrite the existing resource group.

b. Choose the **`subaccount_id`** label key and enter a value.
c. Choose the **`zone_id`** label key and enter a value.
d. Choose the **`instance_id`** label key and enter a value.
e. Enter the **`document-grounding`** label key and switch the value *On*.
f. If additional labels are required, enter their keys and corresponding values.

5. Choose *Create* to create the resource group.

   The *All Resource Groups* screen appears and shows the new resource group.

Results

The new resource group is created. Executions, deployments, configurations, and artifacts that are created using the resource group are uniquely associated with the resource group.

Next Steps

To edit a resource group, see Edit a Resource Group.

To delete a resource group, see Delete a Resource Group.

6.1.8.2.2 Create a Generic Secret for AI Data Management

Prerequisites

- You have the role `aicore_admin_genericsecret_editor` or a role collection that contains it. For more information, see Roles and Authorizations.
- You're using the SAP AI Core runtime.
- You've prepared your SharePoint integration. For more information, see Prepare SharePoint Integration with Joule.

Context

You use the *SAP AI Core Administration* app to add generic secrets at the level of the main tenant or resource group. For tenant-wide generic secrets, use the SAP AI Core API.

To allow the rotation of tenant-wide secrets for long-running deployments without requiring a restart, the deployment must mount the tenant-wide secret. It must also monitor the mounted secret for changes instead of relying on an in-memory copy. When a tenant-wide secret is updated, the tenant must observe the `resourceGroupSecretReplicationStatus` field in the `Get Secret` endpoint to confirm that the secret has been successfully replicated across the required resource groups. For more information, see Consume Generic Secrets in Executions or Deployments.

Procedure

1. In the *Workspaces* app, choose the AI API connection.
2. If you want to add your secret at the resource group level, choose the resource group. Alternatively, use the toggles in the header or dialog box, where the system prompts you to specify a resource group.
3. Open the *SAP AI Core Administration* app and choose *Generic Secrets*.

   The *Generic Secrets* screen appears with a tile for each existing secret.
4. Choose *Add* to enter reference details for a new secret.
5. Complete the fields in the *Add Generic Secret* dialog box as follows:
   a. Switch between tenant-level secrets and resource-group-level secrets.
   b. If your secret is at the resource-group level, confirm the resource group. To change the resource group, choose *(Change Value)*.
   c. Enter a name for the secret, making sure that the name meets the following criteria.
      - It contains only lowercase alphanumeric characters, hyphens (-), and numbers.
      - It doesn't start or end with a hyphen (-).

d. Enter the secret in `JSON` format. For example:

```
{
  "some-credential": "bXktc2VjcmV0LWNyZWRlbnRpYWw=",
  "other-credentials": "bXktc2VjcmV0LW90aGVyLWNyZWRlbnRpYWw=",
}
```

> **Note**
>
> The API expects sensitive data to be Base64-encoded. You can easily encode your data in Base64 format using the following command on Linux or MacOS: `echo -n 'my-sensitive-data' | base64`

6. In the *Labels* field, enter *Key* **`document-grounding`** and choose *Value* **`true`**, and enter *Key* **`documentRepositoryType`** and choose your repository type from the dropdown list.
7. Choose *Add* to save the secret details.

Related Information

Edit a Secret [page 195]
Remove a Secret [page 196]
Generic Secrets for Grounding

6.1.8.3 Build Your Orchestration Workflow

You build your orchestration workflow by designing and testing a prompt that can be populated with different data during inference.

Prerequisites

- You have an orchestration deployment running. For more information, see Create a Deployment for Orchestration [page 101].
- You have the role `orchestration_executor`, `genai_experimenter`, or `genai_manager`, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations [page 321].

The Orchestration Workflow

Navigate to the landing page for orchestration by choosing *Generative AI Hub* > *Orchestration* and clicking *Create*. SAP AI Launchpad displays the following workflow, which you can use to build your orchestration.

In a basic orchestration scenario, you can combine different modules from orchestration into a pipeline that can be executed with a single API call. Within the pipeline, the response from one module is used as the input for the next module.

The order in which the pipeline is executed is defined centrally in orchestration. However, you can configure the details for each module and omit optional modules by passing an orchestration configuration in JSON format with the request body.

The image below is interactive. Click each part of the workflow for more information.

- Grounding [page 110]
- Templating [page 111]
- Input Translation [page 112]
- Data Masking [page 113]
- Input Filtering [page 114]
- Model Configuration [page 115]
- Output Filtering [page 117]
- Output Translation [page 118]

Editing the Orchestration Workflow

You can change the orchestration workflow by showing or hiding modules. To do so, click the *Edit* button and toggle the switch to show or hide the modules as required. Note that you can only hide optional modules.

> **⚠ Restriction**
>
> Templating and Model Configuration modules are mandatory and therefore can't be hidden.

Uploading an Orchestration Workflow

You can upload your own orchestration workflow in JSON format. The workflow structure must match the following example:

```
{
    "module_configurations": {
        "llm_module_config": {
            "model_name": "gpt-35-turbo",
            "model_params": {},
            "model_version": "0613"
        },
        "templating_module_config": {
            "template": [
            {
                    "role": "user",
                    "content": "what is {{?context}} ?"
               }
        ],
            "defaults": {
                "context": "artificial intelligence"
            }
        },
        "filtering_module_config": {
            "input": {
                "filters": [
                    {
                        "type": "azure_content_safety",
                        "config": {
                            "Hate": "2",
                            "SelfHarm": "2",
                            "Sexual": "2",
                            "Violence": "2"
                        }
                    }
                ]
            },
            "output": {
                "filters": [
                    {
                        "type": "azure_content_safety",
                        "config": {
                            "Hate": "2",
                            "SelfHarm": "2",
                            "Sexual": "2",
                            "Violence": "2"
                        }
                    }
                ]
            }
```

```
            }
        }
    }
```

> **Restriction**
>
> Orchestration workflows containing images can be downloaded in JSON format, but cannot be uploaded.

1. Click *JSON* to show the JSON file for your orchestration workflow.
2. Click *Upload* to upload your edited JSON file.
   The size of your file cannot exceed 200 KB.
3. If necessary, you can download the JSON file again by clicking *Download*.

> **Note**
>
> If the parameters in your uploaded JSON are invalid to the selected model, they will be ignored.

6.1.8.3.1 Grounding

The grounding module is optional. It integrates external, contextually relevant, domain-specific, or real-time data into AI processes. This data supplements the natural language processing capabilities of pre-trained models, which are trained on general material.

Grounding is a service designed to handle data-related tasks, for example grounding and retrieval, using vector databases. It provides specialized data retrieval through vector databases, grounding the retrieval process using your own external and context relevant data. Grounding combines generative AI capabilities with the capacity to use real-time, precise data to improve decision-making and business operations, for specific business AI driven solutions.

The Pipeline API is proxied through the SAP AI Core generative AI hub, and incorporates vector stores, such as the managed HANA database. For more information, see Pipeline API in SAP AI Core.

To use the grounding module, you must first create a resource group and generic secret for grounding purposes. For more information, see Create a Resource Group for AI Data Management [page 104] and Create a Generic Secret for AI Data Management [page 106].

You can provide input variables for data retrieval, which can be used in the template definition. Variables must have unique names.

You can select a data source. If no sources are selected, all available sources will be used.

To include the grounding module in your orchestration workflow, activate it via the *Advanced* modules switch.

6.1.8.3.2 Templating

In this section, you enter your prompt to be passed to the generative AI model. You can also enter system messages to specify how the model is to answer a question. You can specify the role that the message is for, and you can provide sample queries and responses for the model.

When you enter the prompt and the system message, you can define placeholders by surrounding the text with two curly brackets and preceding the placeholder name with a question mark. For example, Create a job description for the role of **`{{?job_role1}}`**. The names of your placeholders must meet the following criteria:

- Start with a letter
- End with a letter or a number
- Do not include special characters other than an underscore (_) or a hyphen (-). Note that consecutive underscores or hyphens are not permitted.

For each placeholder, you can enter a default value that is used to test your model. If you do not specify a default value, you will be prompted to enter a value when you test the workflow.

You can add more message blocks using the + *(add)*.

For selected models, image inputs are supported, and can be added using the *(add image)* icon or copy and paste. Prompt messages are limited to 5.00mb across all inputs.

You can check which models support image input in the *Model Library*, or you can choose your model configuration through the catalog view. For more information, see Model Library [page 75] and Model Configuration [page 115].

If you include an image in your template, and then configure a model that does not support images in your model configuration, your orchestration workflow will result in an error.

You can use prompts saved in your prompt registry by clicking *Select* and choosing a template. For more information, see Prompt Registry in SAP AI Core.

6.1.8.3.3 Input Translation

The input translation module is optional. It allows you to translate LLM text prompts into a chosen target language.

The input translation module helps improve answer quality when the configured model performs better with input in a specific language, for example English.

To use input translation, configure it as part of your orchestration workflow.

The module supports SAP's Document Translation service (part of the SAP Translation Hub).

> ⚠ Caution
>
> Input translation is performed **before** any configured input masking. Any content added to the prompt as part of prompt templating is sent to the translation service unmodified.

For source language, you can choose one of the available languages, or no language is chosen, *Auto-Detect* will be used.

The target language is mandatory, choose from the available languages. The available languages may change based on your source language selection. For more information about compatible language pairs, see SAP Translation Hub Supported Languages.

> ⚠ Caution
>
> Using unsupported pairs through auto-detect or source language input errors may result in error or poor quality results.

To include the input translation module in your orchestration workflow, activate it via the *Advanced* modules switch.

6.1.8.3.4 Data Masking

The data masking module is optional and serves to anonymize or pseudonymize personally identifiable information from the input for selected entities.

There are two ways in which you can mask data:

- The *Anonymize* button replaces personally identifiable information in chosen categories with a `MASKED_ENTITY` placeholder. Anonymized data can't be unmasked because the original data isn't retained.
- The *Pseudonymize* button substitutes personally identifiable information in selected categories with a `MASKED_ENTITY_ID` placeholder. Pseudonymized data can be unmasked in the response.

You can also choose whether to apply data masking to the content provided by the Grounding module.

When masking data, you can choose the types of data that are to be masked. For example, you can mask names, genders, credit card numbers, and so on. If there are any entities that you do not want to mask, you can add them to the exception list.

> **⚠ Caution**
>
> The masking service can obscure personally identifiable information in prompts. However, since it relies on automated detection mechanisms, it can't guarantee that all such information will be identified and masked.
>
> Anonymization replaces personally identifiable information in an irreversible way. This leads to a loss of context, which may limit the model's ability to process the input. For instance, if tasked with writing a story about Michael and Donna, anonymization of profile-person would result in a story about `MASKED_PERSON` and `MASKED_PERSON`, making it impossible to distinguish between the two.

You can enter strings that you do not want to mask in the *Allow List*.

To include the data masking module in your orchestration workflow, activate it via the *Advanced* modules switch.

For more information about supported anonymization services, see the Orchestration Workflow in SAP AI Core and select *Data Masking*.

6.1.8.3.5 Input Filtering

Input filtering lets you decide the type of content that is passed to the generative AI model.

The module supports the following filters:

- **Azure Content Safety:** This service recognizes four distinct content categories: `Hate`, `Violence`, `Sexual`, and `SelfHarm`. For more information, see Harm categories in Azure AI Content Safety . Text can have more than one label (for example, a text sample can be classified as both `Hate` and `Violence`). The returned content categories include a severity level rating of 0, 2, 4, or 6. The value increases with the severity of the content. This service also includes prompt shield, which detects and mitigates prompt injection attacks. For more information, seeContent Filtering in SAP AI Core .

- **Llama Guard 3:** This service reognizes a range of categories inclduing `Hate`, `Specialized Advice`, `Defamation` and `Sexual Content`. For more information, see Llama Guard 3 . Harm categories are Boolean values.

> **Note**
>
> Some generative AI models have content filters incorporated into their standard offering. This means that content may be filtered irrespective of the settings that you configure here.

If you edit the workflow, you can hide the filter section.

To include the input filtering module in your orchestration workflow, activate it via the *Advanced* modules switch.

6.1.8.3.6 Model Configuration

In this section, you select the model that you want to use in your workflow. If you don't select a model, the default model will be used. You can also include further parameters in JSON format. For information about the supported parameters, see the documentation of the model provider.

You can see which models are available within an orchestration deployment by selecting the deployment ID.

You can adjust the parameters by using the sliders

You can add custom parameters by choosing the {·} *(code)* icon and entering additional parameters in JSON format. Custom parametersand their values must be supported by the model selected. For information about supported parameters, check the documentation from the model provider.

When you run the workflow, you will receive one response from the model. If you want to receive multiple responses, you can set a value for the **n** parameter in the model configuration.

Example

If you enter `"n":3` in the model configuration, the model will generate three different responses based on your prompt template.

Note

For models from Anthropic, the `max_tokens` parameter is mandatory and is added to *additional parameters* automatically. Users can update the parameter value, but the parameter should not be removed.

6.1.8.3.7 Output Filtering

Output filtering lets you decide the type of content that is received from the generative AI model.

The module supports the following filters:

- **Azure Content Safety:**
  - This service recognizes four distinct content categories: `Hate`, `Violence`, `Sexual`, and `SelfHarm`. For more information, see Harm categories in Azure AI Content Safety . Text can have more than one label (for example, a text sample can be classified as both `Hate` and `Violence`). The returned content categories include a severity level rating of 0, 2, 4, or 6. The value increases with the severity of the content.

- **Llama Guard 3:** This service reognizes a range of categories inclduing `Hate`, `Specialized Advice`, `Defamation` and `Sexual Content`. For more information, see Llama Guard 3 . Harm categories are Boolean values.

> **Restriction**
>
> For DeepSeek and Aleph Alpha Pharia models, output filtering is mandatory and cannot be disabled. All available content categories are selected by default, and content categories cannot be deselected individually. If all content categories for all content filters are manually deselected, all content categories for at least one filter will be selected automatically.

> **Note**
>
> Some generative AI models have content filters incorporated into their standard offering. This means that content may be filtered irrespective of the settings that you configure here.

If you edit the workflow, you can hide the filter section.

6.1.8.3.8 Output Translation

The output translation module is optional. It allows you to translate LLM output into a chosen target language.

To use output translation, configure it as part of your orchestration workflow.

The module supports SAP's Document Translation service (part of the SAP Translation Hub).

> **Caution**
>
> Output translation is performed **after** any configured output unmasking. Unmasked output is sent to the translation service.

For source language, you can choose one of the available languages, or no language is chosen, *Auto-Detect* will be used.

The target language is mandatory, choose from the available languages. The available languages may change based on your source language selection. For more information about compatible language pairs, see SAP Translation Hub Supported Languages.

> **⚠ Caution**
>
> Using unsupported pairs through auto-detect or source language input errors may result in error or poor quality results.

To include the output translation module in your orchestration workflow, activate it via the *Advanced* modules switch.

6.1.8.4 Test Your Orchestration Workflow

After you have built your orchestration workflow, you can test it to generate output from your chosen model.

Procedure

1. Change the mode from *Build* to *Test*.
2. Check the orchestration deployment and change it if necessary.
3. If you have included placeholders in the template, enter the values that you want to be passed to the model.
4. **Optional:** As the context, give the model additional instructions to further refine the output. You can also provide sample queries and responses here.
5. Choose *Run*.

Results

SAP AI Launchpad displays the response to your prompt, which you can copy to the clipboard. You can also examine the trace to see the JSON for a particular response. The JSON shows the trace for the orchestration workflow that you designed.

If you want to download the JSON, you can do so by clicking *Download*.

Next Steps

You can save your workflow as an orchestration configuration. For more information, see Save an Orchestration Config [page 121].

6.1.8.5 Orchestration Registry

6.1.8.5.1 Save an Orchestration Config

Prerequisites

- You have either the `genai_manager` or `orchestration_manager` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations [page 321].
- Users with only the `genai_experimenter` or `orchestration_experimenter` roles are not able to save orchestration configs. For more information, see Roles and Authorizations [page 321].
- You've built your orchestration workflow. For more information, see Build Your Orchestration Workflow [page 108] and Test Your Orchestration Workflow [page 120].

Context

After preparing your orchestration workflow in the *Orchestration* app, you can save it as an orchestration config for later use. You can then continue to update and refine your workflow, incorporate versioning, and manage the lifecycle of your orchestration configs.

> **Note**
>
> Orchestration configs are saved in one region only and can only be retrieved or deleted by an instance of AI launchpad in that region.

Procedure

Saving a New Orchestration Config

You can save a new config by pressing *Save*.

A wizard will open.

- Provide a name for your config.
- Choose an existing scenario from the drop down list or enter a new one.

Results

Your orchestration config is saved, and a version number has been automatically be applied.

Saving a New Orchestration Config Version

You can save a new config version by pressing the › *(Arrow)* icon on the *Save* button and selecting *Save New Version*.

A wizard will open.

You can use the automated versioning, or provide a new version number in SemVer format.

Results

Your orchestration config version is saved.

Next Steps

You can view your existing configs. For more information, see View Orchestration Configs [page 122].

You can update your existing configs. For more information, see Update an Orchestration Config [page 123].

You can delete a revision, version or config. For more information, see Delete an Orchestration Config [page 124].

6.1.8.5.2 View Orchestration Configs

Prerequisites

- You have either the `genai_manager` or `orchestration_manager` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations [page 321].
- You've built your orchestration workflow. For more information, see Build Your Orchestration Workflow [page 108] and Test Your Orchestration Workflow [page 120].

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app and choose a resource group.
   The *Generative AI Hub* app is now clickable in your side navigation panel and resource groups are listed.
2. In the *Generative AI Hub* app, choose *Orchestration*.
   Existing versions of your orchestration configs will be listed.

Next Steps

You can test your new config. For more information, see Test Your Orchestration Workflow [page 120].

You can update your config. For more information, see Update an Orchestration Config [page 123].

You can delete a revision, version or config. For more information, see Delete an Orchestration Config [page 124].

6.1.8.5.3 Update an Orchestration Config

Prerequisites

- You have either the `genai_manager` or `orchestration_manager` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations [page 321].
- Users with only the `genai_experimenter` or `orchestration_experimenter` roles are not able to save orchestration configs. For more information, see Roles and Authorizations [page 321].
- You've saved an orchestration workflow. For more information, see Save an Orchestration Config [page 121].

> **Note**
>
> Orchestration configs are saved in one region only and can only be retrieved or deleted by an instance of AI launchpad in that region.

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app and choose a resource group.
   The *Generative AI Hub* app is now clickable in your side navigation panel and resource groups are listed.
2. In the *Generative AI Hub* app, choose *Orchestration*.
   Existing versions of your orchestration configs will be listed.
3. Select the config and version that you want to update, and make your changes.

Next Steps

You can save your progress by choosing the *Save* button. This will save a new revision. Your revisions will be visible in the *Revisons* panel.

You can save a new version by pressing the arrow on the *Save* button and selecting *Save New Version*. You can use the automated version number, or enter your own in SemVer format. For more information, see Save an Orchestration Config [page 121].

You can save a new version by choosing the *Save* button and changing the config details. For more information, see Save an Orchestration Config [page 121].

You can test your new config. For more information, see Test Your Orchestration Workflow [page 120].

6.1.8.5.4 Delete an Orchestration Config

Prerequisites

- You have either the `genai_manager` or `orchestration_manager` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations [page 321].
- Users with only the `genai_experimenter` or `orchestration_experimenter` roles are not able to delete orchestration configs. For more information, see Roles and Authorizations [page 321].
- You've saved an orchestration workflow. For more information, see Save an Orchestration Config [page 121].

> **Note**
>
> Orchestration configs are saved in one region only and can only be retrieved or deleted by an instance of AI launchpad in that region.

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app and choose a resource group.
   The *Generative AI Hub* app is now clickable in your side navigation panel and resource groups are listed.
2. In the *Generative AI Hub* app, choose *Orchestration*.
   Existing versions of your orchestration configs will be listed.
3. Select the config and version that you want to delete.
   Select the -••• *(Menu)* icon and choose *Delete* from the drop down list.
   If you have multiple revisions, the current revision will be deleted.
   If you have a single revision, the current version will be deleted.
   If you have a single revision and a single version, the config will be deleted.

6.1.9 Optimizations

Optimizations lets you evaluate and refine your generative AI workflows and prompts.

Key features of optimizations include:

- **Evaluations:** Systematically assess and validate AI model performance using evaluation workflows and metrics. Manage the complete lifecycle of your evaluation processes and analyze results to ensure your AI solutions meet quality standards before deployment. For more information, see Evaluations [page 129].
- **Prompt Optimization:** Enhance prompt effectiveness by optimizing templates against target datasets to maximize specified performance metrics. Leverage prompt registry integration and systematic testing to improve model outputs and achieve better alignment with business objectives. For more information, see Prompt Optimization [page 139].
- **Metrics:** Monitor model performance using built-in and custom indicators to ensure your AI applications meet business requirements. For more information, see Metrics [page 144].

6.1.9.1 Prerequisites

6.1.9.1.1 Register an Object Store for Optimizations

As an administrator, you can add object store secrets for use within your AI processes.

Prerequisites

- You're using the `extended` service plan. For more information, see Service Plans [page 45].
- You have the `aicore_admin_objectstoresecret_editor` role or a role collection that contains it. For more information, see Roles and Authorizations.

Context

To use optimizations, you must register an object store with the name **`default`**. Optionally, you can register an additional object store with a name of your choice.

You can use the *SAP AI Core Administration* app to add secrets for multiple object stores. The object stores must already exist with valid credentials.

Supported cloud object stores include Amazon S3 (S3), Alibaba Cloud Object Storage Service (OSS), Azure, and SAP HANA Cloud, data lake (WebHDFS).

Procedure

1. In the *Workspaces* app, choose the AI API connection and resource group.
2. Open the *SAP AI Core Administration* app and choose *Object Store Secrets*.

   The *Object Store Secrets* screen appears with a tile for each existing secret.
3. Choose *Add* to enter reference details for a new secret.
4. Complete the fields in the *Add Object Store Secret* dialog box as follows:
   a. Confirm the resource group. To change the resource group, choose *(Change Value)*.
   b. Enter a name for the secret.

      Secret names must comply with the following criteria:

      - Contain only lowercase alphanumeric characters, hyphens (-), or periods (.)
      - Start with an alphanumeric character
      - End with an alphanumeric character

   c. Choose the type of object store.
   d. Enter the path prefix. The path prefix is used to differentiate between different projects that are stored in the same location.
   e. Complete the information requested in the dialog box.

      > **Note**
      >
      > The type of object store determines what fields are required for the object store secret.

      For S3:

      - Choose **`Verify SSL`** to apply the SSL security protocol to data transferred from the object store.
      - Choose **`Use HTTPS`** to apply the HTTPS communication protocol to data transferred from the object store.

   f. Enter the secret in `JSON` format. For more information, see Register Your Object Store Secret.
5. Choose *Add* to save the secret details.

Results

The new secret appears on the *Object Store Secrets* screen.

The saved secret enables read access to the nominated hyperscaler object store, enabling stored files to be used in your launchpad processes.

6.1.9.1.2 Register an Artifact for Optimizations

Prerequisites

- You have the `genai_manager` or `custom_evaluation` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations [page 321].
- You're using the `extended` service plan. For more information, see Service Plans [page 45].

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app, and choose the resource group used for your generative AI hub deployment.
2. In the side navigation, expand the *Generative AI Hub* and choose *Optimizations*.
3. Choose the *Artifacts* tab and choose *Add*.
4. Choose *Create*.
   A wizard appears to guide you through the process of uploading an artifact for optimizations.
5. Complete the fields with information about your artifact
   a. Choose a scenario
   b. Give your artifact a name
   c. **Optional:** Provide a description of your artifact
   d. Choose *Add*
6. Your can add a new artifact through the following methods:
   - **Option 1:** Upload a file from your by choosing the *Upload File* switch

     > **Restriction**
     > Available to users with `genai_manager` or `custom_evaluation` roles only.

     1. Add your object store
     2. Specify a sub path, relative to your object store
     3. Use the switch to specify whether or not your upload should replace an existing file
     4. Select your file

- **Option 2:** Use an existing URL by choosing the *Existing URL* switch

> **Note**
>
> Users without `genai_manager` or `custom_evaluation` roles do not need to choose *Existing URL*

1. Add your object store
2. Specify a sub path, relative to your object store

7. Add labels to your artifact by choosing *Add Labels*.

   You can add more labels using the + *(Add)* icon. You can delete labels using the 🗑 *(Delete)* icon.
8. Choose *Add*

Results

Your artifact is registered.

Next Steps

You can use your artifact as part of an evaluations workflow. For more information, see Create an Evaluation [page 130].

You can use your artifact as part of a prompt optimizations workflow. For more information, see Create a Prompt Optimization [page 141].

Users with the `genai_manager` or `custom_evaluation` role can update their artifact with new contents. Navigate to *Optimizations* > *Artifact* > *select your artifact* and choose *Update File* on the artifact details view.

6.1.9.2 Evaluations

Evaluations provides tools for benchmarking large language models and prompts via orchestration configurations.

Evaluations can be used for the following use cases:

- Evaluating prompt templates and models as orchestration configurations to determine the most effective combination for a use case.
- Use industry-standard predefined metrics for model and prompt comparison with your use-case-specific dataset.
- Use your own custom-defined metrics for your prompt and model evaluations.

Evaluations Overview

6.1.9.2.1 Create an Evaluation

Prerequisites

- You're using the `extended` service plan. For more information, see Service Plans [page 45].
- You have the `genai_manager` or `custom_evaluation` role, or you are assigned a role collection that contains one of these roles.
- You've prepared your evaluation files and registered them as an artifact. For more information, see Register an Artifact for Optimizations [page 127].
- You've prepared your variable mapping. For more information, see Variable Mappings in SAP AI Core.
- You have an object store with the name **`default`**. For more information, see Register an Object Store for Optimizations [page 125].
- If you're creating an evaluation with a prompt template: you've prepared a prompt template. For more information, see Prompt Experimentation [page 89] and Save a Template [page 99].
- If you're creating an evaluation with an orchestration configuration: you've prepared an orchestration configuration. For more information, see Build Your Orchestration Workflow [page 108] and Save an Orchestration Config [page 121].
- If you want to use a custom LLM-as-a-judge metric: you've created a custom metric. For more information, see Create a Custom Metric Evaluator [page 144].

Context

> → Tip
>
> Additional metrics are available for evaluation workflows that include an orchestration configuration.

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app, and choose the resource group used for your generative AI hub deployment.
2. In the side navigation, expand the *Generative AI Hub* and choose *Optimization*.
3. Choose *Create* and choose *Evaluation* if prompted.
   A wizard appears to guide you through the process of creating an evaluation job.
4. Use the switch to choose an input type.
5. Select your artifact by completing the input fields with details of your workflow and test data.
   a. For *Prompt and Model* input type: Choose a prompt template and one or more models. Optionally, you can change the orchestration deployment from the selected default.

b. For *Orchestration Configuration* input type: Choose an orchestration configuration.
c. Choose your test dataset.
d. Use the switch to specify the file type
e. Provide the subpath to your test dataset, relative to your selected test dataset artifact URL
f. **Optional:** Specify the number of test samples.

   By doing so, you limit the number of rows from the test dataset that are included in the evaluation. If you don't restrict the number of test samples, all rows in the test dataset are used.
g. Choose *Next*

6. Choose from the available metrics.

   You can refine the selection by applying the filters.

   To search for a metric by name, use the search bar.

   To view the details of a metric, see View Metric Details [page 146].

7. **Optional:** Add additional configuration such as:
   a. Adjust the number of repetitions

      Repetition count specifies the number of times the same input is submitted to the LLM to evaluate the consistency of LLM output. The minimum value is 1. The default is 1.
   b. Add variable mappings.

      Variable mappings can be added in JSON format, or through the interface in list mode.

      By default, variable names in the prompt templates are matched against the field names in the test dataset. Where variable names and field names are mismatched, mapping is required.

      For more information, see Variable Mappings in SAP AI Core.
   c. Add meaningful tags. You can add more tags using the + *(Add)* icon. You can delete tags using the 🗑 *(Delete)* icon.
   d. Choose *Review*
8. Review the details of your evaluation job, and choose *Create* to create your job.
   Your evaluation job starts. It can take some time for your results to be calculated.

Next Steps

You can check the status of the evaluation job status for your evaluation in the *Jobs* tab of the *Optimizations* app. For more information, see View Evaluations [page 132].

When your evaluation job status is complete, you can view your results. For more information, see View Evaluation Run Details [page 133].

You can reuse your configuration by navigating to it in the *Configurations* area of the *ML Ops* app and choosing *Create Execution*.

6.1.9.2.2 View Evaluations

Prerequisites

- You have the `custom_evaluation`, `genai_manager` or `genai_experimenter` role, or you are assigned a role collection that contains one of these roles.
- You have deployed an evaluation run. For more information, see Create an Evaluation [page 130].
- You're using the `extended` service plan. For more information, see Service Plans [page 45].

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app and choose the resource group that was used for your generative AI hub deployment.
2. In the side navigation, expand the *Generative AI Hub* and choose *Optimizations*.

Runs

You can filter by name or by using tags by using the *(Filter)*icon. Your input must be an exact match.

Runs related to evaluations workflows are labelled with “evaluation” in the *Task* column.

You can select an evaluation run to see its details containing meta data, metrics and tags for your evaluation.

You can view the result of your evaluation, For more information, see View Evaluation Run Details [page 133].

You can compare the metrics of multiple runs. For more information, see Compare Runs [page 136].

Artifacts

The *Artifacts* tab lists your optimization artifacts.

To see an unfiltered list, use the *Clear All Filters* button.

To see evaluation artifacts only, filter by using the `genai-evaluations` scenario

You can search for an artifact using the search bar.

You can filter the list of artifacts using the *(Filter)* icon. Text input must be an exact match.

Jobs

The *Jobs* tab lists your jobs and their details.

You can filter the list of jobs using the ⊽ *(Filter)* icon. Text input must be an exact match.

You can view the details of a job by selecting it from the list. For more information, see View Evaluation Job Details [page 135].

You can stop one or more job by using the checkboxes and choosing *Stop*.

You can delete one or more job by using the checkboxes and choosing *Delete*.

> ⓘ Note
>
> Where jobs with a mixture of states that can be stopped **and** deleted are selected, only items eligible to the chosen request will be successfully processed.

6.1.9.2.3 View Evaluation Run Details

Prerequisites

- You have the `custom_evaluation`, `genai_manager` or `genai_experimenter` role, or you are assigned a role collection that contains one of these roles.
- You have deployed an evaluation run. For more information, see Create an Evaluation [page 130].
- You're using the `extended` service plan. For more information, see Service Plans [page 45].

Context

The evaluation job will calculate and report the aggregate job statistics for each completion.

Aggregate job statistics include the following:

Statistic	Description
average_latency	The average time taken in seconds to get a completion from the orchestration service
completion_count	Number of completions evaluated
total_prompt_tokens	Sum of prompt_tokens of completions

Statistic	Description
total_completion_tokens	Sum of completion_tokens of completions

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app and choose the resource group that was used for your generative AI hub deployment.
2. In the side navigation, expand the *Generative AI Hub* and choose *Optimizations*.
3. Choose the *Runs* tab and select name of the evaluation that you want to view.

   Runs related to evaluations workflows are labelled with "evaluation" in the *Task* column.

   You can filter by name or by using tags by using the *(Filter)*icon. Your input must be an exact match.

   The *Run Details* will open, containing meta data, metrics and tags for your evaluation.

Next Steps

You can refresh the view using the *(Refresh)* icon.

You can add more categorical metrics to your view using the *(Add)* icon.

You can edit the values of existing tags using the *(Edit)* icon, and delete existing tags using the *(Delete)* icon.

You can delete an evaluation using the *Delete* button.

You can create a duplicate evaluation job by choosing the *Duplicate* button.

Related Information

Compare Runs [page 136]

6.1.9.2.4 View Evaluation Job Details

Prerequisites

- You're using the `extended` service plan. For more information, see Service Plans [page 45].
- You have the `custom_evaluation`, `genai_manager` or `genai_experimenter` role, or you are assigned a role collection that contains one of these roles.
- You have deployed an evaluation run. For more information, see Create an Evaluation [page 130].

Context

Only *running* or *pending* evaluation jobs can be stopped.

You can delete an evaluation job if it has the status *Completed*, *Dead*, *Stopped*, or *Unknown*. For evaluation jobs in other statuses, the *Delete* button is not enabled.

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app and choose the resource group that was used for your generative AI hub deployment.
2. In the side navigation, expand the *Generative AI Hub* and choose *Optimizations*.
3. Navigate to the *Jobs* tab, and select the evaluation job that you want to view or choose > *(More)*.

   You can filter by ID or attributes by using the ⏷ *(Filter)*icon. Your input must be an exact match.

   You'll see an overview of the evaluation job, including the overview steps, and tabs with more information.

> → Tip
>
> Timestamp details for each step in the evaluation job process can be seen in the header. These dates and times show when the evaluation job was created or changed. Timestamps are displayed in the user's local time zone.
>
> The *Overview* tab shows the operations involved in the evaluation job.

4. **Optional:** Choose the *Status* tab to check detailed message, status, and severity information for the evaluation job.
5. **Optional:** Choose the *Logs* tab to check or download the logs for the evaluation job.
6. **Optional:** Choose the *Associated Runs* tab to see a list of runs associated with this evaluation job.

Next Steps

You can stop an evaluation job by choosing *Stop*.

You can delete an evaluation job by choosing *Delete*.

.

6.1.9.2.5 Compare Runs

Prerequisites

- You have the `custom_evaluation`, `genai_manager` or `genai_experimenter` role, or you are assigned a role collection that contains one of these roles.
- You have deployed an evaluation run. For more information, see Create an Evaluation [page 130].
- You're using the `extended` service plan. For more information, see Service Plans [page 45].

Context

You can compare the metrics of up to five runs.

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app and choose the resource group that was used for your generative AI hub deployment.

2. In the side navigation, expand *Generative AI Hub* and choose *Optimizations*.
3. Choose the *Runs* tab.

   You can filter by name or by using tags by using the ▼ *(Filter)* icon. Your input must be an exact match.

   Alternatively, you can see runs associated with an evaluation job by choosing *Jobs* > *choose your job* > *Associated Runs*.
4. Select the runs to compare and choose *Compare*.

   The *Runs Comparison* screen appears, showing comparisons of your selected metrics.

Next Steps

You can add more comparisons to your view using the + *(Add)* icon.

You can choose a different metric using the drop down menu.

You can remove a comparison using the *(Delete)* icon.

You can enlarge a comparison using the *(Full Screen)* icon.

6.1.9.2.6 Prerequisites

6.1.9.3 Prompt Optimization

Prompt Optimization utilizes an input prompt template from the prompt registry and a JSON dataset in the object store containing desirable responses. It optimizes the input prompt to maximize a specified metric, with metrics tracked in the ML Tracking Service. The resulting optimized prompt is saved back to the prompt registry, and additional result information is stored in the object store. This optimization process is model-specific to accommodate different model behaviors and is executed as a distinct run.

> **Note**
>
> Prompt optimization executions can take from minutes to multiple hours to run, and will submit a large number of prompt requests to the target models.

> **Restriction**
>
> Mistral and DeepSeek models are not supported for prompt optimization.

6.1.9.3.1 Prerequisites

6.1.9.3.1.1 Dataset Preparation

The dataset shows the desirable outcome of an input prompt and corresponding values for placeholders in the input prompt.

Your dataset must meet the following conditions:

- It has a minimum of 25 and maximum of 200 samples
- It contains the exact placeholder names that are used in the template in the `fields` object.

> ⚠ Caution
>
> Do not include confidential of personally identifiable information in the dataset.

For example:

{·} Sample Code

```
[
    {
        "fields": {
            "input": "Subject: Urgent Assistance Required for Specialized
Cleaning Services\n\nDear ProCare
            Facility Solutions Support Team. Could you please arrange for a
specialized cleaning team to
            visit our home at the earliest convenience? We would greatly
appreciate it if this could be
            prioritized since we want to host a large party this
week.\n\nThank you for your prompt
            attention to this matter. We look forward to your swift response
and assistance.\n\nBest
            regards,\n[Sender]"
        },
        "answer": "{\"categories\": {\"routine_maintenance_requests\": false,
        \"customer_feedback_and_complaints\": false,
\"training_and_support_requests\": false,
        \"quality_and_safety_concerns\": false,
\"sustainability_and_environmental_practices\": false,
        \"cleaning_services_scheduling\": false,
\"specialized_cleaning_services\": true,
        \"emergency_repair_services\": false, \"facility_management_issues\":
false,
        \"general_inquiries\": false}, \"sentiment\": \"neutral\",
\"urgency\": \"high\"}"
    },
  {...}
]
```

6.1.9.3.2 Create a Prompt Optimization

Prerequisites

- You have the `genai_manager` or `custom_evaluation` role, or you are assigned a role collection that contains one of these roles.
- You're using the `extended` service plan. For more information, see Service Plans [page 45].
- You have an object store with the name **`default`**. For more information, see Register an Object Store for Optimizations [page 125].
- You've prepared your prompt optimization dataset and registered it as an artifact. For more information, see Register an Artifact for Optimizations [page 127].
- You've prepared a prompt template and your template is available in the prompt registry. For more information, see Save a Template [page 99].

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app, and choose the resource group used for your generative AI hub deployment.
2. In the side navigation, expand the *Generative AI Hub* and choose *Optimization*.
3. Choose *Create* and choose *Prompt Optimization* if prompted.
   A wizard appears to guide you through the process of creating a prompt optimization.
4. Select your artifact by completing the input fields with details of your workflow and test data.
   a. Choose a prompt template
   b. **Optional:** Choose an origin model to establish a baseline.
   c. Choose at least one target model.
   d. Choose a dataset artifact
   e. Provide the subpath to your test dataset, relative to your selected test dataset artifact URL
   f. Choose *Next*
5. Choose a metric from the available metrics:

   You can refine the selection by applying the filters.
6. **Optional:** Add additional configuration such as prompt template name and version and configuration of advanced settings.

   You can auto generate the name and version by choosing *Auto Generate*.
7. Review the details of your prompt optimization, and choose *Create*.
   Your prompt optimization job starts. It can take some time for your results to be calculated.

Results

You can view your best optimized prompt candidate by viewing the run details It is stored as a child node of your prompt optimization job. For more information, see View Prompt Optimizations [page 142].

6.1.9.3.3 View Prompt Optimizations

Prerequisites

- You have the `custom_evaluation`, `genai_manager` or `genai_experimenter` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations [page 321].
- You have deployed a prompt optimization. For more information, see Create a Prompt Optimization [page 141].
- You're using the `extended` service plan. For more information, see Service Plans [page 45].

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app and choose the resource group that was used for your generative AI hub deployment.
2. In the side navigation, expand the *Generative AI Hub* and choose *Optimizations*.

Runs

You can filter by name or by using tags by using the ⏷ *(Filter)*icon. Your input must be an exact match.

Runs related to optimizations workflows are labelled with “optimization” in the *Task* column.

You can select a prompt optimization run to see its details containing baseline scores, optimization scores and tags. For more information, see View Prompt Optimization Run Details [page 143].

Artifacts

The *Artifacts* tab lists your optimization artifacts.

To see an unfiltered list, use the *Clear All Filters* button.

To see prompt optimization artifacts only, filter by using the `genai-optimizations` scenario

You can search for an artifact using the search bar.

You can filter the list of artifacts using the *(Filter)* icon. Text input must be an exact match.

6.1.9.3.4 View Prompt Optimization Run Details

Prerequisites

- You have the `custom_evaluation`, `genai_manager` or `genai_experimenter` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations [page 321].
- You have deployed a prompt optimization. For more information, see Create a Prompt Optimization [page 141].
- You're using the `extended` service plan. For more information, see Service Plans [page 45].

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app and choose the resource group that was used for your generative AI hub deployment.
2. In the side navigation, expand the *Generative AI Hub* and choose *Optimizations*.
3. Choose the *Runs* tab and expand the optimization that you want to view.

   You can filter by name or by using tags by using the *(Filter)*icon. Your input must be an exact match.

   You can view the winning optimization candidate by expanding the run and choosing the child node.

   The *Run Details* will open.

   You will see references to your output artifact and details of the optimized prompt template.

Next Steps

You can refresh the view using the *(Refresh)* icon.

You can add more tags to your prompt optimization using the *(Add)* icon.

You can edit the values of existing tags using the *(Edit)* icon, and delete existing tags using the *(Delete)* icon.

You can delete a prompt optimization using the *(Delete)* button.

6.1.9.4 Metrics

Metrics are quantitative measures used to evaluate the performance of machine generative AI models within SAP AI Core. By tracking and analyzing metrics, you can assess model quality, compare different model versions, and ensure that your prompts generate responses that meet your standards and criteria.

Key features include:

- **System-Defined Metrics:** SAP AI Core offers a set of predefined evaluation methods and metrics tailored for common LLM tasks, such as text generation, classification, and summarization. System-defined metrics ensure consistency and comparability across different models and experiments.
  For more information, see System Designed Metrics in SAP AI Core.
- **Custom Metrics:** In addition to system-defined metrics, you can define and register your own custom metrics to address specific evaluation needs. This flexibility allows you to tailor performance measurement to your unique use cases and business requirements.
  For more information, see Create a Custom Metric Evaluator [page 144].
- **Integration with Evaluations:** Metrics are seamlessly integrated into the evaluation workflows. When you run an evaluation job, you can specify which metrics to compute, and the results are automatically stored and made available for analysis.
  For more information, see Evaluations [page 129].
- **Analysis:** Metric results can be compared across different model runs, making it easy to identify trends, detect regressions, and select the best-performing models.
  For more information, see Compare Runs [page 136].

6.1.9.4.1 Create a Custom Metric Evaluator

Prerequisites

- You have the `genai_manager` or `custom_evaluation` role, or you are assigned a role collection that contains one of these roles.
- You're using the `extended` service plan. For more information, see Service Plans [page 45].

Context

You can run evaluation jobs using system defined metrics. For more information, see System Defined Metrics in SAP AI Core.

You only need to define a custom metric if you want to create a custom LLM-as-a-judge metric.

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app, and choose the resource group used for your generative AI hub deployment.
2. In the side navigation, expand the *Generative AI Hub* and choose *Evaluations*.
3. Choose the *Metric Evaluators* tab and choose *Create*.
   A wizard appears to guide you through the process of creating a custom metric.
4. Complete the fields with information about your metric
   a. Choose a scenario
   b. Give your metric a name
   c. **Optional:** Provide a description of your metric
   d. Give your metric a type
   e. Give your metric an evaluation method
   f. Choose *Next*
5. Complete the fields with your metric configuration
   a. Choose at least one property
   b. Choose a model to act as a judge and configure its parameters
   c. Enter an evaluation task
   d. Enter an evaluation definition
   e. **Optional:** Enter your evaluation criteria
   f. Choose an output type by using the switch
   g. Enter at least one rating rubric

      You can add more rubrics using the + *(Add)* icon. You can delete rubrics using the 🗑 *(Delete)* icon.
   h. **Optional:** Describe any steps the judge model should follow

      You can add more steps using the + *(Add)* icon. You can delete steps using the 🗑 *(Delete)* icon.
   i. **Optional:** Set precedents by adding few shot examples

      You can add more examples using the + *(Add)* icon. You can delete examples using the 🗑 *(Delete)* icon.
   j. Choose *Review*
6. To make changes, choose previous. To continue, choose *Create*.

Next Steps

You can use your custom metric in an evaluation. For more information, see Create an Evaluation [page 130].

6.1.9.4.2 View Metric Details

Prerequisites

- You have the `custom_evaluation`, `genai_manager` or `genai_experimenter` role, or you are assigned a role collection that contains one of these roles.
- You have deployed an evaluation run. For more information, see Create an Evaluation [page 130].
- You're using the `extended` service plan. For more information, see Service Plans [page 45].

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app and choose the resource group that was used for your generative AI hub deployment.
2. In the side navigation, expand the *Generative AI Hub* and choose *Optimizations*.
3. Navigate to the *Metric Evaluators* tab, and select the metric that you want to view.

   You can refine the selection by applying the filters.

   To search for a metric by name, use the search bar.
4. Choose a metric to see its details.

   You'll see an overview of the metric, including timestamps and ID, and tabs with more information.

   You can view the prompt details using the *Definition* tab

   You can view the model parameters using the *Model Configuration* tab

Next Steps

You can update a metric by choosing the *(Edit)* icon.

You can duplicate a metric by choosing the *Duplicate* button. A wizard will open, where you can review and make changes to the configuration.

You can delete a revision by using the *(Delete)* icon. You can delete a metric by deleting all of its revisions.

6.1.10 Prompt Management

6.1.10.1 View a Saved Prompt

Prerequisites

You have either the `genai_manager` or `prompt_manager` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations [page 321].

> **Note**
>
> Prompts and templates are saved in one region only and can only be retrieved or deleted by an instance of AI launchpad in that region.

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app and choose the resource group that was used for your *Generative AI Hub* deployment.
2. In the *Workspaces* app, expand the *Generative AI Hub* and choose *Prompt Management*. Select the *Prompts* tab.

   Your prompts will be listed. Use the filters to navigate to your desired prompt.
3. **Optional:** You can
   - Mark a prompt as favorite using the star icon.
   - See the prompt details by selecting the prompt entry.
   - See version details by selecting the version entry from the prompt details.

6.1.10.2 Edit a Saved Prompt

Prerequisites

- You've selected the AI API connection and resource group that you used in the activation steps.
- You have either the `genai_manager` or `prompt_manager` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations [page 321].

> **Note**
>
> Prompts and templates are saved in one region only and can only be retrieved or deleted by an instance of AI launchpad in that region.

Procedure

1. Navigate to your desired prompt. For more information, see View a Saved Prompt [page 147].
2. Choose the entry for the version you want to edit.
3. Choose *Open in Editor*. Make your changes.

> **Note**
>
> If the model used for your prompt is no longer available you will be notified when you open the prompt in the editor and you must choose another model.

For changes to the prompt message, the *Save* button is available, and saving creates a new prompt version.

For changes to meta data only, the *Update* button is available, and updating the meta data for the same prompt version.

Meta data updates at prompt level, such as *Prompt Name* and *Collection* are updated across all versions of the prompt

Meta data updates at prompt version level, such as *Tags* and *Notes* are updated across the current version of the prompt

Next Steps

You can run your prompt again, making changes to the prompt message, parameters or model selection to change the outcome. For more information, see Prompt Experimentation [page 89].

6.1.10.3 Delete Prompts

Prerequisites

You have either the `genai_manager` or `prompt_manager` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations [page 321].

> **Note**
>
> Prompts and templates are saved in one region only and can only be retrieved or deleted by an instance of AI launchpad in that region.

Procedure

1. Navigate to *Prompt Management*. For more information, see View a Saved Prompt [page 147].
2. Choose the prompt you want to delete using the check boxes. You can choose multiple prompts by choosing multiple check boxes.
3. Choose the *Delete* icon. Alternatively, you can choose *Delete* from the *Prompt Details* view.

   > **Note**
   >
   > Deleting a prompt deletes all versions.

Delete a Prompt Version

Procedure

1. Navigate to your desired prompt. For more information, see View a Saved Prompt [page 147].
2. Switch the *Version* list to the *Select* view.
3. Choose the prompt version you want to delete using the check boxes. You can choose multiple prompt versions by choosing multiple check boxes.
4. Choose the *Delete* icon.

Alternatively, you can delete all prompt versions by choosing *Delete*.

6.1.10.4 View a Saved Template

Prerequisites

You have either the `genai_manager` or `prompt_manager` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations [page 321].

> ⓘ Note
>
> Prompts and templates are saved in one region only and can only be retrieved or deleted by an instance of AI launchpad in that region.

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app and choose the resource group that was used for your *Generative AI Hub* deployment.
2. In the *Workspaces* app, expand the *Generative AI Hub* and choose *Prompt Management*. Select the *Templates* tab.

   Your templates will be listed. Use the filters to navigate to your desired template.
3. **Optional:** You can

- See the template details by selecting the template entry.
- See revision details by selecting the revision entry from the template details.

Next Steps

Make changes to the prompt by selecting *Open in Prompt Editor*.

6.1.10.5 Edit a Saved Template

Prerequisites

- You’ve selected the AI API connection and resource group that you used in the activation steps.
- You have either the `genai_manager` or `prompt_manager` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations [page 321].

> **Note**
>
> Prompts and templates are saved in one region only and can only be retrieved or deleted by an instance of AI launchpad in that region.

Procedure

1. Navigate to your desired template. For more information, see View a Saved Template [page 151].
2. Choose the entry for the version you want to edit.
3. Choose *Open in Prompt Editor*. Make your changes.

   > **Note**
   >
   > If you change the access level of your template, you'll create a separate template item. Both templates can be updated and saved independently of each other

   > **Note**
   >
   > If the model used for your prompt is no longer available you will be notified when you open the prompt in the editor and you must choose another model.

Next Steps

You can run your prompt again, making changes to the prompt message, parameters or model selection to change the outcome. For more information, see Prompt Experimentation [page 89].

6.1.10.6 Delete a Template

Prerequisites

You have either the `genai_manager` or `prompt_manager` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations [page 321].

Context

This procedure explains how to delete templates in the generative AI hub. You can also delete templates through the *Scenarios* view in the *ML Ops* app. For more information, see View a Scenario [page 202].

> **Note**
>
> Prompts and templates are saved in one region only and can only be retrieved or deleted by an instance of AI launchpad in that region.

> **Restriction**
>
> Only imperative templates can be deleted.

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app and choose the resource group that was used for your *Generative AI Hub* deployment.
2. In the *Workspaces* app, expand *Generative AI Hub* and choose *Prompt Management*.
3. Select the *Templates* tab.
4. Choose the template that you want to delete.
   The system lists all revisions for your chosen template.
5. Choose a revision and use the 🗑 *(Delete)* icon to delete it.

   To delete a template, delete all of its revisions.

6.1.11 Administration

The *Administration* app provides the following capabilities:

- Monitoring storage consumption
- Manual user offboarding
- Creating and managing disclaimers

Storage consumption is displayed on a progress bar as a proportion of total available storage.

Other capabilities are accessed through the corresponding tabs.

Related Information

Manual User Offboarding [page 155]
Create a Disclaimer [page 156]

6.1.11.1 Manual User Offboarding

Prerequisites

- You have either the `genai_administrator` or `prompt_administrator` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations [page 321].

> **Note**
>
> - User data is saved in one region only and can only be retrieved or deleted by an instance of AI launchpad in that region.
> - In addition to manual user offboarding, prompt data can be deleted automatically. For more information, see Data Protection and Privacy [page 319].

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app.
2. In the side navigation, expand the *Generative AI Hub* and choose *Administration*.
3. Check for a user in the system by entering the user's email in the *User Data* tab bar. Choose search.

4. If the user is found, you can choose *Delete* to delete their entire data.

> **Note**
>
> The user data won't be fetched.

If the user is not found, you will be informed.

6.1.11.2 Create a Disclaimer

Prerequisites

- You have either the `genai_administrator` or `prompt_administrator` role, or you are assigned a role collection that contains one of these roles. For more information, see Roles and Authorizations [page 321].

Context

You can create a custom disclaimer message which will be shown to every user of the *Prompt Editor* and *Chat* applications, within your tenant. You can use this to implement internal standards, for example to inform users of AI ethics, or data guidelines.

Procedure

1. Select the connection to your SAP AI Core runtime in the *Workspaces* app.
2. In the side navigation, expand the *Generative AI Hub* and choose *Administration*.
3. In the *Disclaimer Message* tab bar, enter a title for your disclaimer. Optionally, you can add more detail in the *Details* field. You can save your progress using the *Save* button.

You will see a preview of your disclaimer message in the *Preview* field.

4. Publish your disclaimer message using the *Activate Disclaimer* toggle button.

If you want to remove your disclaimer message in the future, you do so by deactivating the *Activate Disclaimer* toggle button.

After publishing, your disclaimer message will show on the *Prompt Editor* and *Chat* applications.